Chương 2

Nền tảng của mạng nơ-ron

Mạng nơ-ron nhân tạo (*artificial neural networks*) (hay gọi tắt là **mạng nơ-ron** — *neural networks*, *neural nets*) là những công cụ học máy mạnh mẽ đã đẩy trình độ tiên tiến nhất (*state-of-the-art*) của NLP tiến lên trong những năm gần đây. Mặc dù lịch sử của mạng nơ-ron có thể truy ngược về thập niên 1940 [McCulloch và Pitts, 1943], trong một thời gian dài chúng không vượt trội một cách ổn định so với các phương pháp học máy khác. Điều này thay đổi vào khoảng năm 2006, khi những ý tưởng mới được phát triển để huấn luyện các **mạng nơ-ron sâu** (*deep neural networks*) [Hinton và cộng sự, 2006; Hinton, 2007]. Từ đó, các phương pháp này được biết đến với tên gọi **học sâu** (*deep learning*). Đến nay, học sâu chắc chắn đã trở thành một trong những lĩnh vực sôi động và có ảnh hưởng nhất của trí tuệ nhân tạo (*artificial intelligence, AI*); nó được hưởng lợi không chỉ từ các kiến trúc mô hình "sâu" mà còn từ vô số kỹ thuật giúp huấn luyện và triển khai các mô hình như vậy.

Trong chương này, chúng tôi trình bày các khái niệm nền tảng của mạng nơ-ron và học sâu. Dù không tập trung vào những nghiên cứu mới nhất, chương này bao quát nhiều khái niệm và kỹ thuật quan trọng được dùng rộng rãi khi xây dựng các hệ thống nơ-ron. Trong đó có các kiến trúc mô hình cơ bản, các phương pháp huấn luyện và chính quy hóa (*regularization*), các kỹ thuật học không giám sát (*unsupervised learning*) và bộ tự mã hóa (*auto-encoder*). Chúng tôi cũng trình bày một ví dụ áp dụng mạng nơ-ron vào bài toán mô hình hóa ngôn ngữ (*language modeling*).

Chú giải của người dịch: "Nơ-ron nhân tạo" và "sâu" nghĩa là gì?

Tên gọi "mạng nơ-ron" bắt nguồn từ việc mô phỏng (rất thô sơ) các tế bào thần kinh trong não. Nhưng về bản chất, một mạng nơ-ron chỉ là một *hàm số* rất lớn: nhận vào một dãy số, thực hiện nhiều phép nhân, cộng và áp các hàm đơn giản, rồi trả ra một dãy số khác. Mạng được gọi là "sâu" khi nó gồm nhiều *tầng* xử lý chồng lên nhau (đầu ra của tầng này là đầu vào của tầng kế tiếp), giống như một dây chuyền sản xuất có nhiều công đoạn nối tiếp.

2.1 Mạng nơ-ron nhiều tầng

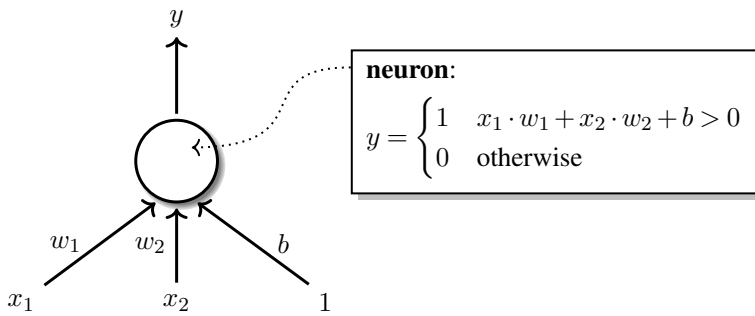
Để bắt đầu, chúng tôi giới thiệu ngắn gọn về **perceptron một tầng** (*single-layer perceptrons*), rồi mở rộng sang trường hợp tổng quát hơn, trong đó nhiều tầng được xếp chồng lên nhau để tạo thành một mạng phức tạp hơn.

2.1.1 Perceptron một tầng

Perceptron một tầng (gọi tắt là **perceptron**) có lẽ là những mạng nơ-ron đơn giản nhất đã được phát triển để sử dụng trong thực tế [Rosenblatt, 1957; Minsky và Papert, 1969]. Chúng thường được xem là các mô hình lấy cảm hứng từ sinh học, biến đổi một đầu vào nào đó thành một đầu ra nào đó. Một perceptron gồm một số **nơ-ron** (*neurons*) kết nối các biến đầu vào với các biến đầu ra. Hình 2.1 minh họa một perceptron chỉ có một nơ-ron. Trong ví dụ này, có hai biến nhận giá trị thực x_1 và x_2 làm đầu vào và một biến nhị phân (*binary variable*) y làm đầu ra. Nơ-ron đọc các biến đầu vào và quyết định chọn giá trị đầu ra nào. Quá trình này tương tự như ở một nơ-ron sinh học — nó nhận các tín hiệu điện hóa từ các nơ-ron khác và quyết định có truyền tiếp tín hiệu điện hóa đó hay không.

(Trong hình: *neuron* = nơ-ron; *otherwise* = trường hợp còn lại.)

Về mặt toán học, một perceptron có thể được mô tả như một hàm ánh xạ. Gọi $\mathbf{x}$ là vectơ các biến đầu vào (tức là một **vectơ đặc trưng** (*feature vector*)). Một **phép biến đổi affine** (*affine transformation*) của $\mathbf{x}$ được cho



Hình 2.1: Minh họa một perceptron với hai biến đầu vào $\{x_1, x_2\}$ và một biến đầu ra y. Có hai trọng số $\{w_1, w_2\}$, mỗi trọng số tương ứng với một biến đầu vào. Đầu ra phụ thuộc vào tổng có trọng số của các biến đầu vào cộng với số hạng độ lệch b, chẳng hạn $y = 1$ nếu $x_1 \cdot w_1 + x_2 \cdot w_2 + b > 0$, và $y = 0$ trong trường hợp ngược lại.

bởi[1]:

$$\begin{aligned} f(\mathbf{x}) &= \mathbf{x} \cdot \mathbf{w} + b \\ &= \sum_i x_i \cdot w_i + b, \end{aligned} \tag{2.1}$$

trong đó **w** là vectơ trọng số (*weight*) và b là số hạng độ lệch (bias) (*bias*). Khi đó, một perceptron chuẩn có thể được định nghĩa là:

$$\begin{aligned} y &= \psi(f(\mathbf{x})) \\ &= \begin{cases} 1 & f(\mathbf{x}) > 0 \\ 0 & \text{trường hợp còn lại} \end{cases}, \end{aligned} \tag{2.2}$$

trong đó $\psi(\cdot)$ là một **hàm bậc thang** (*step function*) nhị phân. Hàm $\psi(\cdot)$ còn được gọi là hàm kích hoạt (*activation function*). Điều này liên hệ perceptron với các mô hình phân loại đã bàn ở Chương 1. Nói cách khác, bản thân Phương trình (2.2) đã là một bộ phân loại (*classifier*): $f(\mathbf{x})$ là

[1] Trong toán học, một **phép biến đổi tuyến tính** (*linear transformation*) ánh xạ mỗi vectơ **v** trong một không gian thành $f(\mathbf{v})$ trong một không gian khác, sao cho với mọi vectơ **x**, **y** và mọi số vô hướng α, β, ta có $f(\alpha\mathbf{x} + \beta\mathbf{y}) = \alpha f(\mathbf{x}) + \beta f(\mathbf{y})$. Phép biến đổi affine là một phép biến đổi tuyến tính theo sau bởi một phép tịnh tiến, thường được viết dưới dạng $f(\mathbf{x}) + \mathbf{b}$.

một hàm phân biệt (*discriminant function*) xác định trên mỗi đầu vào $\mathbf{x}$, theo sau là một hàm kích hoạt $\psi(\cdot)$ dùng để tạo ra đầu ra mong muốn[2].

Chú giải của người dịch: Tính tay một perceptron

Giả sử $w_1 = 2$, $w_2 = -1$, $b = -1$. Với đầu vào $x_1 = 1$, $x_2 = 0{,}5$: $f(\mathbf{x}) = 1 \cdot 2 + 0{,}5 \cdot (-1) + (-1) = 0{,}5 > 0$, nên $y = 1$. Với $x_1 = 0$, $x_2 = 1$: $f(\mathbf{x}) = 0 - 1 - 1 = -2 \leq 0$, nên $y = 0$. Như vậy perceptron chỉ làm ba việc: nhân mỗi đầu vào với "mức độ quan trọng" của nó (trọng số), cộng lại kèm một hằng số (độ lệch), rồi so sánh với 0 để ra quyết định "có/không". Tập các điểm thỏa $2x_1 - x_2 - 1 = 0$ là một đường thẳng trong mặt phẳng (x_1, x_2); nó chia mặt phẳng thành hai nửa, mỗi nửa ứng với một quyết định — đó chính là "biên quyết định tuyến tính" nhắc tới trong chú thích chân trang. Ký hiệu $\sum_i x_i \cdot w_i$ nghĩa là "cộng tất cả các tích $x_i w_i$ với i chạy qua mọi chỉ số", và $\mathbf{x} \cdot \mathbf{w}$ chính là tích vô hướng quen thuộc ở lớp 10, chỉ là với nhiều tọa độ hơn.

Khi có từ hai nơ-ron trở lên, ta có thể nhóm các nơ-ron này thành một **tầng** (*layer*). Như minh họa trong Hình 2.2, mọi nơ-ron trong một tầng đều nhận tín hiệu từ cùng một vectơ đặc trưng đầu vào nhưng được đánh trọng số theo những cách khác nhau. Đầu ra của tầng là một vectơ đặc trưng mới, mỗi phần tử của nó tương ứng với một nơ-ron. Một cách hình thức hơn, khi xem $\psi(\cdot)$ và $f(\cdot)$ là các hàm vectơ, dạng toán học của perceptron một tầng được cho bởi các phương trình:

$$\mathbf{y} = \psi(f(\mathbf{x})), \tag{2.3}$$

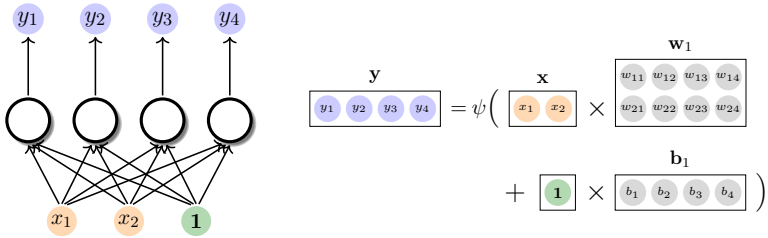
$$f(\mathbf{x}) = \mathbf{x} \cdot \mathbf{w} + \mathbf{b}, \tag{2.4}$$

trong đó $\mathbf{x} \in \mathbb{R}^m$, $\mathbf{y} \in \mathbb{R}^n$, $\mathbf{w} \in \mathbb{R}^{m \times n}$ và $\mathbf{b} \in \mathbb{R}^n$.

Chú giải của người dịch: Phép nhân vectơ với ma trận trong Hình 2.2

Ở đây $\mathbf{x} = [x_1 \;\; x_2]$ là một hàng gồm $m = 2$ số, còn $\mathbf{w}$ là một bảng số (ma trận) gồm 2 hàng, 4 cột ($\mathbf{w} \in \mathbb{R}^{2 \times 4}$). Tích $\mathbf{x} \cdot \mathbf{w}$ cho ra một hàng gồm 4 số; số thứ j là tích vô hướng của $\mathbf{x}$ với *cột* thứ j của $\mathbf{w}$: $x_1 w_{1j} + x_2 w_{2j}$. Ví dụ, với $\mathbf{x} = [1 \;\; 2]$ và $\mathbf{w} = \begin{bmatrix} 1 & 0 & 2 & -1 \\ 0 & 1 & 1 & 3 \end{bmatrix}$, ta có $\mathbf{x} \cdot \mathbf{w} = [1 \;\; 2 \;\; 4 \;\; 5]$. Như

[2] Vì biên quyết định là tuyến tính nên perceptron được xem là một bộ phân loại tuyến tính, mặc dù bản thân hàm bậc thang là phi tuyến.



Hình 2.2: Minh họa một perceptron một tầng gồm bốn nơ-ron. Tất cả các nơ-ron này đều nhận thông tin từ các biến đầu vào $\{x_1, x_2\}$. Perceptron mô tả một quá trình trong đó 1) trước hết ta biến đổi vectơ các biến đầu vào bằng một phép biến đổi affine $f(\mathbf{x}) = \mathbf{x} \cdot \mathbf{w} + \mathbf{b}$; và 2) tính đầu ra bằng cách đưa $f(\mathbf{x})$ vào hàm kích hoạt $\psi(\cdot)$.

vậy, mỗi cột của $\mathbf{w}$ chính là bộ trọng số của *một* nơ-ron, và phép nhân ma trận cho phép tính đồng thời cả 4 nơ-ron. Ký hiệu $\mathbb{R}^{m \times n}$ chỉ tập hợp các ma trận số thực có m hàng, n cột; $\mathbb{R}^n$ là tập các vectơ có n số thực. Trong hình, số 1 màu xanh lá đi kèm $\mathbf{b}_1$ thể hiện "mẹo" coi độ lệch như trọng số của một đầu vào luôn bằng 1.

Một lưu ý nữa về hàm kích hoạt: hàm bậc thang, dù được dùng rộng rãi, không phải là dạng duy nhất của hàm kích hoạt. Có nhiều cách khác nhau để thực hiện phép kích hoạt. Ví dụ, ta có thể dùng hàm Softmax nếu muốn đầu ra có dạng giống một phân phối xác suất (*probability distribution*); dùng hàm Sigmoid nếu muốn đầu ra đơn điệu, liên tục và dễ tối ưu; dùng hàm ReLU nếu muốn đầu ra có dạng "đường dốc". Bảng 2.1 liệt kê một số hàm kích hoạt thường dùng. Lưu ý rằng, mặc dù một tầng nơ-ron được trang bị các hàm kích hoạt này có thể được gọi một cách không chặt chẽ là perceptron một tầng, nó có thể được xếp vào một khái niệm tổng quát hơn, gọi là **mạng nơ-ron một tầng** (*single-layer neural network*). Nếu không nói gì khác, chúng tôi sẽ dùng thuật ngữ *mạng nơ-ron một tầng* trong suốt cuốn sách này.

Chú giải của người dịch: Đọc Bảng 2.1 bằng ví dụ số

$\exp(s)$ nghĩa là e^s với $e \approx 2{,}718$. Lấy $\mathbf{s} = [2 \;\; 0 \;\; -1]$:

- ReLU giữ số dương, biến số âm thành 0: $[2 \;\; 0 \;\; 0]$.

Tên	Công thức (cho phần tử thứ i của vectơ)
Đồng nhất (*Identity*)	$y_i = s_i$
Bậc thang nhị phân (*Binary Step*)	$y_i = \begin{cases} 1 & s_i > 0 \\ 0 & s_i \leq 0 \end{cases}$
Tang hyperbolic (*Hyperbolic Tangent*)	$y_i = \dfrac{\exp(s_i) - \exp(-s_i)}{\exp(s_i) + \exp(-s_i)}$
Tang cứng (*Hard Tangent*)	$y_i = \begin{cases} 1 & s_i > 1 \\ s_i & -1 \leq s_i \leq 1 \\ -1 & s_i < -1 \end{cases}$
Sigmoid (Logistic)	$y_i = \dfrac{1}{1 + \exp(-s_i)}$
ReLU (*Rectified Linear Unit*)	$y_i = \begin{cases} s_i & s_i > 0 \\ 0 & s_i \leq 0 \end{cases}$
Softplus	$y_i = \ln(1 + \exp(s_i))$
Gauss (*Gaussian*)	$y_i = \exp\Big(-\dfrac{1}{2} \cdot \dfrac{(s_i - \mu_i)^2}{\sigma_i^2}\Big)$
Softmax	$y_i = \dfrac{\exp(s_i)}{\sum_{i'=1}^{n} \exp(s_{i'})}$
Maxout	$y_i = \max(s_1, \ldots, s_n)$

Bảng 2.1: Các hàm kích hoạt ($\mathbf{y} = \psi(\mathbf{s})$, trong đó $\mathbf{s}, \mathbf{y} \in \mathbb{R}^n$). Tất cả các hàm này đều là hàm vectơ. Chúng tôi trình bày công thức cho phần tử thứ i của vectơ đầu vào và đầu ra. μ_i và σ_i^2 lần lượt là giá trị trung bình và phương sai.

- Sigmoid "nén" mọi số vào khoảng $(0;1)$: $\frac{1}{1+e^{-2}} \approx 0{,}88$, $\frac{1}{1+e^{0}} = 0{,}5$, $\frac{1}{1+e^{1}} \approx 0{,}27$.
- Tang hyperbolic (tanh) giống Sigmoid nhưng nén vào khoảng $(-1;1)$: $\approx [0{,}96 \;\; 0 \;\; -0{,}76]$.
- Softmax biến cả vectơ thành các số dương có tổng bằng 1 (giống một phân phối xác suất): $e^{2} \approx 7{,}39$, $e^{0} = 1$, $e^{-1} \approx 0{,}37$, tổng $\approx 8{,}76$, nên kết quả $\approx [0{,}84 \;\; 0{,}11 \;\; 0{,}04]$. Số lớn nhất nhận "phần" lớn nhất.

Khác với các hàm còn lại (tính riêng từng phần tử), Softmax và Maxout cần nhìn *cả* vectơ để tính mỗi phần tử.

2.1.2 Xếp chồng nhiều tầng

Bước tiếp theo hiển nhiên là tạo ra một mạng nơ-ron gồm nhiều tầng. Để làm điều này, ta chỉ cần xếp chồng nhiều tầng để tạo thành một **mạng nơ-ron nhiều tầng** (*multi-layer neural network*). Hình 2.3 minh họa một ví dụ. Trong kiến trúc này, đầu ra của mỗi nơ-ron trong một tầng được nối tới tất cả các nơ-ron ở tầng kế tiếp. Vì vậy mạng này là **kết nối đầy đủ** (*fully connected*). Về bản chất, một mạng nơ-ron nhiều tầng mô tả một phép hợp các hàm. Ví dụ, ta có thể biểu diễn mạng nơ-ron trong Hình 2.3 như một hàm phức tạp được tạo thành bằng cách hợp nhiều hàm đơn giản hơn:

$$\mathbf{y} = \text{Softmax}(\text{Sigmoid}(\text{ReLU}(\mathbf{x} \cdot \mathbf{w}_1) \cdot \mathbf{w}_2) \cdot \mathbf{w}_3 + \mathbf{b}_3), \tag{2.5}$$

trong đó $\mathbf{w}_1 \in \mathbb{R}^{3\times4}$, $\mathbf{w}_2 \in \mathbb{R}^{4\times3}$, $\mathbf{w}_3 \in \mathbb{R}^{3\times3}$ và $\mathbf{b}_3 \in \mathbb{R}^3$ là các tham số (*parameters*).

Chú giải của người dịch: Hàm hợp — đọc Phương trình (2.5) từ trong ra ngoài

Ở THPT, ta đã gặp hàm hợp như $h(x) = \sin(x^2)$: tính x^2 trước, rồi lấy sin. Phương trình (2.5) cũng vậy, chỉ là nhiều lớp hơn: (1) nhân $\mathbf{x}$ (3 số) với $\mathbf{w}_1$ được 4 số, áp ReLU; (2) nhân kết quả với $\mathbf{w}_2$ được 3 số, áp Sigmoid; (3) nhân với $\mathbf{w}_3$, cộng $\mathbf{b}_3$, áp Softmax được 3 số cuối cùng. Kích thước các ma trận phải "khớp nhau": một hàng 3 số chỉ nhân được với ma trận có 3 hàng, và số cột của ma trận quyết định số phần tử của kết quả.

Thông thường, **độ sâu** (*depth*) của một mạng nơ-ron (đôi khi gọi là

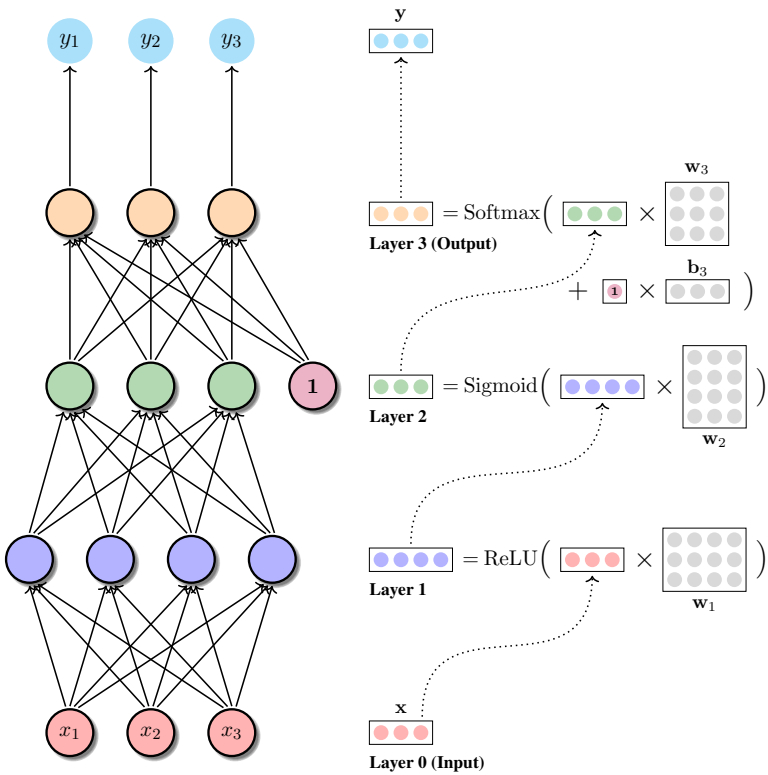
độ sâu mô hình (*model depth*)) được đo bằng số tầng của nó. Chẳng hạn, nếu coi vectơ đầu vào là tầng khởi đầu thì độ sâu của mạng ví dụ trong Hình 2.3 là 4. Một khái niệm liên quan là **độ rộng mô hình** (*model width*), được định nghĩa cho từng tầng chứ không phải cho toàn mạng. Một thước đo phổ biến cho độ rộng của một tầng là số nơ-ron mà tầng đó chứa. Ví dụ, độ rộng của tầng đầu ra trong Hình 2.3 là 3. Nếu mọi tầng của một mạng nơ-ron có cùng độ rộng n, ta có thể nói đơn giản rằng độ rộng của toàn mô hình là n. Cả độ sâu lẫn độ rộng mô hình đều có ảnh hưởng quan trọng tới năng lực biểu diễn (*representational capacity*) của mạng nơ-ron thu được. Ví dụ, người ta đã chứng minh bằng toán học rằng ngay cả một mạng nơ-ron chỉ có hai tầng nơ-ron với hàm kích hoạt Sigmoid cũng có thể xấp xỉ mọi hàm liên tục [Cybenko, 1989]. Tuy nhiên, với các bài toán phức tạp hơn, tăng độ sâu của mạng nhìn chung mang lại cải thiện đáng kể về hiệu năng.

Chú giải của người dịch: "Xấp xỉ mọi hàm liên tục" nghĩa là gì?

Kết quả của Cybenko (thường gọi là *định lý xấp xỉ phổ quát*) nói rằng: cho trước một hàm liên tục bất kỳ (trên một miền giới hạn) và một sai số cho phép nhỏ tùy ý, luôn tồn tại một mạng hai tầng với *đủ nhiều* nơ-ron mà đầu ra của nó sai khác hàm đó không quá sai số cho phép. Trực giác: mỗi nơ-ron Sigmoid tạo ra một "bậc thang mềm"; ghép đủ nhiều bậc thang nhỏ, ta có thể dựng lại hình dạng của bất kỳ đồ thị liên tục nào, giống như xếp các khối gạch nhỏ để mô phỏng một con dốc. Định lý chỉ khẳng định *tồn tại* mạng như vậy; nó không nói mạng cần bao nhiêu nơ-ron hay làm sao tìm được trọng số — vì thế trong thực tế mạng sâu thường hiệu quả hơn.

Việc xếp chồng các tầng tạo ra một loại mạng nơ-ron rất phổ biến — **mạng nơ-ron truyền thẳng** (*feed-forward neural networks, FFNNs*). Các mạng này được gọi là "truyền thẳng" vì không có chu trình trong các kết nối giữa các tầng và mọi dữ liệu đều di chuyển theo một chiều. Như ta sẽ thấy trong suốt cuốn sách, nhiều mạng nơ-ron hiện đại về bản chất là mạng truyền thẳng, dù một vài ngoại lệ đáng chú ý sẽ được thảo luận ở Mục 2.3.

(Trong hình: *Layer 0 (Input)* = Tầng 0 (đầu vào); *Layer 1, 2* = Tầng 1, 2; *Layer 3 (Output)* = Tầng 3 (đầu ra).)



Hình 2.3: Minh họa một mạng nơ-ron nhiều tầng. Tầng đầu vào gồm ba biến $\{x_1, x_2, x_3\}$. Các biến này được kết nối đầy đủ tới mọi nơ-ron của tầng 1. Đầu ra của tầng 1 là một vectơ mới $\mathbf{h}_1 = \text{ReLU}(\mathbf{x} \cdot \mathbf{w}_1)$. Vectơ này lại được kết nối đầy đủ tới tầng 2, thực hiện ánh xạ $\mathbf{h}_2 = \text{Sigmoid}(\mathbf{h}_1 \cdot \mathbf{w}_2)$. Đầu ra $\mathbf{h}_2$ của nó được đưa vào tầng 3, tầng này tạo ra đầu ra cuối cùng $\mathbf{y} = \text{Softmax}(\mathbf{h}_2 \cdot \mathbf{w}_3 + \mathbf{b}_3)$. Các tham số của mạng nơ-ron này là $\mathbf{w}_1$, $\mathbf{w}_2$, $\mathbf{w}_3$ và $\mathbf{b}_3$.

2.1.3 Đồ thị tính toán

Đồ thị tính toán (*computation graphs*) là một cách phổ biến để biểu diễn mạng nơ-ron. Giống như đồ thị trong toán học, một đồ thị tính toán được tạo thành từ các nút (*nodes*) và các cạnh (*edges*) nối giữa các nút. Mỗi nút biểu diễn hoặc một phép toán, hoặc một biến; mỗi cạnh biểu diễn luồng dữ liệu đi từ nút này sang nút khác. Do đó, đồ thị tính toán là đồ thị có hướng (*directed*)[3]. Chẳng hạn, xét ba hàm sau:

$$\mathbf{y} = \mathbf{x} + \mathbf{w}, \tag{2.6}$$

$$\mathbf{y} = \text{Softmax}(\mathbf{x} \cdot \mathbf{w} + \mathbf{b}), \tag{2.7}$$

$$\mathbf{y} = \text{Sigmoid}(\mathbf{x} \cdot \mathbf{w}_1 + \mathbf{b}_1) - \text{ReLU}(\mathbf{x} \cdot \mathbf{w}_2). \tag{2.8}$$

Hình 2.4 cho thấy đồ thị tính toán của các hàm này. Từ góc độ phân tích cú pháp (*parsing*), mọi mạng nơ-ron đều có thể được xem là các biểu thức toán học. Vì vậy, đồ thị tính toán chính là biểu diễn của kết quả thu được khi phân tích cú pháp một biểu thức toán học. Theo cách này, mỗi nút của đồ thị cho ra một biểu thức con, còn nút gốc (*root node*) cho ra toàn bộ biểu thức.

Trong một đồ thị tính toán, một nút có thể được nối với nhiều nút ở bên dưới và/hoặc bên trên nó. Điều này cho phép tái sử dụng các đồ thị con khi biểu diễn những hàm phức tạp. Ví dụ, trong Phương trình (2.8), biến $\mathbf{x}$ được dùng hai lần và nút tương ứng có hai cạnh đi ra. Thực ra, việc tổ chức mạng nơ-ron thành đồ thị tính toán phản ánh bản chất *hợp thành* của mạng nơ-ron — thông thường, một mạng lớn được xây dựng bằng cách ghép các mạng nhỏ. Lấy Phương trình (2.8) làm ví dụ. Nó có thể được viết lại thành một hệ ba phương trình:

$$\mathbf{y} = \mathbf{h}_1 - \mathbf{h}_2, \tag{2.9}$$

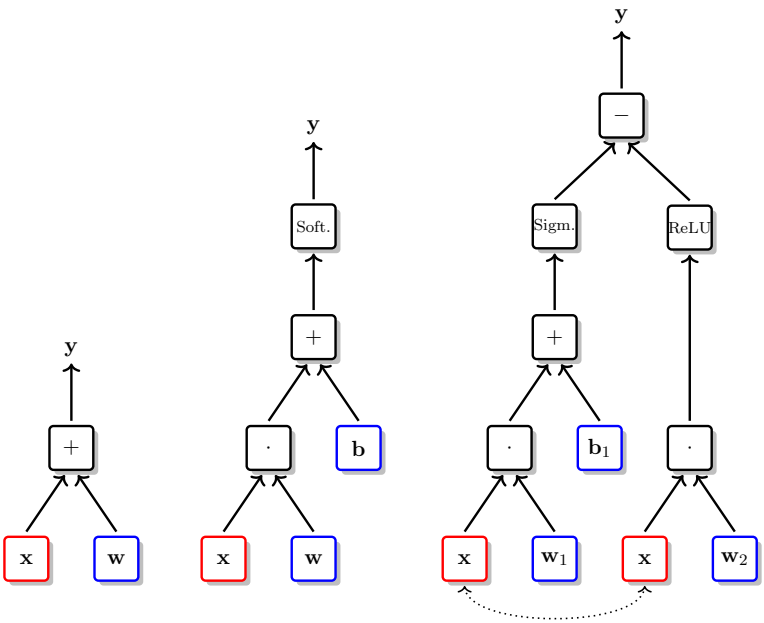
$$\mathbf{h}_1 = \text{Sigmoid}(\mathbf{x} \cdot \mathbf{w}_1 + \mathbf{b}_1), \tag{2.10}$$

$$\mathbf{h}_2 = \text{ReLU}(\mathbf{x} \cdot \mathbf{w}_2). \tag{2.11}$$

Trong phép hợp, các nút $\mathbf{h}_1$ và $\mathbf{h}_2$ trong Phương trình (2.9) được thay thế bởi đồ thị của các Phương trình (2.10–2.11).

(Trong hình: *Soft.* = Softmax; *Sigm.* = Sigmoid; mũi tên chấm chấm ở (c) cho biết hai ô $\mathbf{x}$ thực chất là cùng một nút.)

[3] Mặc dù một số mô hình học máy có thể được biểu diễn bằng đồ thị tính toán vô hướng, chúng không phải là trọng tâm của cuốn sách này.



(a) $\mathbf{y} = \mathbf{x} + \mathbf{w}$ (b) $\mathbf{y} = \text{Softmax}(\mathbf{x} \cdot \mathbf{w} + \mathbf{b})$ (c) $\mathbf{y} = \text{Sigmoid}(\mathbf{x} \cdot \mathbf{w}_1 + \mathbf{b}_1) - \text{ReLU}(\mathbf{x} \cdot \mathbf{w}_2)$

Hình 2.4: Đồ thị tính toán của ba mạng nơ-ron ví dụ. Các hộp màu đen biểu diễn các phép toán, còn các hộp có màu biểu diễn các biến. Một nút phép toán có các cạnh đi vào từ các nút khác, và mỗi nút đó có thể được coi là một đối số của phép toán. Ví dụ, trong hình con (a), nút phép cộng có hai nút con được gán nhãn lần lượt là **x** và **w**. Nút này đọc đầu ra của các nút **x** và **w**, rồi sinh ra đầu ra $\mathbf{y} = \mathbf{x} + \mathbf{w}$. Các đồ thị lớn hơn thể hiện những luồng dữ liệu phức tạp hơn. Trong đồ thị con (b), đầu ra của nút phép nhân chấm (tức $\mathbf{x} \cdot \mathbf{w}$) được truyền dọc theo cạnh tới nút phép cộng. Sau đó, nút phép cộng tính tổng của $\mathbf{x} \cdot \mathbf{w}$ và **b** làm đầu ra của nó. Ta có thể lặp lại cùng quá trình này trên tất cả các nút phép toán theo thứ tự từ dưới lên, và thu được kết quả cuối cùng của việc tính toàn bộ biểu thức tại nút trên cùng.

Công dụng chính của đồ thị tính toán là để thực thi hàm số. Việc này hoàn toàn giống với việc dự đoán đầu ra của một mạng nơ-ron. Phương pháp khá đơn giản. Trước hết, các nút của đồ thị được sắp xếp tô-pô (*topologically sorted*), tức là được đặt theo một thứ tự nhất quán với luồng thông tin. Sau đó, với các giá trị được đưa vào các nút đầu vào, ta duyệt đồ thị sao cho tính được đầu ra của từng nút rồi chuyển tiếp nó tới các nút cha. Kết quả cuối cùng nhận được tại nút đầu ra. Thủ tục này thường được gọi là **lượt truyền xuôi** (*forward pass*). Lượt truyền xuôi có thể hiệu quả, vì mỗi nút chỉ cần được duyệt một lần và đầu ra của nó có thể được nhiều nút dùng lại mà không cần tính lại. Hơn nữa, lượt truyền xuôi có thể được tối ưu bằng cách tái cấu trúc đồ thị. Cách này có thể phát triển thêm ý tưởng tái sử dụng và tránh các phép tính cũng như mức tiêu thụ bộ nhớ không cần thiết.

Chú giải của người dịch: Lượt truyền xuôi — ví dụ bằng số

Xét Hình 2.4(a) với số thực đơn giản: $x = 3$, $w = 2$. Nút "+" đọc hai nút con và cho $y = 5$. Ở Hình 2.4(c), giả sử mọi thứ là số: $x = 1$, $w_1 = 2$, $b_1 = -2$, $w_2 = -1$. Đi từ dưới lên: $x \cdot w_1 = 2$; $2 + b_1 = 0$; $\text{Sigmoid}(0) = 0{,}5$. Nhánh phải: $x \cdot w_2 = -1$; $\text{ReLU}(-1) = 0$. Nút trên cùng: $y = 0{,}5 - 0 = 0{,}5$. "Sắp xếp tô-pô" chỉ đơn giản là sắp thứ tự tính sao cho *mọi nút được tính sau các nút mà nó cần*, giống như khi tính một biểu thức có ngoặc, ta luôn tính trong ngoặc trước. Nút x được dùng cho cả hai nhánh nhưng chỉ cần đọc một lần.

Một công dụng khác của đồ thị tính toán là tính gradient (*gradient*) một cách tự động. Khi huấn luyện mạng nơ-ron, nói chung ta cần tính các đạo hàm riêng (*partial derivatives*) của hàm mất mát (*loss function*) $\mathcal{L}$ theo mọi ma trận trọng số ($\mathbf{w}$) và mọi số hạng độ lệch ($\mathbf{b}$), chẳng hạn $\frac{\partial \mathcal{L}}{\partial \mathbf{w}}$ và $\frac{\partial \mathcal{L}}{\partial \mathbf{b}}$. Tuy nhiên, trước khi xem các đạo hàm riêng này được dùng như thế nào để cập nhật mô hình (xem Mục 2.4.1), chúng ta hãy tìm hiểu cách tính đạo hàm trong một đồ thị tính toán. Ví dụ, xét hàm dưới đây:

$$\mathbf{y} = \psi(\mathbf{x} \cdot \mathbf{w}_1 + \mathbf{b}_1) \cdot \mathbf{w}_2. \tag{2.12}$$

Chú giải của người dịch: Hàm mất mát, đạo hàm riêng và gradient

Hàm mất mát $\mathcal{L}$ là một con số đo xem đầu ra của mạng "sai" bao nhiêu so với đáp án đúng; huấn luyện là tìm cách chỉnh trọng số để con số này nhỏ đi. $\mathcal{L}$ phụ thuộc vào *rất nhiều* biến (mọi phần tử của $\mathbf{w}$, $\mathbf{b}$, …). **Đạo hàm riêng** $\frac{\partial \mathcal{L}}{\partial w}$ là đạo hàm của $\mathcal{L}$ theo *một* biến w, khi coi mọi biến khác là hằng số — cách tính y hệt đạo hàm một biến ở lớp 11–12. Ví dụ, với $\mathcal{L} = w^2 + 3wb$, ta có $\frac{\partial \mathcal{L}}{\partial w} = 2w + 3b$ và $\frac{\partial \mathcal{L}}{\partial b} = 3w$. Ký hiệu ∂ (đọc là "đạo hàm riêng") thay cho chữ d để nhắc rằng còn các biến khác. Khi $\mathbf{w}$ là một ma trận, $\frac{\partial \mathcal{L}}{\partial \mathbf{w}}$ là ma trận cùng kích thước gồm các đạo hàm riêng theo từng phần tử; tập hợp tất cả các đạo hàm riêng này gọi là **gradient**. Nó cho biết nếu nhích nhẹ từng trọng số thì mất mát tăng hay giảm, và nhanh tới mức nào.

Để tính $\frac{\partial \mathcal{L}}{\partial \mathbf{w}_1}$, $\frac{\partial \mathcal{L}}{\partial \mathbf{b}_1}$ và $\frac{\partial \mathcal{L}}{\partial \mathbf{w}_2}$, cách tự nhiên là dùng quy tắc chuỗi của phép lấy đạo hàm (*chain rule of differentiation*). Ví dụ, với một hàm hợp $y = p(q(x))$, công thức của quy tắc chuỗi là:

$$\frac{\partial y}{\partial x} = \frac{\partial p}{\partial q} \cdot \frac{\partial q}{\partial x}. \tag{2.13}$$

Tuy nhiên, công thức giải tích của đạo hàm dựa trên Phương trình (2.13) sẽ dẫn tới một biểu thức rất dài. Thay vào đó, ta có thể phân rã một hàm phức tạp thành nhiều hàm, mỗi hàm đại diện cho một phép toán. Khi đó, Phương trình (2.12) có thể được viết lại thành:

$$\mathbf{y} = \mathbf{h}_1 \cdot \mathbf{w}_2, \tag{2.14}$$

$$\mathbf{h}_1 = \psi(\mathbf{h}_2), \tag{2.15}$$

$$\mathbf{h}_2 = \mathbf{h}_3 + \mathbf{b}_1, \tag{2.16}$$

$$\mathbf{h}_3 = \mathbf{x} \cdot \mathbf{w}_1. \tag{2.17}$$

Tất cả các biến này có thể được hiểu một cách trực quan hơn qua đồ thị tính toán: mỗi biến là một nút của đồ thị, và các nút được nối với nhau bằng các phép toán đại số và phép hợp hàm. Dùng Phương trình (2.13) và một số kiến thức cơ bản về giải tích, ta tính đạo hàm của các biến như sau:

nút 1: $$\frac{\partial \mathcal{L}}{\partial \mathbf{y}} = \delta_{\mathbf{y}}, \tag{2.18}$$

nút 2: $$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_1} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \cdot \mathbf{w}_2^\top, \tag{2.19}$$

nút 3: $$\frac{\partial \mathcal{L}}{\partial \mathbf{w}_2} = \mathbf{h}_1^\top \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{y}}, \tag{2.20}$$

nút 4: $$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_2} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_1} \odot \psi'(\mathbf{h}_2), \tag{2.21}$$

nút 5: $$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_3} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_2}, \tag{2.22}$$

nút 6: $$\frac{\partial \mathcal{L}}{\partial \mathbf{b}_1} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_2}, \tag{2.23}$$

nút 7: $$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_3} \cdot \mathbf{w}_1^\top, \tag{2.24}$$

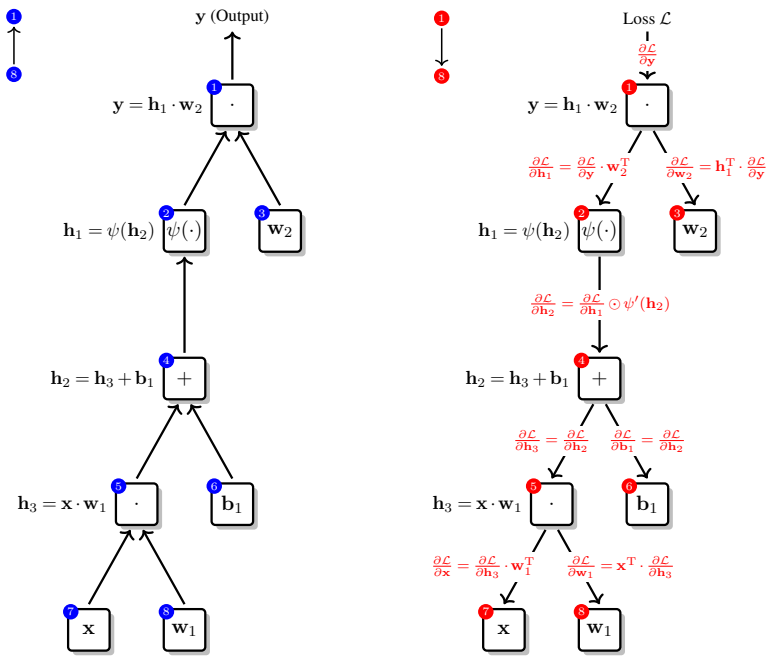
nút 8: $$\frac{\partial \mathcal{L}}{\partial \mathbf{w}_1} = \mathbf{x}^\top \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{h}_3}, \tag{2.25}$$

trong đó $\delta_\mathbf{y}$ là đạo hàm của hàm mất mát theo đầu ra của mô hình. $\delta_\mathbf{y}$ phụ thuộc vào cách chọn hàm mất mát; ví dụ, nếu dùng hàm mất mát bình phương (*squared loss*) $\mathcal{L} = \frac{1}{2}(\mathbf{y} - \mathbf{y}_{\text{gold}})^2$, trong đó $\mathbf{y}_{\text{gold}}$ là đáp án đúng (*ground-truth*), thì $\delta_\mathbf{y} = \mathbf{y} - \mathbf{y}_{\text{gold}}$. Quá trình trên về bản chất là một **lượt truyền ngược** (*backward pass*), vì các gradient được truyền theo hướng từ trên xuống dưới. Một tên gọi khác của nó là **lan truyền ngược** (*backpropagation*). Đây đã trở thành tiêu chuẩn trên thực tế cho việc huấn luyện các mạng nơ-ron sâu. Để hiểu rõ hơn cách hoạt động của lượt truyền xuôi và lượt truyền ngược, Hình 2.5 trình bày hai ví dụ minh họa từng bước.

Chú giải của người dịch: Quy tắc chuỗi và lan truyền ngược với số vô hướng

Quy tắc chuỗi (2.13) chính là công thức đạo hàm hàm hợp ở lớp 12: $(p(q(x)))' = p'(q) \cdot q'(x)$. Để thấy các Phương trình (2.18–2.25) hoạt động, hãy coi mọi thứ là số (khi đó dấu chuyển vị $^\top$ không có tác dụng, còn $\odot$ là phép nhân thường). Chọn $\psi = \text{ReLU}$, $x = 1$, $w_1 = 2$, $b_1 = 1$, $w_2 = 3$, đáp án $y_{\text{gold}} = 5$. *Truyền xuôi:* $h_3 = 2$, $h_2 = 3$, $h_1 = \text{ReLU}(3) = 3$, $y = 9$. *Truyền ngược:* $\delta_y = y - y_{\text{gold}} = 4$; $\frac{\partial \mathcal{L}}{\partial h_1} = 4 \cdot 3 = 12$; $\frac{\partial \mathcal{L}}{\partial w_2} = h_1 \cdot 4 = 12$; vì $h_2 = 3 > 0$ nên $\psi'(h_2) = 1$, do đó $\frac{\partial \mathcal{L}}{\partial h_2} = 12$; $\frac{\partial \mathcal{L}}{\partial b_1} = \frac{\partial \mathcal{L}}{\partial h_3} = 12$; $\frac{\partial \mathcal{L}}{\partial w_1} = x \cdot 12 = 12$. Kiểm tra trực tiếp: $\mathcal{L} = \frac{1}{2}\big((xw_1 + b_1)w_2 - 5\big)^2$, đạo hàm theo w_2 là $\big((xw_1+b_1)w_2-5\big)(xw_1+b_1) = 4\cdot 3 = 12$. Khớp! Ý tưởng then chốt: mỗi nút chỉ cần nhân "gradient nhận từ phía trên" với đạo hàm cục bộ của chính nó rồi chuyển xuống dưới; nhờ đó không phải viết ra một công

thức đạo hàm khổng lồ. Ký hiệu $\odot$ là nhân theo từng phần tử (*element-wise product*): $[1 \;\; 2] \odot [3 \;\; 4] = [3 \;\; 8]$; $\mathbf{w}^\top$ là chuyển vị (*transpose*) của ma trận (đổi hàng thành cột).



(a) Forward Pass (b) Backward Pass

Hình 2.5: Minh họa lượt truyền xuôi và lượt truyền ngược trên một đồ thị tính toán ví dụ. Trong lượt truyền xuôi (bên trái), các nút được duyệt theo thứ tự từ đầu vào tới đầu ra, ví dụ từ nút 8 tới nút 1. Tại mỗi nút, ta thực hiện hàm tương ứng, chẳng hạn phép cộng, để sinh ra đầu ra; đầu ra này sau đó được các nút tiếp theo sử dụng. Ngược lại, trong lượt truyền ngược (bên phải), các nút được duyệt theo thứ tự đảo ngược, ví dụ từ nút 1 tới nút 8. Trong quá trình này, ta truyền gradient của hàm mất mát (hay lỗi) từ đầu ra về đầu vào; nghĩa là, với mỗi nút, ta tính gradient tại điểm đầu vào của nút bằng quy tắc chuỗi, khi đã biết gradient tại điểm đầu ra của nút.

(Trong hình: *Forward Pass* = lượt truyền xuôi; *Backward Pass* = lượt truyền ngược; *Output* = đầu ra; *Loss* = mất mát. Các số trong chấm tròn là số thứ tự nút trong

Phương trình (2.18–2.25).)

2.2 Ví dụ: mô hình hóa ngôn ngữ bằng mạng nơ-ron

Mô hình hóa ngôn ngữ là một bài toán NLP nền tảng, nhằm ước lượng một phân phối xác suất trên các chuỗi từ. Cho một chuỗi gồm m từ $w_1 \cdots w_m$, xác suất đồng thời (*joint probability*) $\Pr(w_1, \ldots, w_m)$ được cho bởi phương trình:

$$\Pr(w_1, \ldots, w_m) = \prod_{i=1}^{m} \Pr(w_i | w_1, \ldots, w_{i-1}). \tag{2.26}$$

Như vậy, bài toán mô hình hóa ngôn ngữ được phát biểu thành bài toán dự đoán từ tiếp theo khi biết tất cả các từ ngữ cảnh đứng trước. Một cách đơn giản để mô hình hóa $\Pr(w_i | w_1, \ldots, w_{i-1})$ là cho dự đoán chỉ phụ thuộc vào một cửa sổ ngữ cảnh (*context window*) bao gồm một số cố định các từ đứng trước, như sau:

$$\Pr(w_i | w_1, \ldots, w_{i-1}) \approx \Pr(w_i | w_{i-n+1}, \ldots, w_{i-1}), \tag{2.27}$$

trong đó n là kích thước cửa sổ. Một cách để ước lượng xác suất này là phương pháp **mô hình ngôn ngữ n-gram** (*n-gram language modeling*): ta tính tần suất tương đối (*relative frequency*) của mỗi n-gram $w_{i-n+1} \cdots w_i$, tức là $\Pr(w_i | w_{i-n+1}, \ldots, w_{i-1}) = \frac{\text{count}(w_{i-n+1} \ldots w_i)}{\text{count}(w_{i-n+1} \ldots w_{i-1})}$. Mặc dù mô hình ngôn ngữ n-gram đã thống trị lĩnh vực NLP trong nhiều thập kỷ, chúng đòi hỏi những bảng khổng lồ để lưu mọi xác suất n-gram. Hệ quả là, khi kho ngữ liệu huấn luyện lớn dần và ngữ cảnh được xét dài hơn, dữ liệu trở nên cực kỳ thưa. Đây là một ví dụ kinh điển của lời nguyền số chiều (*curse of dimensionality*).

Chú giải của người dịch: Mô hình ngôn ngữ và n-gram qua ví dụ

Phương trình (2.26) là *quy tắc nhân xác suất* áp dụng liên tiếp. Với câu "tôi đi học": $\Pr(\text{tôi}, \text{đi}, \text{học}) = \Pr(\text{tôi}) \cdot \Pr(\text{đi} \mid \text{tôi}) \cdot \Pr(\text{học} \mid \text{tôi}, \text{đi})$. Ký hiệu $\Pr(A|B)$ đọc là "xác suất của A khi biết B" (xác suất có điều kiện), và $\prod_{i=1}^{m}$ là "tích của m thừa số", tương tự $\sum$ nhưng với phép nhân. Mô hình n-gram

"quên bớt" ngữ cảnh xa: với $n = 2$ (bigram), $\Pr(\text{học}|\text{tôi, đi}) \approx \Pr(\text{học}|\text{đi})$. Ước lượng bằng đếm: nếu trong kho ngữ liệu cụm "đi" xuất hiện 100 lần, trong đó "đi học" xuất hiện 20 lần, thì $\Pr(\text{học}|\text{đi}) \approx 20/100 = 0{,}2$. Vấn đề: với bộ từ vựng 10 000 từ và $n = 3$ đã có 10^{12} bộ ba có thể có; hầu hết chưa từng xuất hiện trong dữ liệu nên bị đếm bằng 0 — đó là tính "thưa" của dữ liệu.

Ở đây, chúng ta xét việc dùng mạng nơ-ron để giải quyết bài toán mô hình hóa ngôn ngữ [Bengio và cộng sự, 2000; 2003b]. Khác với mô hình n-gram, vốn hoạt động trong một không gian rời rạc đòi hỏi một số lượng lớn theo cấp số mũ các vectơ đặc trưng khác nhau, các **mô hình ngôn ngữ nơ-ron** (*neural language models*) khái quát hóa trong một không gian liên tục bằng cách mã hóa các từ thành những vectơ số thực dày đặc và có số chiều thấp. Cụ thể, ở đây ta dùng một mạng truyền thẳng để dự đoán khả năng từ w_i xuất hiện khi biết ngữ cảnh $w_{i-n+1} \cdots w_{i-1}$.

Hình 2.6 trình bày kiến trúc của **mô hình ngôn ngữ dựa trên mạng nơ-ron truyền thẳng** (*feed-forward neural network based language model, FFNNLM*). Đầu vào là các từ ngữ cảnh $w_{i-n+1} \cdots w_{i-1}$. Mỗi từ là một biến rời rạc nhận giá trị trong một bộ từ vựng (*vocabulary*) V. Vì mạng nơ-ron làm việc với các vectơ, mọi từ đều được vectơ hóa dưới dạng **biểu diễn one-hot** (*one-hot representations*). Khi đó, từ $w = V_k$ là một vectơ $|V|$ chiều, trong đó phần tử thứ k bằng 1 và mọi phần tử khác đều bằng 0. Ví dụ, xét bộ từ vựng $V = \{$"I", "you", "he", "she", "they"$\}$ ("tôi", "bạn", "anh ấy", "cô ấy", "họ"). Biểu diễn one-hot của "you" được viết là

$$\mathbf{1}_{\text{you}} = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \end{bmatrix}. \tag{2.28}$$

Mặc dù các vectơ one-hot xác định duy nhất từng từ, chúng không nắm bắt được các quan hệ ngữ nghĩa (tức là kỳ vọng rằng các từ tương tự nhau nên nằm gần nhau trong không gian vectơ). Bằng cách nới lỏng các biểu diễn dựa trên chỉ thị này thành các vectơ số thực liên tục, ta có thể nắm bắt quan hệ giữa các từ bằng cách tính độ tương đồng giữa các vectơ. Để làm được điều đó, một kỹ thuật hiệu quả là biến đổi biểu diễn one-hot thành **biểu diễn phân tán** (*distributed representations*). Một cách hình thức hơn, gọi $\mathbf{x}$ là vectơ one-hot của một từ w. Biểu diễn phân tán của nó là một vectơ số thực, được cho bởi:

$$\mathbf{e} = \mathbf{x} \cdot \mathbf{C}, \tag{2.29}$$

trong đó biểu diễn **e** là một vectơ thuộc $\mathbb{R}^{d_e}$, và d_e là số chiều của biểu diễn. Mỗi chiều của **e** có thể được xem như một khía cạnh đo đếm được nào đó của từ, dù không bắt buộc phải diễn giải được về mặt ngôn ngữ học. **C** là một ma trận $|V| \times d_e$, trong đó hàng thứ k tương ứng với vectơ của V_k. Do đó, phép $\mathbf{x} \cdot \mathbf{C}$ có tác dụng "chọn ra" một hàng của **C**. Ví dụ, với $\mathbf{C} \in \mathbb{R}^{5\times 3}$, biểu diễn phân tán của "you" được cho bởi:

$$
\begin{aligned}
e(\text{“you”}) &= \mathbf{1}_{\text{you}} \cdot \mathbf{C} \\
&= \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} 73 & 12 & 0.1 \\ 12 & 0.5 & 18 \\ 37 & 0.7 & 28 \\ 61 & 0.4 & 23 \\ 62 & 11 & 0.4 \end{bmatrix} \\
&= \begin{bmatrix} 12 & 0.5 & 18 \end{bmatrix}. \qquad (2.30)
\end{aligned}
$$

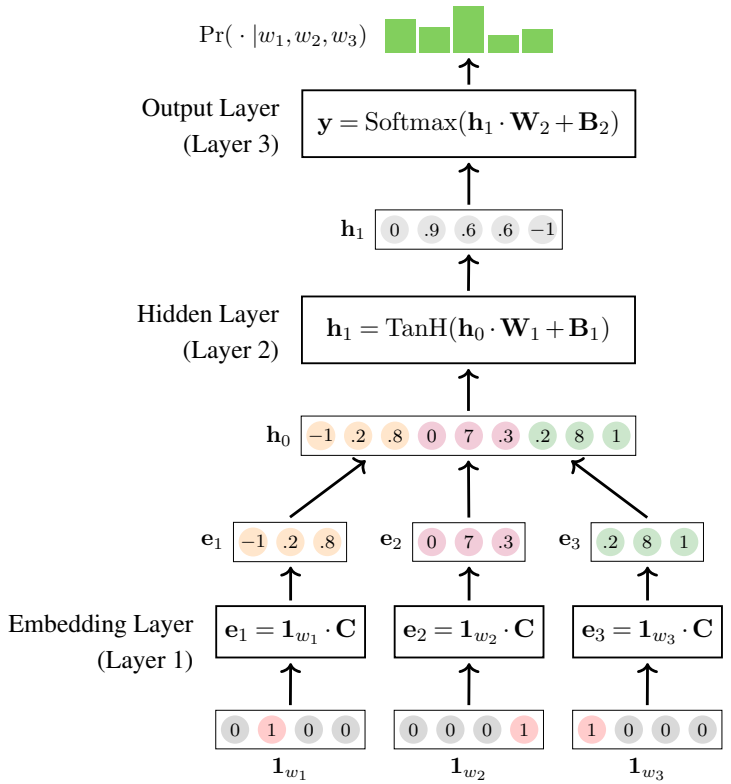
(Trong hình: *Embedding Layer (Layer 1)* = Tầng nhúng (tầng 1); *Hidden Layer (Layer 2)* = Tầng ẩn (tầng 2); *Output Layer (Layer 3)* = Tầng đầu ra (tầng 3).)

Chú giải của người dịch: One-hot và vectơ nhúng: vì sao cần?

Vectơ one-hot giống một "thẻ căn cước": mỗi từ có một vị trí riêng bật số 1. Nhưng mọi cặp vectơ one-hot khác nhau đều có tích vô hướng bằng 0 và khoảng cách như nhau, nên "anh ấy" và "cô ấy" cũng xa nhau y như "anh ấy" và "họ" — máy không thấy được từ nào giống từ nào. Phương trình (2.30) cho thấy nhân vectơ one-hot với ma trận **C** thực chất chỉ là *tra bảng*: số 1 ở vị trí thứ 2 lấy ra đúng hàng thứ 2 (màu đỏ). Mỗi từ vì thế được gán một dãy số thực ngắn (ở đây 3 số) gọi là vectơ nhúng; các số này được *học* từ dữ liệu, và sau khi học, những từ có cách dùng giống nhau thường có vectơ gần nhau.

Phương trình (2.29) giới thiệu khái niệm nền tảng về việc học biểu diễn từ, vốn đã thúc đẩy những tiến bộ to lớn trong NLP. Thông thường, vectơ **e** được gọi là **vectơ nhúng từ** (*word embedding*), còn ma trận tham số **C** được gọi là **ma trận nhúng** (*embedding matrix*). Dù có nhiều phương pháp học vectơ nhúng từ, chúng tôi sẽ không tập trung vào chúng trong chương này. Bạn đọc có thể xem Chương 3 để biết thảo luận chi tiết về chủ đề này.

Để mã hóa các từ ngữ cảnh $\{w_{i-n+1}, \ldots, w_{i-1}\}$, một cách đơn giản là ghép nối các vectơ nhúng từ $\{\mathbf{e}_{i-n+1}, \ldots, \mathbf{e}_{i-1}\}$ thành một vectơ mới



Hình 2.6: Minh họa một mô hình ngôn ngữ nơ-ron [Bengio và cộng sự, 2003b]. Các hộp biểu diễn các tầng của mạng nơ-ron. Đầu vào là ba từ ngữ cảnh ở dạng biểu diễn one-hot $\{\mathbf{1}_{w_1}, \mathbf{1}_{w_2}, \mathbf{1}_{w_3}\}$, và đầu ra là phân phối xác suất của từ tiếp theo $\Pr(w_4|w_1, w_2, w_3)$. Trước hết, một tầng nhúng được dùng để ánh xạ mỗi từ thành biểu diễn phân tán (tức là vectơ nhúng từ). Các vectơ nhúng của những từ này được ghép nối (*concatenated*) để tạo thành một vectơ lớn hơn $\mathbf{h}_0$, sao cho vectơ ghép nối này mã hóa toàn bộ thông tin đầu vào. Sau đó, $\mathbf{h}_0$ được lấy làm đầu vào cho một tầng thông thường thực hiện ánh xạ $\mathbf{h}_1 = \text{TanH}(\mathbf{h}_0 \cdot \mathbf{W}_1 + \mathbf{B}_1)$. Tầng cuối cùng đọc $\mathbf{h}_1$ và tạo ra một phân phối trên bộ từ vựng, tức là $\mathbf{y} = \text{Softmax}(\mathbf{h}_1 \cdot \mathbf{W}_2 + \mathbf{B}_2)$, trong đó $y_k = \Pr(V_k|w_1, w_2, w_3)$.

$\mathbf{h}_0$:

$$\mathbf{h}_0 = [\mathbf{e}_{i-n+1}, \ldots, \mathbf{e}_{i-1}].$$

Phần tiếp theo của mô hình là một mạng nơ-ron truyền thẳng 2 tầng. Tầng thứ nhất, gọi là **tầng ẩn** (*hidden layer*), là một tầng nơ-ron chuẩn, theo sau là hàm kích hoạt tang hyperbolic. Tầng này tạo ra một vectơ d_h chiều:

$$\mathbf{h}_1 = \text{TanH}(\mathbf{h}_0 \cdot \mathbf{W}_1 + \mathbf{B}_1). \tag{2.31}$$

Tầng thứ hai là tầng đầu ra. Nó tạo ra một phân phối trên V. Điều này có thể được biểu diễn là:

$$\Pr(\cdot|w_{i-n+1}, \ldots, w_{i-1}) = \text{Softmax}(\mathbf{h}_1 \cdot \mathbf{W}_2 + \mathbf{B}_2). \tag{2.32}$$

Các tham số của mô hình là $\mathbf{C} \in \mathbb{R}^{|V| \times d_e}$, $\mathbf{W}_1 \in \mathbb{R}^{(n-1)d_e \times d_h}$, $\mathbf{B}_1 \in \mathbb{R}^{d_h}$, $\mathbf{W}_2 \in \mathbb{R}^{d_h \times |V|}$ và $\mathbf{B}_2 \in \mathbb{R}^{|V|}$. Một cách phổ biến để tối ưu các tham số này là cực tiểu hóa hàm mất mát entropy chéo (*cross-entropy loss*) bằng hạ gradient (*gradient descent*). Ngoài ra, việc huấn luyện có thể được cải thiện nhờ chính quy hóa. Các phương pháp này sẽ được thảo luận ở Mục 2.4 và Mục 2.5.

Chú giải của người dịch: Đếm kích thước trong mô hình

Giả sử $n = 4$ (dùng 3 từ ngữ cảnh), $d_e = 3$ như Hình 2.6. Ghép ba vectơ nhúng 3 chiều được $\mathbf{h}_0$ có $(n-1)d_e = 9$ chiều — đúng 9 ô trong hình. Vì vậy $\mathbf{W}_1$ phải có 9 hàng; số cột d_h là độ rộng tầng ẩn (trong hình $d_h = 5$). Tầng ra nhân $\mathbf{h}_1$ (5 số) với $\mathbf{W}_2$ ($5 \times |V|$) được $|V|$ số — mỗi từ trong bộ từ vựng một điểm số — rồi Softmax biến chúng thành xác suất để từ đó là từ tiếp theo. Chữ "hạ gradient" ở đây nghĩa là liên tục nhích các tham số theo hướng làm hàm mất mát giảm, dùng gradient tính bằng lan truyền ngược như Mục 2.1.3.

Một vài nhận xét về mô hình ngôn ngữ nơ-ron: thứ nhất, nhờ dùng các vectơ đặc trưng phân tán, các từ khác nhau có thể chia sẻ một phần "nghĩa" với nhau. Điều này cho phép học được nghĩa của từ, qua đó độ tương đồng giữa các từ được xét đến một cách ngầm định. Một ưu điểm của mô hình như vậy là một thay đổi nhỏ trong vectơ từ sẽ không dẫn tới thay đổi lớn trong kết quả. Ví dụ, giả sử ta đã thấy câu "grapes are fruits" ("nho là trái cây") nhiều lần nhưng chưa bao giờ thấy "peaches are fruits" ("đào là trái cây"). Nếu vectơ nhúng của "grapes" và "peaches" gần nhau,

mô hình có thể suy ra một cách hợp lý rằng cụm "peaches are fruits" có xác suất cao. Điều này làm mô hình ngôn ngữ nơ-ron khác biệt rất lớn so với mô hình ngôn ngữ n-gram, nơi các dạng bề mặt khác nhau được xem là những thực thể hoàn toàn độc lập.

Thứ hai, biểu diễn dày đặc của từ giúp mô hình nhỏ hơn. Ví dụ, một thiết lập phổ biến của d_e và d_h là nhỏ hơn 1000, giữ cho tổng số tham số ở mức kiểm soát được. Ngược lại, kích thước của một mô hình ngôn ngữ n-gram tăng lên gấp $|V|$ lần mỗi khi n tăng. Chẳng hạn, với một bộ từ vựng thông thường, sẽ có một bảng xác suất khổng lồ nếu n lớn hơn 3.

Thứ ba, mô hình ngôn ngữ nơ-ron tốn kém về tính toán vì sử dụng rất nhiều phép toán trên vectơ và ma trận, chẳng hạn phép nhân ma trận. Đây là vấn đề chung của hầu hết các hệ thống dựa trên mạng nơ-ron sâu. Một giải pháp phổ biến là chia bài toán tính toán thành các bài toán con độc lập để có thể xử lý song song các bài toán con này. Ở mức thấp, ta có thể dùng GPU hoặc các thiết bị tính toán song song khác để tăng tốc các phép toán đại số tuyến tính. Ở mức cao hơn, ta có thể phân phối các phần của mô hình hoặc các phần của dữ liệu lên nhiều thiết bị để tăng tốc ở mức mô hình hoặc mức tác vụ.

2.3 Các kiến trúc mô hình cơ bản

Bây giờ chúng ta mô tả chi tiết hơn một số khối xây dựng cơ bản của mạng nơ-ron. Chúng được dùng rộng rãi để phát triển các mô hình nơ-ron tiên tiến nhất (*state-of-the-art*) trong NLP.

2.3.1 Đơn vị hồi quy

Mạng nơ-ron hồi quy (*recurrent neural networks*) (**RNN**) là một lớp mạng nơ-ron đọc và/hoặc sinh ra dữ liệu dạng chuỗi hoặc dữ liệu chuỗi thời gian (*time-series*). Cũng như mạng nơ-ron truyền thẳng (*feed-forward neural network*), một RNN gồm các tầng nơ-ron và các kết nối giữa các nơ-ron [Hopfield, 1984; Rumelhart và cộng sự, 1986; Williams và Zipser, 1989; Elman, 1990]. Một số nơ-ron được dùng như một "bộ nhớ" lưu giữ trạng thái của bài toán khi quá trình xử lý tiến dần dọc theo một chuỗi tín hiệu. Nhờ vậy, việc dùng RNN để giải quyết các bài toán có độ dài

thay đổi, chẳng hạn dịch máy (*machine translation*) và nhận dạng tiếng nói (*speech recognition*), là rất tự nhiên.

Ý tưởng chính đằng sau RNN là sử dụng lặp đi lặp lại một **đơn vị hồi quy** (*recurrent unit*) (hay **ô hồi quy** (*recurrent cell*)) để tính đầu ra tại mỗi vị trí của chuỗi đầu vào. Chính xác hơn, cho một chuỗi vectơ $\mathbf{x}_1 \cdots \mathbf{x}_m$, một **đơn vị hồi quy** chuẩn có thể được mô tả như một hàm $\text{RNN}(\cdot)$, tại mỗi bước thời gian (*time step*) nhận một đầu vào $\mathbf{x}_i$ và một trạng thái $\mathbf{s}_{i-1}$, rồi sinh ra một trạng thái mới $\mathbf{s}_i$:

$$\mathbf{s}_i \;=\; \text{RNN}(\mathbf{s}_{i-1}, \mathbf{x}_i). \tag{2.33}$$

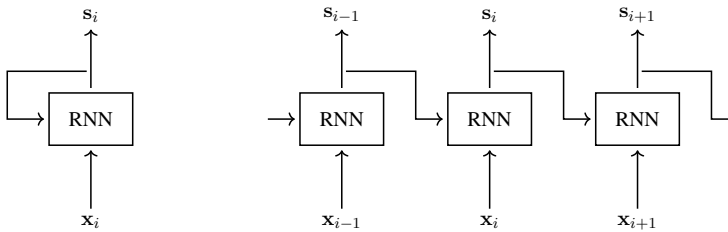
Trạng thái $\mathbf{s}_i$ có thể được xem như một "bộ nhớ" tóm tắt thông tin trong quá khứ và được cập nhật khi có dữ liệu mới đến. Xem Hình 2.7 (a) để thấy minh họa của Phương trình (2.33). Vòng lặp trong hình biểu thị việc tái sử dụng đơn vị hồi quy. Điều này có thể hiểu được bằng cách viết lại Phương trình (2.33) thành một dãy các lời gọi hàm $\text{RNN}(\cdot)$:

$$\begin{aligned}
\text{RNN}(\mathbf{s}_{i-1}, \mathbf{x}_i) &= \text{RNN}(\text{RNN}(\mathbf{s}_{i-2}, \mathbf{x}_{i-1}), \mathbf{x}_i) \\
&= \text{RNN}(\text{RNN}(\text{RNN}(\mathbf{s}_{i-3}, \mathbf{x}_{i-2}), \mathbf{x}_{i-1}), \mathbf{x}_i) \\
&= \dots \\
&= \text{RNN}(\text{RNN}(\cdots(\text{RNN}(\mathbf{s}_0, \mathbf{x}_1), \mathbf{x}_2)\cdots \mathbf{x}_{i-1}), \mathbf{x}_i).
\end{aligned} \tag{2.34}$$

Hình 2.7 (b) minh họa cấu trúc của mạng đã được khai triển này. Cách biểu diễn này thường được gọi là cấu trúc **trải phẳng** (*unrolled*) (hay **mở rộng** (*unfolded*)) của RNN. Về bản chất, Hình 2.7 (a) và (b) biểu diễn đúng cùng một tập phép toán. Trong khi RNN dạng cuộn (*rolled*) cho một dạng toán học gọn gàng và đẹp đẽ, thì RNN dạng trải phẳng giúp hình dung rõ hơn dòng dữ liệu chảy qua thời gian. Vì vậy, trong suốt cuốn sách này chúng tôi sẽ dùng cách biểu diễn trải phẳng của RNN.

Hơn nữa, đáng lưu ý rằng một RNN trải phẳng có thể được xem như một mạng nơ-ron truyền thẳng sâu, trong đó mỗi lần áp dụng đơn vị hồi quy đóng vai trò một "tầng" nhận thông tin từ "tầng" liền trước (tức là bước thời gian trước đó). Theo nghĩa này, một RNN thực chất là một chồng các tầng trải dài theo chiều ngang qua thời gian. Lợi ích của góc nhìn này là ta có thể dùng những kỹ thuật tối ưu tương tự để huấn luyện cả hai kiến trúc. Chẳng hạn, cả RNN và mạng truyền thẳng đều dựa vào tối ưu hóa dựa trên

gradient, chỉ có điều RNN dùng riêng một biến thể của lan truyền ngược gọi là **lan truyền ngược theo thời gian** (*back-propagation through time*) (**BPTT**) để lan truyền sai số ngược lại qua đồ thị tính toán (*computation graph*) đã trải phẳng.



(a) Rolled RNN Structure (b) Unrolled RNN Structure

Hình 2.7: Ví dụ về RNN (dạng cuộn và dạng trải phẳng). Một đơn vị RNN đọc đầu vào tại mỗi bước thời gian i và đầu ra tại bước thời gian trước $i-1$, rồi sinh ra một đầu ra mới s_i. Như vậy, ta có thể tái sử dụng cùng một đơn vị RNN để đưa ra dự đoán trên một chuỗi đầu vào (xem hình con (a)): với mỗi i, đầu vào hiện tại $\mathbf{x}_i$ và đầu ra trước đó $\mathbf{s}_{i-1}$ được tiếp nhận và ánh xạ thành đầu ra, đầu ra này lại được đưa vào chính đơn vị RNN đó để xử lý ở bước thời gian kế tiếp. Một cách tốt hơn để hình dung RNN là trải nó ra thành một mạng không có chu trình (xem hình con (b)). RNN trải phẳng có thể được xem như một mạng nơ-ron truyền thẳng sâu, trong đó tất cả các đơn vị RNN dùng chung một bộ tham số.

Nhãn trong hình: *Rolled RNN Structure* = cấu trúc RNN dạng cuộn; *Unrolled RNN Structure* = cấu trúc RNN dạng trải phẳng.

Chú giải của người dịch: RNN giống như đọc truyện từng trang

Hãy tưởng tượng bạn đọc một câu chuyện từng chữ một và giữ trong đầu một "bản tóm tắt" $\mathbf{s}$. Mỗi khi đọc thêm một chữ $\mathbf{x}_i$, bạn cập nhật bản tóm tắt: $\mathbf{s}_i = \text{RNN}(\mathbf{s}_{i-1}, \mathbf{x}_i)$. Điều quan trọng là bạn dùng *cùng một cách cập nhật* (cùng một hàm, cùng tham số) cho mọi chữ, nên câu dài 5 chữ hay 500 chữ đều xử lý được. Phương trình (2.34) chỉ là phép "thay thế liên tiếp" giống như với dãy số truy hồi ở lớp 11: với $u_n = 2u_{n-1} + 1$ ta có $u_n = 2(2u_{n-2} + 1) + 1 = \ldots$, cho đến khi gặp u_0. Ở đây, $\mathbf{s}_0$ đóng vai trò "số hạng đầu".

Có nhiều biến thể RNN, khác nhau ở cách định nghĩa hàm $\text{RNN}(\cdot)$. Đơn giản nhất là xây dựng $\text{RNN}(\cdot)$ như một mạng nơ-ron một tầng. Giả

sử $\mathbf{s}_{i-1}$ và $\mathbf{x}_i$ thuộc $\mathbb{R}^{d_h}$. Dạng của RNN$(\cdot)$ là:

$$\text{RNN}(\mathbf{s}_{i-1}, \mathbf{x}_i) \;=\; \psi(\mathbf{s}_{i-1} \cdot \mathbf{U} + \mathbf{x}_i \cdot \mathbf{V}), \tag{2.35}$$

trong đó $\mathbf{U} \in \mathbb{R}^{d_h \times d_h}$ và $\mathbf{V} \in \mathbb{R}^{d_h \times d_h}$ là các tham số. Những lựa chọn phổ biến cho hàm kích hoạt (*activation function*) ψ là TanH$(\cdot)$, Sigmoid$(\cdot)$, ReLU$(\cdot)$, v.v. Phương trình (2.35) là một mạng nơ-ron một tầng vì nó có cùng dạng với Phương trình (2.3–2.4):

$$\psi(\mathbf{s}_{i-1} \cdot \mathbf{U} + \mathbf{x}_i \cdot \mathbf{V}) \;=\; \psi([\mathbf{s}_{i-1}, \mathbf{x}_i] \cdot \mathbf{W}), \tag{2.36}$$

trong đó $[\mathbf{s}_{i-1}, \mathbf{x}_i]$ là phép ghép nối (*concatenation*) $\mathbf{s}_{i-1}$ và $\mathbf{x}_i$, còn $\mathbf{W} \in \mathbb{R}^{2d_h \times d_h}$ là ma trận tham số được tạo thành bởi $\begin{bmatrix}\mathbf{U}\\ \mathbf{V}\end{bmatrix}$.

Chú giải của người dịch: Vì sao Phương trình (2.36) đúng?

Ghép nối nghĩa là đặt hai vectơ cạnh nhau: nếu $\mathbf{s} = [1 \;\; 2]$ và $\mathbf{x} = [3 \;\; 4]$ thì $[\mathbf{s}, \mathbf{x}] = [1 \;\; 2 \;\; 3 \;\; 4]$. Khi nhân vectơ dài $2d_h$ này với ma trận $\mathbf{W}$ gồm $\mathbf{U}$ chồng lên trên $\mathbf{V}$, các phần tử của $\mathbf{s}$ chỉ "gặp" các hàng của $\mathbf{U}$, còn các phần tử của $\mathbf{x}$ chỉ "gặp" các hàng của $\mathbf{V}$, nên kết quả đúng bằng $\mathbf{s} \cdot \mathbf{U} + \mathbf{x} \cdot \mathbf{V}$. Vậy một bước RNN đơn giản thực chất chỉ là một tầng mạng nơ-ron bình thường có đầu vào là "trạng thái cũ + dữ liệu mới". Hàm TanH$(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$ nhận giá trị trong khoảng $(-1; 1)$.

Trong thực tế, RNN thường được dùng như các thành phần con trong những kiến trúc lớn hơn. Chẳng hạn, đầu vào của một đơn vị hồi quy có thể là biểu diễn trực tiếp của dữ liệu thô (như vectơ nhúng từ (*word embeddings*)) hoặc đầu ra của một mô-đun mạng nơ-ron khác. Tương tự, các tầng mạng nơ-ron bổ sung thường được xếp chồng lên trên các đơn vị hồi quy. Trong nhiều hệ thống NLP thực tế, một tầng đầu ra (*output layer*) được đặt trực tiếp trên trạng thái ẩn (*hidden state*) $\mathbf{s}_i$ để chiếu nó vào không gian đầu ra mong muốn cho các tác vụ phân loại hoặc dự đoán.

2.3.2 Đơn vị tích chập

Mạng nơ-ron tích chập (*convolutional neural networks*) (**CNN**) là một lớp mạng nơ-ron nổi tiếng khác [Waibel và cộng sự, 1989; LeCun và cộng sự, 1989]. Về mặt sinh học, chúng được lấy cảm hứng từ hệ thị giác của

con người: các nơ-ron phản ứng với kích thích trong một vùng giới hạn của thị trường (gọi là **trường tiếp nhận** (*receptive field*)) [Hubel và Wiesel, 1959]. Trong CNN, trường tiếp nhận mô tả vùng trong không gian đầu vào tham gia vào việc tạo ra đầu ra của một nơ-ron. Do đó, CNN là các mô hình "kết nối một phần", trong đó mỗi nơ-ron chỉ xét các đặc trưng đầu vào nằm trong một vùng giới hạn. Điều này phân biệt CNN với các mạng nơ-ron truyền thẳng kết nối đầy đủ (*fully connected*). Nhìn chung, CNN nắm bắt một cách tự nhiên tính phân cấp của các đặc trưng (*feature*) mô tả dữ liệu và co giãn hiệu quả hơn về mặt độ phức tạp.

Mặc dù CNN có nhiều ứng dụng trong xử lý dữ liệu 2 chiều, chẳng hạn phân loại ảnh, ở đây chúng tôi bàn về chúng trong bối cảnh xử lý dữ liệu chuỗi để giữ cách tiếp cận nhất quán trong suốt chương này. Thông thường, một CNN gồm một **tầng tích chập** (*convolutional layer*), một **tầng gộp** (*pooling layer*), và có thể thêm các loại tầng khác. Nó bắt đầu bằng tầng tích chập, nơi trường tiếp nhận được định nghĩa bởi một tập các **nhân tích chập** (*convolution kernels*) hay **bộ lọc** (*filters*). Bộ lọc là một hàm ánh xạ tuyến tính, thực hiện tích chập trên các đặc trưng đầu vào trong trường tiếp nhận để tạo thành một đặc trưng đầu ra duy nhất. Ví dụ, xét một dãy số $x_1 \cdots x_m$. Bộ lọc trải từ vị trí i đến vị trí $i + r - 1$ được định nghĩa là:

$$\begin{aligned} v &= \text{Conv}(\mathbf{x}_{[i,i+r-1]}, \mathbf{W}) \\ &= \mathbf{x}_{[i,i+r-1]} \cdot \mathbf{W} \\ &= \sum_{k=0}^{r-1} x_{i+k} \cdot W_k, \end{aligned} \tag{2.37}$$

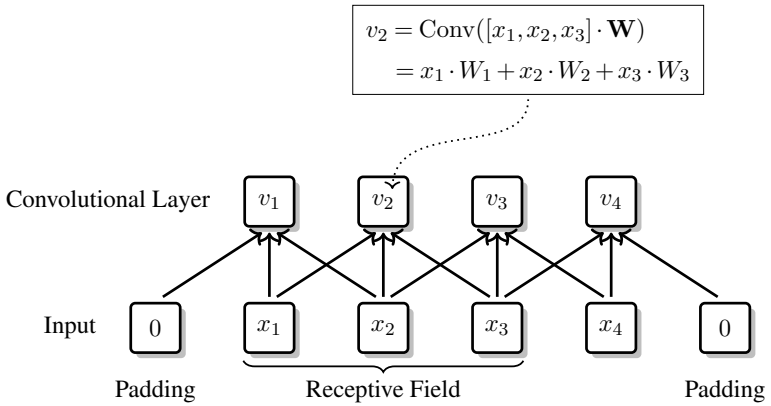
trong đó r là kích thước của trường tiếp nhận, $\mathbf{x}_{[i,i+r-1]}$ là chuỗi con $x_i \cdots x_{i+r-1}$, và $\mathbf{W} \in \mathbb{R}^r$ là các tham số của bộ lọc. Sau đó, có thể sinh ra một chuỗi đặc trưng đầu ra bằng cách di chuyển bộ lọc dọc theo chuỗi đầu vào. Gọi stride (bước trượt (*stride*)) là khoảng cách giữa hai lần di chuyển liên tiếp. Đầu ra của lần di chuyển thứ i khi đó được định nghĩa là:

$$v_i \;=\; \text{Conv}(\mathbf{x}_{[\text{stride}\times i,\ \text{stride}\times i+r-1]}, \mathbf{W}). \tag{2.38}$$

Bằng cách này, tầng tích chập biến chuỗi đầu vào $x_1 \cdots x_m$ thành chuỗi đầu ra $v_1 \cdots v_{\lfloor \frac{m}{\text{stride}} \rfloor}$[4]. Một lưu ý ở đây là các tham số $\mathbf{W}$ được dùng chung

[4] $\lfloor \cdot \rfloor$ ký hiệu hàm phần nguyên (làm tròn xuống).

cho mọi vị trí của chuỗi đầu vào. Phương pháp này được gọi là **chia sẻ tham số** (*parameter sharing*) hay **chia sẻ trọng số** (*weight sharing*). Chia sẻ tham số làm CNN hiệu quả vì nó cần ít tham số hơn một mạng nơ-ron truyền thẳng có cùng số nơ-ron.



Hình 2.8: Minh họa phép tích chập trên một dãy số $\{x_1, x_2, x_3, x_4\}$ ($r = 3$ và stride $= 1$). Trường tiếp nhận xác định vùng của đầu vào được dùng để tính đầu ra. Ở đây trường tiếp nhận có kích thước 3, tức là phép tích chập bao phủ ba số liên tiếp trong chuỗi đầu vào. Bộ lọc (hay nhân tích chập) cho ra tổng có trọng số của các số này. Mỗi lần ta trượt trường tiếp nhận trên đầu vào, bộ lọc sinh ra một đầu ra mới. Như vậy, đầu ra của tầng tích chập là một vectơ các số. Ngoài ra, một số đệm (tức là 0) được thêm vào mỗi đầu của chuỗi để phép tích chập thực hiện được khi trường tiếp nhận không đầy đủ.

Nhãn trong hình: *Convolutional Layer* = tầng tích chập; *Input* = đầu vào; *Padding* = phần đệm; *Receptive Field* = trường tiếp nhận.

Một vấn đề với cách xây dựng trên là bộ lọc có thể không "lát" khít được chuỗi đầu vào. Chẳng hạn, khi stride $\times\, i + r - 1 > m$, đầu vào của bộ lọc bị thiếu. Một giải pháp thường dùng là **đệm** (*padding*). Nó đơn giản đặt các đặc trưng nằm ngoài chuỗi đầu vào bằng một hằng số. Ví dụ, ta có thể nối thêm các vectơ đặc trưng giả vào mỗi đầu của chuỗi để mọi phép tích chập đều thực hiện được. Xem Hình 2.8 để có ví dụ về một bộ lọc được tính trên một dãy số. Lưu ý rằng trường tiếp nhận của các lần áp dụng bộ lọc khác nhau có thể chồng lấn nhau. Điều này đôi khi có lợi vì nó giảm mất mát thông tin trong biểu diễn đặc trưng khi một đặc trưng

cấp thấp được dùng để tạo nên nhiều đặc trưng cấp cao.

Chú giải của người dịch: Tính tay một phép tích chập

Lấy dãy $x = (1,\ 2,\ 3,\ 4)$, bộ lọc $\mathbf{W} = (W_0, W_1, W_2) = (1,\ 0,\ -1)$, $r = 3$, stride $= 1$, đệm một số 0 ở mỗi đầu: $(0,\ 1,\ 2,\ 3,\ 4,\ 0)$. Trượt "cửa sổ" 3 ô qua dãy đã đệm: $v_1 = 0 \cdot 1 + 1 \cdot 0 + 2 \cdot (-1) = -2$, $v_2 = 1 - 3 = -2$, $v_3 = 2 - 4 = -2$, $v_4 = 3 - 0 = 3$. Cùng ba số $(1, 0, -1)$ được dùng ở mọi vị trí — đó chính là *chia sẻ tham số*. Bộ lọc này đo "độ chênh" giữa hai bên cửa sổ; các bộ lọc khác có thể phát hiện các "mẫu" khác. Ký hiệu $\lfloor a \rfloor$ là số nguyên lớn nhất không vượt quá a, ví dụ $\lfloor 3{,}7 \rfloor = 3$.

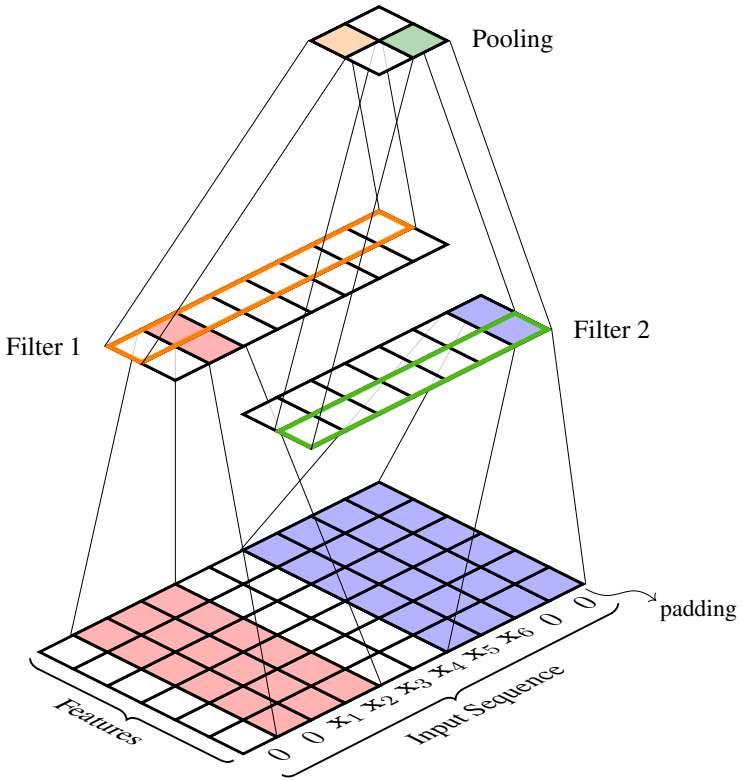
Ngoài ra, một tầng tích chập có thể kèm theo một hàm kích hoạt $\psi(\cdot)$ để thực hiện ánh xạ phi tuyến trên đầu ra của bộ lọc. Gọi $m_k = \lfloor \frac{m}{\text{stride}} \rfloor$ là số lần áp dụng bộ lọc. Đầu ra của một tầng tích chập được cho bởi

$$\begin{bmatrix} h_1 & \cdots & h_{m_k} \end{bmatrix} = \psi\left(\begin{bmatrix} v_1 & \cdots & v_{m_k} \end{bmatrix}\right). \tag{2.39}$$

Nói chung, một tầng tích chập không nhất thiết bị giới hạn ở đầu vào là các số vô hướng hay chỉ một bộ lọc. Thường thì ta có thể lấy một vectơ làm biểu diễn của một token trong chuỗi đầu vào, và dùng một tập các bộ lọc làm bộ trích xuất đặc trưng. Để làm vậy, ta có thể áp dụng cùng cách xây dựng như trong Phương trình (2.37–2.39), nhưng thay x_i, v_i và h_i bằng các phiên bản dạng vectơ tương ứng.

Nhãn trong hình: *Filter* = bộ lọc; *Pooling* = gộp; *Features* = các đặc trưng; *Input Sequence* = chuỗi đầu vào; *padding* = phần đệm.

Một tầng tích chập thường được theo sau bởi một tầng gộp. Giống như tích chập, gộp là một hàm quét một bộ lọc trên một chuỗi. Nhưng phép gộp không có tham số nào. Thay vào đó, có thể xem nó như một hàm tổng hợp thực hiện giảm mẫu (*down-sampling*) trên chuỗi đầu vào. Có một số cách thiết kế hàm gộp. Một trong những phương pháp phổ biến nhất là **gộp cực đại** (*max pooling*), cho ra giá trị lớn nhất trong trường tiếp nhận. Một phương pháp khác là **gộp trung bình** (*average pooling*), cho ra giá trị trung bình trên trường tiếp nhận. Để có bức tranh đầy đủ về cách một CNN hoạt động, Hình 2.9 mô tả một ví dụ trong đó các phép tích chập và gộp được thực hiện trên một chuỗi vectơ đặc trưng thông qua hai bộ lọc.



Hình 2.9: Ví dụ về CNN. Tầng tích chập có hai bộ lọc. Đầu vào là một chuỗi 6 token được biểu diễn bằng các vectơ đặc trưng $\{\mathbf{x}_1, \ldots, \mathbf{x}_6\}$. Để các bộ lọc "lát" khít chuỗi đầu vào, hai vectơ đệm được gắn vào mỗi đầu của chuỗi. Khi áp dụng một bộ lọc, ta ánh xạ các vectơ đặc trưng trong trường tiếp nhận thành một vectơ đặc trưng mới. Ví dụ, với bộ lọc 1, trường tiếp nhận là một hình chữ nhật 6×3 trong đầu vào, và đầu ra là một vectơ đặc trưng 2 chiều. Bằng cách quét bộ lọc trên chuỗi, ta thu được một chuỗi các vectơ đặc trưng, chẳng hạn một chuỗi 8 vectơ đặc trưng, mỗi vectơ có 2 đặc trưng. Tầng gộp hợp nhất các đặc trưng dọc theo chuỗi. Ví dụ, thực hiện phép gộp trên đầu ra của bộ lọc 1 cho ra 2 đặc trưng đã hợp nhất. Đầu ra cuối cùng của CNN là hai vectơ đặc trưng 2 chiều.

Chú giải của người dịch: Gộp cực đại và gộp trung bình

Giả sử đầu ra tầng tích chập là $(-2,\ -2,\ -2,\ 3)$. Gộp cực đại với cửa sổ 2, bước trượt 2: $\max(-2,-2) = -2$, $\max(-2,3) = 3 \Rightarrow (-2,\ 3)$. Gộp trung bình: $(-2,\ 0{,}5)$. Chuỗi bị "rút gọn" một nửa mà vẫn giữ được thông tin nổi bật, và phép này không cần học tham số nào.

2.3.3 Đơn vị cổng

Trong mạng nơ-ron, một **cổng** (*gate*) được dùng để quyết định lượng thông tin được truyền đi [Hochreiter và Schmidhuber, 1997]. Lấy một RNN chuẩn làm ví dụ. Tại mỗi bước thời gian i, thay vì truyền trực tiếp trạng thái trước $\mathbf{s}_{i-1} \in \mathbb{R}^{d_h}$ vào đơn vị hồi quy, thường sẽ hiệu quả hơn nếu ta xác định phần nào của thông tin trong $\mathbf{s}_{i-1}$ là có liên quan cho quyết định tiếp theo. Trong bối cảnh này, ta muốn $\mathbf{s}_{i-1}$ hoạt động giống một bộ nhớ thực thụ hơn: theo thời gian, một số thông tin cần được giữ lại trong khi những thông tin khác bị loại bỏ.

Một cách phổ biến để làm được điều này là đưa vào một vectơ cổng (*gating vector*) để điều khiển độ lớn của dòng dữ liệu. Ở đây, chúng tôi dùng lại ký hiệu của RNN (xem Mục 2.3.1), nhưng mô tả của chúng tôi là tổng quát và áp dụng được cho bất kỳ kiến trúc nào cần một cơ chế như vậy. Gọi $\mathbf{z} \in [0,1]^{d_h}$ là một vectơ cổng, trong đó $z_k = 0$ nghĩa là quên hoàn toàn ở chiều k, còn $z_k = 1$ nghĩa là giữ lại hoàn toàn ở chiều k. Việc áp dụng $\mathbf{z}$ làm cổng cho $\mathbf{s}_{i-1}$ có thể được viết là:

$$\text{Gate}(\mathbf{z}, \mathbf{s}_{i-1}) \ = \ \mathbf{z} \odot \mathbf{s}_{i-1}, \tag{2.40}$$

trong đó $\odot$ ký hiệu phép nhân theo từng phần tử (*element-wise*) của hai vectơ. Phép toán $\text{Gate}(\mathbf{z}, \mathbf{s}_{i-1})$ biểu diễn một bản cập nhật đã được "lọc" của $\mathbf{s}_{i-1}$. Vectơ $\mathbf{z}$ có thể được gọi là cổng cập nhật (*update gate*), cổng quên (*forget gate*), hoặc một thành phần có chức năng tương tự. Một cách khác, hàm cổng có thể được định nghĩa là:

$$\text{Gate}(\mathbf{z}, \mathbf{s}_{i-1}) \ = \ (1 - \mathbf{z}) \odot \mathbf{s}_{i-1}. \tag{2.41}$$

Phương trình (2.40) và (2.41) mô tả cùng một cơ chế nền tảng nhưng mang những cách hiểu khác nhau về $\mathbf{z}$.

Câu hỏi trung tâm là tính **z** như thế nào. Một phương pháp chuẩn là định nghĩa **z** là đầu ra của một tầng mạng nơ-ron bên trong. Ví dụ, trong một đơn vị hồi quy, **z** có thể được định nghĩa là:

$$\mathbf{z} = \text{Sigmoid}(\mathbf{s}_{i-1} \cdot \mathbf{W}_1 + \mathbf{x}_i \cdot \mathbf{W}_2 + \mathbf{b}). \tag{2.42}$$

Việc dùng hàm kích hoạt Sigmoid bảo đảm các giá trị đầu ra nằm hẳn trong khoảng $[0, 1]$. Lưu ý rằng Phương trình (2.42) mô tả một cổng *học được*. Điều này đưa cơ chế cổng trực tiếp vào trong mô hình, cho phép nó được tối ưu bằng các quy trình huấn luyện chuẩn. Có nhiều kiến trúc cổng khác nhau, một số trong đó sẽ được tìm hiểu chi tiết ở Chương 4.

Chú giải của người dịch: Cổng như một "núm vặn âm lượng"

Nhân theo từng phần tử nghĩa là nhân các phần tử cùng vị trí: $[0{,}9 \;\; 0 \;\; 0{,}5] \odot [4 \;\; 7 \;\; 2] = [3{,}6 \;\; 0 \;\; 1]$. Mỗi z_k giống như một núm vặn âm lượng cho chiều k của bộ nhớ: $z_k = 1$ giữ nguyên, $z_k = 0$ tắt hẳn (quên), $z_k = 0{,}5$ giữ một nửa. Hàm Sigmoid $\text{Sigmoid}(a) = \frac{1}{1+e^{-a}}$ luôn cho giá trị trong khoảng $(0; 1)$, nên rất hợp để tạo ra các "núm vặn" này. Vì **z** được tính từ trạng thái cũ và dữ liệu mới bằng các tham số học được, mô hình tự học được *khi nào* nên nhớ và *khi nào* nên quên.

2.3.4 Đơn vị chuẩn hóa

Một mạng nơ-ron hoạt động bằng cách biến đổi các vectơ đặc trưng qua từng tầng. Mặc dù bản chất nhiều tầng, nhiều chiều của mạng nơ-ron cho phép mô hình tính được những hàm phức tạp, nó có thể dẫn tới những phân phối rất khác nhau của các giá trị kích hoạt đầu ra giữa các tầng hoặc giữa các đặc trưng. Đây là một vấn đề với các mạng nơ-ron sâu vì những mô hình như vậy phải thích nghi với các phân phối khác nhau ở các tầng khác nhau hoặc các đặc trưng khác nhau [Ioffe và Szegedy, 2015]. Đôi khi, do tham số mô hình được khởi tạo ngẫu nhiên ở mọi tầng và mọi chiều đặc trưng, một số đặc trưng có khả năng nhận những giá trị lớn bất thường. Trong trường hợp này, mô hình sẽ bị thiên lệch về phía những đặc trưng có độ lớn lớn hơn.

Một cách để giảm nhẹ vấn đề này là **chuẩn hóa** (*normalization*), tức là chuẩn hóa một vectơ đặc trưng n chiều **s** theo

$$\text{Normalize}(\mathbf{s}) = \boldsymbol{\alpha} \odot \frac{\mathbf{s} - \boldsymbol{\mu}}{\boldsymbol{\sigma} + \boldsymbol{\epsilon}} + \boldsymbol{\beta}, \tag{2.43}$$

trong đó $\boldsymbol{\mu} \in \mathbb{R}^n$ và $\boldsymbol{\sigma} \in \mathbb{R}^n$ lần lượt là trung bình và độ lệch chuẩn của $\mathbf{s}$. ϵ là một số nhỏ dùng để bảo đảm ổn định số học [Chiang và Cholak, 2022]. $\boldsymbol{\alpha} \in \mathbb{R}^n$ và $\boldsymbol{\beta} \in \mathbb{R}^n$ là các tham số của đơn vị chuẩn hóa. Một lựa chọn đơn giản là $\alpha = \mathbf{1}$ và $\beta = \mathbf{0}$, còn một phương pháp tinh vi hơn là học α và β cùng với các tham số khác.

Ta có thể hiện thực Phương trình (2.43) theo nhiều cách, khác nhau ở cách định nghĩa $\boldsymbol{\mu}$ và $\boldsymbol{\sigma}$. Hãy xét điều này cho một chiều trong $\mathbf{s}$, gọi là s, trong một bối cảnh tổng quát. Giả sử s được lấy từ một tập các giá trị đặc trưng Ω_k. Trung bình và độ lệch chuẩn trên Ω_k khi đó được định nghĩa là:

$$\mu_k = \frac{1}{|\Omega_k|} \sum_{s \in \Omega_k} s, \tag{2.44}$$

$$\sigma_k = \sqrt{\frac{1}{|\Omega_k|} \sum_{s \in \Omega_k} (s - \mu_k)^2}. \tag{2.45}$$

Có một số phương pháp để định nghĩa Ω_k. Ví dụ, ta có thể định nghĩa Ω_k là các đặc trưng trong cùng một tầng [Ba và cộng sự, 2016], hoặc là các đặc trưng cùng một chiều trên các mẫu hoặc các vị trí đầu vào khác nhau [Ioffe và Szegedy, 2015; Ulyanov và cộng sự, 2016], hoặc là một cách nhóm lai giữa các chiều này [Wu và He, 2018].

Một ưu điểm của chuẩn hóa là đưa các đặc trưng về cùng một thang đo và làm cho chúng so sánh được với nhau. Điều này được thấy là rất hữu ích cho việc ổn định quá trình huấn luyện và cải thiện tính hội tụ. Như ta sẽ thấy ở các chương sau, chuẩn hóa đóng vai trò quan trọng trong nhiều hệ thống thành công.

Chú giải của người dịch: Tính tay một phép chuẩn hóa

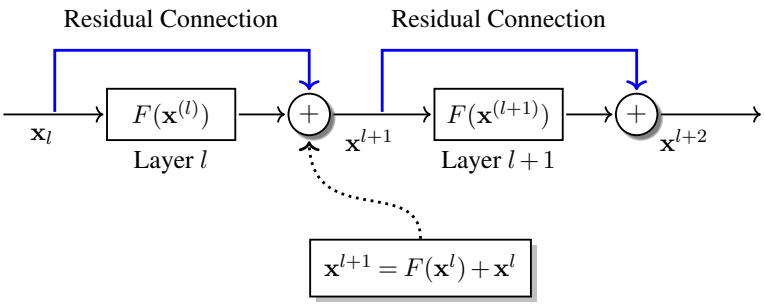
Đây chính là "điểm chuẩn" (z-score) trong thống kê mô tả. Lấy $\mathbf{s} = (2,\ 4,\ 6)$ và chuẩn hóa trên chính các phần tử của nó (tức Ω_k là các đặc trưng trong cùng một tầng). Trung bình $\mu = \frac{2+4+6}{3} = 4$; phương sai $\frac{(2-4)^2+0^2+(6-4)^2}{3} = \frac{8}{3}$, nên $\sigma = \sqrt{8/3} \approx 1{,}63$. Với $\alpha = 1$, $\beta = 0$ và bỏ qua ϵ: Normalize$(\mathbf{s}) \approx (-1{,}22;\ 0;\ 1{,}22)$. Dù ban đầu là $(2, 4, 6)$ hay $(200, 400, 600)$, kết quả đều như nhau: các giá trị được "kéo" về quanh 0 với độ phân tán cỡ 1. Số ϵ rất nhỏ (ví dụ 10^{-6}) chỉ để tránh chia cho 0 khi mọi phần tử bằng nhau. Ký hiệu $|\Omega_k|$ là số phần tử của tập Ω_k.

Nhân tiện nói thêm, mặc dù thuật ngữ *chuẩn hóa* (*normalization*)

trong học sâu thường chỉ quá trình trừ đi trung bình và chia cho độ lệch chuẩn, thực ra đó là một quá trình **chuẩn tắc hóa** (*standardization*). Ngược lại, ở các lĩnh vực khác, chuẩn hóa thường được hiểu là một kỹ thuật co giãn mọi phần tử của một vectơ về khoảng $[0, 1]$. Chuẩn tắc hóa không có yêu cầu như vậy; thay vào đó nó có xu hướng làm cho đầu vào tập trung quanh giá trị 0. Theo nghĩa này, thuật ngữ *chuẩn hóa* trong học sâu phần nào là một cách gọi chưa chuẩn xác. Tuy vậy, trong cuốn sách này, chuẩn hóa và chuẩn tắc hóa được dùng thay thế cho nhau khi nói về các quá trình như Phương trình (2.43).

2.3.5 Đơn vị phần dư

Thành công của các mạng nơ-ron sâu phần lớn được cho là nhờ việc tăng số tầng dùng để tạo thành những hàm phức tạp hơn. Mặc dù xếp chồng thật nhiều tầng là cách đơn giản nhất để có một mô hình sâu, người ta đã chỉ ra rằng những mô hình như vậy khó huấn luyện. Có một số nguyên nhân, chẳng hạn vấn đề gradient biến mất (*vanishing gradient*) hoặc gradient bùng nổ (*exploding gradient*). Hơn nữa, việc đưa tuần tự một biểu diễn duy nhất qua mọi tầng có thể bất lợi khi các bộ trích xuất đặc trưng ở tầng trên thực ra có thể hưởng lợi từ việc truy cập trực tiếp các biểu diễn trung gian từ những tầng trước đó.



Hình 2.10: Minh họa một mạng nơ-ron phần dư 2 tầng. Với mỗi tầng (hoặc khối), có một kết nối bỏ qua hay kết nối tắt (màu xanh) cộng trực tiếp đầu vào vào đầu ra.

Nhãn trong hình: *Residual Connection* = kết nối tắt (residual); *Layer* = tầng.

Mạng nơ-ron phần dư (*residual neural networks*) là một trong những cách hiệu quả nhất để giải quyết các vấn đề này [He và cộng sự, 2016a]. Chúng là một loại mạng nơ-ron đặc biệt có thêm các **kết nối tắt (residual)** (*residual connections*) (hay **kết nối bỏ qua** (*skip connections*), hoặc **kết nối đường tắt** (*shortcut connections*)) xuyên qua các tầng trong một chồng tầng. Gọi $F(\mathbf{x})$ là một tầng hoặc một dãy tầng ánh xạ $\mathbf{x}$ thành một đầu ra nào đó. Một khối phần dư (*residual block*) xây dựng trên $F(\mathbf{x})$ được định nghĩa bằng cách cộng ánh xạ $F(\mathbf{x})$ với ánh xạ đồng nhất (*identity mapping*) $\mathbf{x}$:

$$\mathbf{y} = F(\mathbf{x}) + \mathbf{x}. \tag{2.46}$$

Một cách dùng phổ biến hơn của kết nối tắt là trong mạng nơ-ron gồm nhiều tầng giống hệt nhau. Gọi $\mathbf{x}^l$ và $\mathbf{y}^l$ là đầu vào và đầu ra của tầng l trong một mạng nơ-ron phần dư nhiều tầng. Đầu ra của tầng l có thể được định nghĩa là:

$$\mathbf{x}^{l+1} = F(\mathbf{x}^l) + \mathbf{x}^l, \tag{2.47}$$

hoặc

$$\mathbf{y}^l = F(\mathbf{y}^{l-1}) + \mathbf{y}^{l-1}. \tag{2.48}$$

Hình 2.10 cho thấy kiến trúc của một mạng nơ-ron phần dư 2 tầng. Rõ ràng, các kết nối tắt cộng trực tiếp đầu vào của một tầng (hoặc khối) vào đầu ra của nó. Ánh xạ đồng nhất được cộng thêm này nhìn chung được xem là một trong những cách hiệu quả nhất để đơn giản hóa việc tối ưu và làm thông suốt dòng thông tin trong một mô hình sâu.

Chú giải của người dịch: Vì sao "cộng thêm $\mathbf{x}$" lại giúp ích?

Với $\mathbf{y} = F(\mathbf{x}) + \mathbf{x}$, tầng F chỉ cần học "phần chênh lệch" (*phần dư*) giữa đầu ra mong muốn và đầu vào, chứ không phải học lại toàn bộ. Nếu một tầng là thừa, mô hình chỉ cần đẩy $F(\mathbf{x})$ về gần 0 là đầu ra giữ nguyên đầu vào. Về mặt đạo hàm (một biến): nếu $y = F(x) + x$ thì $y' = F'(x) + 1$; số hạng "+1" giúp tín hiệu gradient đi ngược qua nhiều tầng mà không bị nhân với những số nhỏ liên tiếp rồi teo dần về 0 (hiện tượng gradient biến mất).

2.4 Huấn luyện mạng nơ-ron

Trong mục này, chúng ta chuyển sang bài toán huấn luyện, một vấn đề nền tảng trong việc phát triển các hệ thống dựa trên mạng nơ-ron. Phần lớn

nội dung thảo luận ở đây tập trung vào các phương pháp trong bối cảnh học có giám sát (*supervised learning*). Chúng ta sẽ bàn về các phương pháp không giám sát ở Mục 2.6.

2.4.1 Hạ gradient

Phương pháp gradient đã được chứng minh là một trong những phương pháp thành công nhất để huấn luyện mạng nơ-ron. Ý tưởng cơ bản là cập nhật tham số một cách lặp đi lặp lại sao cho ta có thể cực tiểu hóa một hàm mất mát (*loss function*) khả vi. Trong mỗi lần cập nhật, giá trị của các tham số được điều chỉnh theo cách làm cho mất mát giảm nhanh nhất. Về mặt toán học, điều này đòi hỏi bước cập nhật phải đi theo hướng ngược với gradient của mất mát. Cách làm này được gọi là **hạ gradient** (*gradient descent*) hay **hạ dốc nhất** (*steepest descent*). Gọi θ_t là các tham số tại bước t (gọi là **bước cập nhật** (*update step*) hay **bước huấn luyện** (*training step*)), và $\mathcal{L}(\theta_t)$ là mất mát tính bởi mô hình có tham số θ_t. **Quy tắc cập nhật** (*update rule*) (hay **quy tắc delta** (*delta rule*)) của hạ gradient được cho bởi phương trình:

$$\theta_{t+1} \;=\; \theta_t - \eta \cdot \frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t}, \tag{2.49}$$

trong đó $\frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t}$ là gradient của mất mát theo các tham số tại bước t. Nó có thể thu được bằng cách chạy thuật toán lan truyền ngược (*back-propagation*) đã trình bày ở Mục 2.1.3. Vì θ_t thường có nhiều chiều, $\frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t}$ có thể là một vectơ hoặc ma trận có cùng hình dạng với θ_t. η là **tốc độ học** (*learning rate*), điều khiển độ dài bước ta đi theo hướng về điểm cực tiểu. Mặc dù có thể đặt η đơn giản là một hằng số trong suốt quá trình huấn luyện, người ta thường điều chỉnh giá trị của nó khi quá trình huấn luyện diễn ra (xem thảo luận ở Mục 2.4.4). Xem Hình 2.11 để có minh họa về hạ gradient.

Chú giải của người dịch: Hạ gradient như đi xuống thung lũng trong sương mù

Hãy tưởng tượng bạn đứng trên sườn đồi trong sương mù dày, muốn xuống đáy thung lũng. Bạn không nhìn thấy đáy, nhưng cảm nhận được độ dốc dưới chân. Cách hợp lý: bước một bước nhỏ theo hướng dốc xuống nhất,

rồi lặp lại. Gradient chỉ hướng *dốc lên* nhanh nhất, nên ta đi ngược lại (dấu trừ). Ví dụ một biến: $\mathcal{L}(\theta) = \theta^2$, $\mathcal{L}'(\theta) = 2\theta$. Bắt đầu $\theta_0 = 3$, $\eta = 0{,}1$: $\theta_1 = 3 - 0{,}1 \cdot 6 = 2{,}4$; $\theta_2 = 2{,}4 - 0{,}1 \cdot 4{,}8 = 1{,}92$; … dần tiến về cực tiểu $\theta = 0$. Nếu η quá lớn (ví dụ $\eta = 1{,}1$) thì $\theta_1 = 3 - 6{,}6 = -3{,}6$, rồi $4{,}32$, … — nhảy qua lại và xa dần cực tiểu. Ký hiệu $\frac{\partial \mathcal{L}}{\partial \theta}$ (đạo hàm riêng) là đạo hàm của $\mathcal{L}$ theo θ khi coi các biến khác là hằng số; khi θ là vectơ, ta có một đạo hàm riêng cho mỗi phần tử, xếp thành vectơ gradient.

Phương trình (2.49) đưa ra một định nghĩa rất cơ bản của hạ gradient. Có nhiều cải tiến cho dạng của Phương trình (2.49). Một số trong đó là:

- **Hạ gradient với quán tính** (*Gradient Descent with Momentum*). Trong vật lý, **động lượng** (*momentum*) là một đại lượng vectơ mô tả khối lượng đang chuyển động. Nếu ta coi việc cập nhật tham số như di chuyển một vật trong không gian, thì ta cần xét động lượng của vật tại một vị trí để xác định hướng của bước di chuyển tiếp theo. Ý tưởng này có thể được hiện thực bằng cách định nghĩa lại quy tắc cập nhật như sau:

$$\theta_{t+1} \;=\; \theta_t + v_t, \tag{2.50}$$

  trong đó v_t là vectơ vận tốc của động lượng. Trong phương pháp động lượng cổ điển [Polyak, 1964], v_t được định nghĩa là:

$$v_t \;=\; \lambda \cdot v_{t-1} - \eta \cdot \frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t}, \tag{2.51}$$

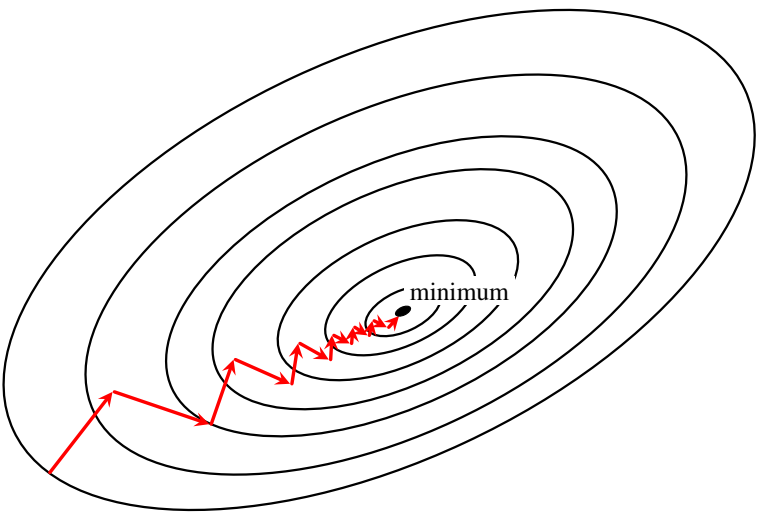
  v_t giữ lại một phần động lượng trước đó (tức là v_{t-1}), sau đó là một hiệu chỉnh dựa trên gradient (tức là $\frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t}$). λ là một số vô hướng dùng để đặt trọng số cho v_t trong một lần cập nhật. Một cải tiến nổi tiếng của Phương trình (2.51) là tính đến động lượng ngay trong gradient, và nhờ đó ngăn vận tốc trở nên quá lớn khi tiến gần điểm cực tiểu [Nesterov, 1983], như sau

$$v_t \;=\; \lambda \cdot v_{t-1} - \eta \cdot \frac{\partial \mathcal{L}(\theta_t + \lambda \cdot v_{t-1})}{\partial \theta_t}. \tag{2.52}$$

  Có thể tìm thấy thảo luận chi tiết hơn về sự khác biệt giữa Phương trình (2.51) và Phương trình (2.52) trong bài báo của Sutskever và cộng sự [2013].

Chú giải của người dịch: Quán tính: hòn bi lăn xuống dốc

Hạ gradient thường giống một người đi từng bước, còn hạ gradient với quán tính giống một hòn bi lăn: nó "nhớ" vận tốc cũ v_{t-1} (nhân với λ, thường khoảng $0{,}9$) và cộng thêm lực đẩy từ độ dốc hiện tại. Khi gradient liên tục chỉ cùng một hướng, vận tốc tích lũy và ta đi nhanh hơn; khi gradient đổi chiều liên tục (đường zig-zag như trong Hình 2.11), các thành phần ngược chiều triệt tiêu bớt nhau nên đường đi mượt hơn. Biến thể Nesterov (2.52) "nhìn trước": tính độ dốc tại điểm mà hòn bi *sắp* tới ($\theta_t + \lambda v_{t-1}$) chứ không phải điểm hiện tại.



Hình 2.11: Minh họa hạ gradient trong không gian 2 chiều (các đường màu đỏ biểu thị các tập mức). Mục tiêu là tìm các tham số (tức là các giá trị theo hai chiều) làm cực tiểu giá trị của một hàm mất mát cho trước. Hạ gradient làm điều này bằng cách bắt đầu từ một điểm ngẫu nhiên và bước dần tới điểm cực tiểu qua một số lần cập nhật tham số. Trong mỗi lần cập nhật, nó điều chỉnh các tham số θ_t theo hướng làm mất mát thấp hơn. Ý tưởng ở đây là bước cập nhật chọn hướng dốc nhất, tức là mô hình di chuyển một bước theo hướng gradient âm của mất mát (tức là $-\frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t}$). Độ dài bước di chuyển được điều khiển bởi một siêu tham số η, gọi là tốc độ học. Như vậy, lượng thay đổi của các tham số là $-\eta \cdot \frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t}$. Cộng lượng này vào θ_t, ta thu được các tham số mới θ_{t+1}. Quá trình này lặp lại qua một số lần cập nhật cho đến khi giá trị của hàm mất mát gần với cực tiểu.

Nhãn trong hình: *minimum* = điểm cực tiểu. Các đường cong khép kín là các đường mức (mọi điểm trên cùng một đường có cùng giá trị mất mát), giống như đường đồng mức trên bản đồ địa hình; các mũi tên là các bước cập nhật.

- **Hạ gradient thích nghi** (*Adaptive Gradient Descent*). Trong các phương pháp hạ gradient thích nghi, quy tắc cập nhật được điều chỉnh riêng cho từng tham số, thay vì cho toàn bộ mô hình. **AdaGrad** là một phương pháp thuộc loại này [Duchi và cộng sự, 2011]. Nó giảm tốc độ học cho các tham số được cập nhật thường xuyên và tăng tốc độ học cho các tham số ít được cập nhật. Giả sử θ_t và $\frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t}$ đều là các vectơ d chiều. Ta có thể định nghĩa một biến mới $G \in \mathbb{R}^{d \times d}$ là tổng các tích ngoài (*outer product*) của gradient qua t bước vừa qua[5]:

$$G_t \;=\; \sum_{i=1}^{t} \Big[\frac{\partial \mathcal{L}(\theta_i)}{\partial \theta_i}\Big]^{\top} \cdot \frac{\partial \mathcal{L}(\theta_i)}{\partial \theta_i}. \tag{2.54}$$

Nhìn chung, $(G_t)^{\frac{1}{2}} \in \mathbb{R}^{d \times d}$ có thể được xem như một chỉ báo mô tả mức độ một tham số đã được cập nhật cho đến thời điểm hiện tại. Tuy nhiên, việc tính $(G_t)^{\frac{1}{2}}$ cực kỳ tốn kém. Vì vậy, người ta thường dùng đường chéo của G_t thay thế. Khi đó, quy tắc cập nhật của AdaGrad được cho bởi:

$$\theta_{t+1} \;=\; \theta_t - \frac{\eta}{\sqrt{\text{diag}(G_t) + \epsilon}} \odot \frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t}, \tag{2.55}$$

trong đó $\text{diag}(G_t)$ là đường chéo của G_t, tức là $\text{diag}(G_t)(k) = G_t(k,k)$. ϵ là một hệ số làm trơn để bảo đảm ổn định số học. Thay

[5]Cho hai vectơ $\mathbf{a} = \begin{bmatrix} a_1 & \cdots & a_d \end{bmatrix}$ và $\mathbf{b} = \begin{bmatrix} b_1 & \cdots & b_d \end{bmatrix}$, tích ngoài của $\mathbf{a}$ và $\mathbf{b}$ là:

$$\begin{aligned} \mathbf{a} \otimes \mathbf{b} &= \mathbf{a}^{\top} \cdot \mathbf{b} \\ &= \begin{bmatrix} a_1 \\ \vdots \\ a_d \end{bmatrix} \cdot \begin{bmatrix} b_1 & \cdots & b_d \end{bmatrix} \\ &= \begin{bmatrix} a_1 b_1 & \cdots & a_1 b_d \\ \vdots & \ddots & \vdots \\ a_d b_1 & \cdots & a_d b_d \end{bmatrix}. \end{aligned} \tag{2.53}$$

vì cộng các bình phương gradient đó một cách không có trọng số, một cách khác là giảm ảnh hưởng của các gradient "cũ" và làm cho các gradient "gần đây" quan trọng hơn. **AdaDelta** xét đến điều này bằng cách tích lũy bình phương gradient với một **hệ số suy giảm** (*decay factor*) [Zeiler, 2012]:

$$g_t^2 \;=\; \sigma \cdot g_{t-1}^2 + (1-\sigma) \cdot \left(\frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t} \odot \frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t} \right), \qquad (2.56)$$

trong đó σ là hệ số suy giảm có giá trị $<\ 1$. Giống Phương trình (2.55), quy tắc cập nhật của AdaDelta có thể thu được bằng cách thay $\text{diag}(G_t)$ bằng g_t^2:

$$\theta_{t+1} \;=\; \theta_t - \frac{\eta}{\sqrt{g_t^2 + \epsilon}} \odot \frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t}. \qquad (2.57)$$

Vì $\sqrt{g_t^2 + \epsilon}$ có thể được xem là căn quân phương (*root mean square*) (RMS) của gradient, Phương trình (2.56–2.57) còn được gọi là phương pháp **RMSProp** [Hinton, 2018].

Chú giải của người dịch: Tốc độ học "riêng" cho từng tham số

Ý tưởng của AdaGrad/RMSProp: tham số nào đã nhận gradient lớn (hoặc thường xuyên) thì chia cho một số lớn, nên bước đi nhỏ lại; tham số nào ít được cập nhật thì chia cho số nhỏ, nên bước đi lớn hơn. Ví dụ với hai tham số, nếu gradient qua hai bước là $(4,\ 0{,}1)$ rồi $(3,\ 0{,}2)$, thì tổng bình phương (đường chéo của G_2) là $(16+9,\ 0{,}01+0{,}04) = (25;\ 0{,}05)$, căn bậc hai $\approx (5;\ 0{,}22)$. Tham số thứ nhất có tốc độ học hiệu dụng $\eta/5$, tham số thứ hai $\approx \eta/0{,}22$ — lớn hơn khoảng 22 lần. Với AdaDelta/RMSProp, thay vì cộng dồn mãi, ta lấy "trung bình trượt": giá trị cũ nhân σ (ví dụ 0,9), cộng 10% giá trị mới, nên các gradient quá xa trong quá khứ dần bị lãng quên.

- **Adam** (**Ước lượng mô-men thích nghi**, *Adaptive Moment Estimation*). Bộ tối ưu Adam kết hợp ưu điểm của cả phương pháp hạ gradient thích nghi và phương pháp động lượng [Kingma và Ba, 2014]. Nó định nghĩa một ước lượng cho trung bình của gradient (mô-men bậc nhất (*first moment*)) và một ước lượng cho phương sai không trừ tâm của gradient (mô-men gốc bậc hai

(*second raw moment*)). Gọi m_t và v_t là hai ước lượng mô-men này. Chúng được cho bởi các phương trình:

$$m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot \frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t}, \tag{2.58}$$

$$v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot \left(\frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t} \odot \frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t} \right), \tag{2.59}$$

trong đó $\beta_1, \beta_2 \in [0, 1]$ là các siêu tham số (*hyper-parameters*) dùng để cân bằng giữa ước lượng trước đó và gradient (hoặc bình phương gradient) tại bước hiện tại. β_1 và β_2 cũng được xem là hệ số suy giảm của các trung bình này. Ví dụ, các lựa chọn phổ biến cho β_1 và β_2 là 0.9 và 0.999. Vì các mô-men ban đầu được đặt bằng 0, các ước lượng này bị thiên lệch về phía vectơ không ở giai đoạn rất sớm của quá trình huấn luyện. Để giải quyết vấn đề này, Adam dùng hiệu chỉnh độ lệch (*bias correction*), dẫn tới các ước lượng đã hiệu chỉnh độ lệch:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \tag{2.60}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}. \tag{2.61}$$

Vì $\beta_1, \beta_2 < 1$, các hiệu chỉnh sẽ trở nên đủ nhỏ nếu thực hiện số lần cập nhật đủ lớn. Cuối cùng, quy tắc cập nhật được định nghĩa là:

$$\theta_{t+1} \; = \; \theta_t - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}. \tag{2.62}$$

Phương trình (2.62) giống dạng tổng quát của hạ gradient, nhưng tận dụng cả phương pháp động lượng (tức là trung bình trượt (*moving average*) của các gradient trong quá khứ) và phương pháp thích nghi (tức là trung bình trượt của bình phương các gradient trong quá khứ). Trong thực tế, Adam đã trở thành một bộ tối ưu phổ biến để huấn luyện mạng nơ-ron.

Chú giải của người dịch: Vì sao cần hiệu chỉnh độ lệch trong Adam?

Với $\beta_1 = 0{,}9$ và $m_0 = 0$, giả sử gradient ở bước 1 là $g = 2$. Khi đó $m_1 = 0{,}9 \cdot 0 + 0{,}1 \cdot 2 = 0{,}2$ — nhỏ hơn nhiều so với gradient thật là 2, chỉ vì ta "khởi đầu từ 0". Chia cho $1 - \beta_1^1 = 0{,}1$ ta được $\hat{m}_1 = 2$, đúng như mong đợi. Khi t lớn, $\beta_1^t \to 0$ (ví dụ $0{,}9^{50} \approx 0{,}005$) nên phép chia gần như không còn tác dụng. Tóm lại, Adam = hướng đi lấy từ "quán tính" ($\hat{m}_t$) + độ dài bước được điều chỉnh riêng cho từng tham số ($\sqrt{\hat{v}_t}$). Lưu ý chữ v_t ở đây là mô-men bậc hai, khác với vận tốc v_t trong Phương trình (2.50–2.52).

Cải tiến hạ gradient là một lĩnh vực con sôi động của học sâu, nhưng thảo luận đầy đủ về tất cả các kỹ thuật đó nằm ngoài phạm vi của cuốn sách này. Một vài vấn đề liên quan sẽ được bàn trong phần còn lại của mục này.

Một lưu ý cuối cùng của tiểu mục này: một vấn đề thực tế cần cân nhắc khi dùng các phương pháp huấn luyện lặp là khi nào dừng huấn luyện. **Tiêu chí dừng** (*stopping criterion*) là một chủ đề chung trong tối ưu hóa. Với hạ gradient và các biến thể của nó, thực tiễn phổ biến là đặt một số bước huấn luyện tối đa hoặc một số **epoch huấn luyện** (*training epochs*)[6] tối đa, chẳng hạn 20,000 bước, hoặc 100 epoch. Một cách khác là ta có thể huấn luyện cho đến khi hội tụ. Ví dụ, ta có thể nói quá trình huấn luyện hội tụ nếu mất mát có xu hướng ổn định qua một số bước huấn luyện. Khi có sẵn một ít dữ liệu để kiểm định mô hình, một phương pháp tốt hơn có thể là kiểm tra trạng thái của mô hình trên dữ liệu kiểm định. Ví dụ, ta có thể dừng huấn luyện khi sai số dự đoán trên dữ liệu kiểm định tăng lên. Phương pháp này, được gọi là **dừng sớm** (*early stopping*), thường được dùng như một cách chính quy hóa (*regularization*). Ở Mục 2.5.3, ta sẽ thấy chi tiết hơn về cách áp dụng dừng sớm trong quá trình huấn luyện bằng một tập kiểm định (*validation set*). Về mặt thuật toán, đã có nhiều quan tâm và nghiên cứu về sự hội tụ và các cận sai số của các phương pháp học máy. Chúng tôi giới thiệu bạn đọc quan tâm một số sách giáo khoa để thảo luận thêm [Mohri và cộng sự, 2018; Kochenderfer và Wheeler, 2019].

[6] Một epoch huấn luyện nghĩa là bộ huấn luyện duyệt qua toàn bộ tập dữ liệu huấn luyện đúng một lần.

2.4.2 Chia lô

Hàm mất mát là một khía cạnh thiết yếu trong việc huấn luyện mạng nơ-ron. Mặc dù có nhiều dạng toán học để định nghĩa hàm mất mát (xem Chương 1), ta vẫn cần quyết định quy mô các mẫu mà trên đó ta tính hàm mất mát. Có lẽ phương pháp đơn giản nhất là **hạ gradient ngẫu nhiên** (*stochastic gradient descent*) (**SGD**). Trong mỗi lần cập nhật tham số, SGD tính hàm mất mát trên một mẫu duy nhất được chọn ngẫu nhiên từ tập dữ liệu huấn luyện. Gọi $\mathcal{D}$ là một tập các mẫu huấn luyện, và $(\mathbf{x}^{(i)}, \mathbf{y}^{(i)}_{\text{gold}})$ là một mẫu được chọn ngẫu nhiên từ $\mathcal{D}$. Cho một mạng nơ-ron

$$\mathbf{y}^{(i)}_{\theta} \;=\; F_{\theta}\left(\mathbf{x}^{(i)}\right), \tag{2.63}$$

mất mát của SGD được định nghĩa là:

$$\mathcal{L}(\theta) \;=\; \mathcal{L}\left(\mathbf{y}^{(i)}_{\theta}, \mathbf{y}^{(i)}_{\text{gold}}\right), \tag{2.64}$$

trong đó $\mathcal{L}(\mathbf{y}^{(i)}_{\theta}, \mathbf{y}^{(i)}_{\text{gold}})$ là một hàm mất mát ở cấp độ mẫu, định lượng sai số của đầu ra mô hình $\mathbf{y}^{(i)}_{\theta}$ so với đáp án chuẩn (*ground truth*) $\mathbf{y}^{(i)}_{\text{gold}}$.

SGD là một trong những phương pháp tối ưu quan trọng nhất trong học máy nhờ tính đơn giản của nó. Tuy nhiên, các lần cập nhật của SGD có phương sai (*variance*) cao vì một mẫu duy nhất chỉ cho một xấp xỉ nhiễu của gradient thật. Để ước lượng gradient chính xác hơn, ta có thể xét một tập các mẫu (gọi là một **lô** (*batch*)) khi tính mất mát. Phương pháp này được gọi là **chia lô** (*batching*). Gọi S là một tập các mẫu lấy từ $\mathcal{D}$. Hàm mất mát khi đó được định nghĩa trên S như sau

$$\begin{aligned}\mathcal{L}(\theta) &= \frac{1}{|S|}\sum_{(\mathbf{x},\mathbf{y}_{\text{gold}})\in S} \mathcal{L}(F_{\theta}(\mathbf{x}), \mathbf{y}_{\text{gold}}) \\ &= \frac{1}{|S|}\sum_{(\mathbf{x},\mathbf{y}_{\text{gold}})\in S} \mathcal{L}(\mathbf{y}_{\theta}, \mathbf{y}_{\text{gold}}).\end{aligned} \tag{2.65}$$

Nếu $S = \mathcal{D}$, ta có phương pháp **hạ gradient theo toàn lô** (*batch gradient descent*) (**BGD**), tức là gradient được ước lượng trên toàn bộ tập mẫu huấn luyện. Nói chung, hạ gradient theo toàn lô chính là cái mà ta thường gọi là hạ gradient. Tuy nhiên, tính mất mát trên tất cả các mẫu huấn luyện cùng một lúc rất tốn thời gian. Trong thực tế, người ta thường

dùng một lô nhỏ hơn $\mathcal{D}$ rất nhiều. Cách này được gọi là **hạ gradient theo lô nhỏ** (*mini-batch gradient descent*). Nó được dùng để huấn luyện các hệ thống thực tế nhờ hiệu quả tính toán tốt và hiệu năng cao.

Chú giải của người dịch: Một mẫu, một lô nhỏ hay toàn bộ dữ liệu?

Hãy hình dung bạn muốn biết điểm trung bình môn Toán của cả trường 1000 học sinh để quyết định "điều chỉnh" phương pháp dạy. Hỏi *một* học sinh ngẫu nhiên (SGD) thì rất nhanh nhưng kết quả dao động mạnh; hỏi *cả* 1000 em (BGD) thì chính xác nhưng tốn công; hỏi một nhóm 32 em ngẫu nhiên (lô nhỏ) là cách dung hòa: khá chính xác mà vẫn nhanh. Gradient tính trên lô nhỏ chính là "trung bình mẫu" của gradient từng mẫu, như trong Phương trình (2.65).

Như một "phần thưởng" khác, chia lô thường được dùng như một cách để thực hiện tính toán dày đặc trên ma trận nhằm tăng tốc hệ thống. Giả sử S gồm m mẫu $\{(\mathbf{x}^{(i_1)}, \mathbf{y}_{\text{gold}}^{(i_1)}), \ldots, (\mathbf{x}^{(i_m)}, \mathbf{y}_{\text{gold}}^{(i_m)})\}$. Ta có thể gộp tất cả các vectơ đầu vào và các vectơ đáp án chuẩn thành các ma trận:

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}^{(i_1)} \\ \vdots \\ \mathbf{x}^{(i_m)} \end{bmatrix}, \tag{2.66}$$

$$\mathbf{Y}_{\text{gold}} = \begin{bmatrix} \mathbf{y}_{\text{gold}}^{(i_1)} \\ \vdots \\ \mathbf{y}_{\text{gold}}^{(i_m)} \end{bmatrix}. \tag{2.67}$$

Sau đó, ta có thể chạy mạng nơ-ron trên đầu vào và đầu ra đã gộp lô, như sau:

$$\mathbf{Y}_\theta = F_\theta(\mathbf{X}). \tag{2.68}$$

Tương tự, ta có thể tính mất mát theo lô

$$\mathcal{L}(\theta) = \frac{1}{m} \cdot \mathcal{L}(\mathbf{Y}_\theta, \mathbf{Y}_{\text{gold}}), \tag{2.69}$$

trong đó $\mathcal{L}(\mathbf{Y}_\theta, \mathbf{Y}_{\text{gold}})$ vectơ hóa phép tính $\sum_{k=1}^{m} \mathcal{L}\left(\mathbf{y}_\theta^{(i_k)}, \mathbf{y}_{\text{gold}}^{(i_k)}\right)$. Phương trình (2.68–2.69) tránh việc gọi lặp lại lượt truyền xuôi (*forward pass*) và lượt truyền ngược (*backward pass*) trên từng mẫu riêng lẻ. Thay vào đó,

chúng gói mọi thứ vào một lượt duy nhất qua mạng. Điều này tận dụng tốt hơn năng lực tính toán tối đa sẵn có trên các GPU hiện đại, vốn là loại thiết bị chủ yếu để chạy các hệ thống học sâu.

2.4.3 Khởi tạo tham số

Hạ gradient đòi hỏi quá trình huấn luyện bắt đầu từ một bộ tham số ban đầu nào đó. Vì hàm mục tiêu (*objective function*) huấn luyện trong một hệ thống thực tế thường là một hàm không lồi ở mức độ cao với nhiều cực tiểu địa phương (*local minima*), hiệu năng của mô hình thu được cực kỳ nhạy cảm với bước khởi tạo tham số. Ở đây, chúng tôi mô tả một số phương pháp khởi tạo tham số phổ biến nhất.

Chú giải của người dịch: Hàm lồi và cực tiểu địa phương

Hàm $y = x^2$ là hàm *lồi*: đồ thị hình "cái bát", chỉ có một đáy, nên đi xuống dốc từ đâu cũng tới cùng một điểm. Hàm như $y = x^4 - 3x^2 + x$ thì có hai "hõm" với độ sâu khác nhau: xuất phát ở bên nào sẽ rơi vào hõm bên đó. Hõm nông hơn gọi là cực tiểu địa phương. Hàm mất mát của mạng nơ-ron có vô số "hõm" như vậy trong không gian hàng triệu chiều, nên điểm xuất phát (khởi tạo) ảnh hưởng lớn đến kết quả.

- **Khởi tạo hằng số** (*Constant Initialization*). Phương pháp thứ nhất là gán cùng một giá trị cho mọi tham số (hoặc mọi phần tử của một ma trận tham số). Phương pháp này tuy đơn giản nhưng khiến mọi đơn vị ẩn được cập nhật giống hệt nhau trong quá trình huấn luyện. Kết quả là mạng không thể học được các khuôn mẫu phức tạp. Nó thường cho kết quả kém trong thực tế trừ khi có đưa vào một yếu tố ngẫu nhiên nào đó trong kiến trúc.

- **Khởi tạo theo các phân phối định trước** (*Initialization with Predefined Distributions*). Một cách tiếp cận thực tế hơn là khởi tạo ngẫu nhiên các tham số bằng các phân phối xác suất (*probability distributions*) được định trước. Phương pháp đơn giản nhất loại này là lấy giá trị tham số từ một phân phối đều hoặc phân phối Gauss, chẳng hạn gán một giá trị ngẫu nhiên trong khoảng $[-0.1, 0.1]$. Điều thú vị là cách làm theo kinh nghiệm này lại đủ tốt cho nhiều ứng dụng thực tế cơ bản.

- **Khởi tạo theo tầng** (*Layer-sensitive Initialization*). Một mở rộng của khởi tạo ngẫu nhiên cơ bản là dùng các phân phối được thiết kế riêng dựa trên số chiều của các tầng mạng nơ-ron khác nhau. Cho một tầng $\mathbf{y} = \psi(\mathbf{x} \cdot \mathbf{W} + \mathbf{B})$, gọi d_{in} và d_{out} lần lượt là số đầu vào (*fan-in*) và số đầu ra (*fan-out*) (tức là số hàng và số cột của $\mathbf{W}$). Phương pháp **khởi tạo LeCun** (*LeCun initialization*) [LeCun và cộng sự, 2012] gán một số ngẫu nhiên cho mọi tham số của $\mathbf{W}$ chỉ dựa trên số chiều đầu vào:

$$\mathbf{W} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}} \sim U\left(-\frac{1}{\sqrt{d_{\text{in}}}}, \frac{1}{\sqrt{d_{\text{in}}}}\right), \tag{2.70}$$

trong đó $U(-a, a)$ ký hiệu phân phối đều trên đoạn $[-a, a]$. Tương tự, số hạng độ lệch (bias) cũng có thể được khởi tạo theo cách tương tự. Dựa trên ý tưởng này, phương pháp **khởi tạo Xavier** (*Xavier initialization*) chuẩn (còn gọi là khởi tạo Glorot) xét cả d_{in} và d_{out} để ổn định phương sai của các giá trị kích hoạt và gradient qua các tầng [Glorot và Bengio, 2010]:

$$\mathbf{W} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}} \sim U\left(-\sqrt{\frac{6}{d_{\text{in}} + d_{\text{out}}}}, \sqrt{\frac{6}{d_{\text{in}} + d_{\text{out}}}}\right). \tag{2.71}$$

Lưu ý rằng các phân phối đều này có thể dễ dàng được thay bằng các phân phối chuẩn có trung bình 0 và phương sai $\frac{1}{d_{\text{in}}}$ (với khởi tạo LeCun) hoặc $\frac{2}{d_{\text{in}}+d_{\text{out}}}$ (với khởi tạo Xavier).

Chú giải của người dịch: Vì sao khoảng khởi tạo phụ thuộc vào d_{in}?

Mỗi đầu ra của tầng là tổng của d_{in} số hạng dạng $x_k \cdot W_{kj}$. Cộng càng nhiều số hạng ngẫu nhiên thì tổng càng "dao động" mạnh (phương sai của tổng các số ngẫu nhiên độc lập bằng tổng các phương sai). Nếu không thu nhỏ trọng số, sau vài chục tầng các giá trị có thể phình to hoặc teo nhỏ theo cấp số nhân. Chọn khoảng tỉ lệ với $1/\sqrt{d_{\text{in}}}$ giữ cho độ lớn đầu ra cỡ tương đương độ lớn đầu vào. Ví dụ, với $d_{\text{in}} = 100$ và $d_{\text{out}} = 200$: LeCun lấy trọng số trong $[-0{,}1;\ 0{,}1]$, còn Xavier lấy trong $[-\sqrt{6/300};\ \sqrt{6/300}] \approx [-0{,}14;\ 0{,}14]$. Ký hiệu "$\sim$" đọc là "được lấy mẫu từ phân phối".

Nhiều phương pháp khởi tạo tham số được thiết kế cho những loại mạng nơ-ron nhất định. Ví dụ, khởi tạo Xavier được suy ra với giả định

dùng các hàm kích hoạt đối xứng, như Sigmoid và tang hyperbolic. Với ReLU, có thể tham khảo công trình của He và cộng sự [2015]. Một ví dụ khác là khởi tạo cho các mạng nơ-ron sâu. Người ta thấy rằng khởi tạo phù hợp là yếu tố then chốt cho thành công của những mô hình cực sâu trong NLP. Xét độ sâu của mô hình như một yếu tố bổ sung khi khởi tạo, ta có thể sửa Phương trình (2.71) thành:

$$\mathbf{W} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}} \sim U\left(-\frac{\alpha_s}{l} \cdot \sqrt{\frac{6}{d_{\text{in}} + d_{\text{out}}}}, \frac{\alpha_s}{l} \cdot \sqrt{\frac{6}{d_{\text{in}} + d_{\text{out}}}}\right), \tag{2.72}$$

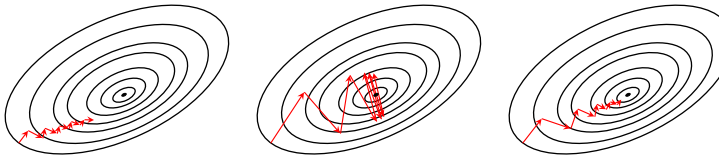
trong đó l là độ sâu của một tầng, và α_s là một siêu tham số. Ngoài ra, một số phương pháp đã được đề xuất để giải quyết việc khởi tạo cho mạng nơ-ron sâu, bao gồm khởi tạo Lipschitz [Xu và cộng sự, 2020], khởi tạo T-Fixup [Huang và cộng sự, 2020a], khởi tạo Admin [Liu và cộng sự, 2020c], v.v.

Lưu ý cuối cùng, trong thực tế, ta không nhất thiết giới hạn việc huấn luyện ở một điểm xuất phát duy nhất. Người ta thường thực hiện nhiều lần thử với các phương pháp khởi tạo hoặc hạt giống ngẫu nhiên (*random seeds*) khác nhau, rồi chọn mô hình tốt nhất trong các lần thử đó. Chiến lược này đặc biệt hữu ích khi "địa hình" tối ưu có rất nhiều cực tiểu địa phương.

2.4.4 Lập lịch tốc độ học

Để đạt được kết quả mong muốn, cần cấu hình cẩn thận tốc độ học trong suốt quá trình huấn luyện. Mặc dù một số quy tắc cập nhật nêu trên đã xét đến việc co giãn gradient cho các tham số khác nhau, **lập lịch tốc độ học** (*learning rate scheduling*) theo truyền thống tập trung nhiều hơn vào việc thiết kế các quy tắc kinh nghiệm để điều chỉnh η qua các bước huấn luyện. Trong thực tế, tốc độ học quá lớn thường dẫn tới việc "vượt quá" quanh điểm cực tiểu, trong khi tốc độ học quá nhỏ dẫn tới hội tụ chậm một cách khổ sở (xem Hình 2.12). Một chiến lược phổ biến là bước những bước lớn lúc đầu (tức là dùng tốc độ học lớn) và những bước nhỏ hơn khi mất mát tiến gần cực tiểu. Dưới đây, chúng tôi trình bày một số phương pháp lập lịch tốc độ học phổ biến.

Nhãn trong hình: (a) tốc độ học nhỏ; (b) tốc độ học lớn; (c) tốc độ học hợp lý.



(a) A small learning rate (b) A large learning rate (c) A desirable learning rate

Hình 2.12: Minh họa việc học với các tốc độ học khác nhau. Tốc độ học nhỏ (trái) giúp ta bước về phía cực tiểu một cách chính xác, nhưng cần nhiều thời gian hơn đáng kể để hội tụ. Tốc độ học lớn (giữa) dẫn tới những cập nhật ban đầu nhanh, điều này rất có lợi khi còn ở xa cực tiểu. Tuy nhiên, khi tiến gần tới cực tiểu, tốc độ học quá lớn gây ra hiện tượng vượt quá. Một chiến lược đáng mong muốn hơn (phải) là cập nhật mô hình nhanh khi còn xa sự hội tụ, và làm chậm các cập nhật khi gần hội tụ.

- **Tốc độ học cố định** (*Fixed Learning Rates*). Cố định tốc độ học nhìn chung là một chiến lược chưa tối ưu, nhưng có thể được dùng khi làm nguyên mẫu hệ thống, chẳng hạn khi thử nhanh một phương pháp mới bằng cách chỉ huấn luyện vài epoch.

- **Suy giảm tốc độ học** (*Learning Rate Decay*). Suy giảm là một kỹ thuật thường dùng để lập lịch tốc độ học. Có nhiều cách hiện thực ý tưởng này. Ví dụ, ta có thể giảm một nửa tốc độ học sau mỗi epoch huấn luyện. Ở đây ta dùng n_t để biểu diễn bước huấn luyện hiện tại, và n_{decay} để biểu diễn tần suất thay đổi tốc độ học (ví dụ 100 bước). Bảng 2.2 trình bày một số hàm suy giảm dùng cho lập lịch tốc độ học.

- **Khởi động và suy giảm** (*Warmup and Decay*). Như đã nói ở trên, người ta thường khởi tạo tham số mô hình bằng các giá trị ngẫu nhiên trước khi huấn luyện một mạng nơ-ron. Tuy nhiên, bắt đầu từ con số không với một tốc độ học lớn thường không được khuyến khích, vì gradient ở các giai đoạn đầu của huấn luyện rất nhiễu và trạng thái của mô hình chưa ổn định. Vì vậy, hợp lý hơn là bắt đầu với một tốc độ học nhỏ và tăng dần nó lên. Sau đó, khi mô hình đã được huấn luyện một thời gian, tốc độ học bắt đầu suy giảm như thường lệ. Trực giác này là động lực cho phương pháp khởi động (*warmup*) và suy giảm. Một cách xây dựng phổ biến của phương

pháp này được Vaswani và cộng sự [2017] đề xuất như sau:

$$\eta_{n_t} \;=\; \eta_0 \cdot \min\left(\left(\frac{n_t}{n_{\text{decay}}}\right)^{-0.5},\; \frac{n_t}{n_{\text{decay}}} \cdot (n_{\text{warmup}})^{-1.5}\right), \quad (2.73)$$

trong đó η_0 là tốc độ học ban đầu, và n_{warmup} là một siêu tham số chỉ định số bước của giai đoạn khởi động. Hình 2.13 vẽ đường cong của Phương trình (2.73) với n_{warmup}, n_{decay} và η_0 lần lượt được đặt là 4,000, 1 và 1. Ta thấy tốc độ học tăng tuyến tính trong n_{warmup} bước đầu tiên rồi suy giảm theo hàm nghịch đảo căn bậc hai.

Bảng 2.2: Các hàm suy giảm. λ = tốc độ suy giảm, η_0 = tốc độ học ban đầu, và $\{\beta_i\}$, $\{\gamma_i\}$ và α = các siêu tham số khác.

Tên	Công thức	Siêu tham số
Suy giảm hằng số từng đoạn	$\eta_{n_t} = \beta_i$ nếu $\gamma_i \le \frac{n_t}{n_{\text{decay}}} < \gamma_{i+1}$	các giá trị $\{\beta_1, \ldots, \beta_m\}$ các ngưỡng $\{\gamma_1, \ldots, \gamma_m\}$
Suy giảm mũ	$\eta_{n_t} = \eta_0 \cdot \lambda^{\frac{n_t}{n_{\text{decay}}}}$	tốc độ suy giảm λ, tốc độ ban đầu η_0
Suy giảm mũ (bậc thang)	$\eta_{n_t} = \eta_0 \cdot \lambda^{\lfloor \frac{n_t}{n_{\text{decay}}} \rfloor}$	tốc độ suy giảm λ, tốc độ ban đầu η_0
Suy giảm mũ tự nhiên	$\eta_{n_t} = \eta_0 \cdot \exp(-\lambda \cdot \frac{n_t}{n_{\text{decay}}})$	tốc độ suy giảm λ, tốc độ ban đầu η_0
Suy giảm nghịch đảo thời gian	$\eta_{n_t} = \eta_0 \cdot \frac{1}{1+\lambda \cdot \frac{n_t}{n_{\text{decay}}}}$	tốc độ suy giảm λ, tốc độ ban đầu η_0
Suy giảm nghịch đảo thời gian (bậc thang)	$\eta_{n_t} = \eta_0 \cdot \frac{1}{1+\lambda \cdot \lfloor \frac{n_t}{n_{\text{decay}}} \rfloor}$	tốc độ suy giảm λ
Suy giảm cosin	$\eta_{n_t} = \eta_0 \cdot \left((1-\alpha) \cdot c_{\text{decay}} + \alpha\right)$ $c_{\text{decay}} = \frac{1}{2} \cdot \left(1 + \cos(\pi \cdot \frac{n_t}{n_{\text{decay}}})\right)$	hệ số α tốc độ ban đầu η_0

Ghi chú: "(bậc thang)" dịch từ *(Drop)* trong bản gốc, chỉ các biến thể mà tốc độ học giảm theo từng nấc nhờ phép lấy phần nguyên $\lfloor \cdot \rfloor$.

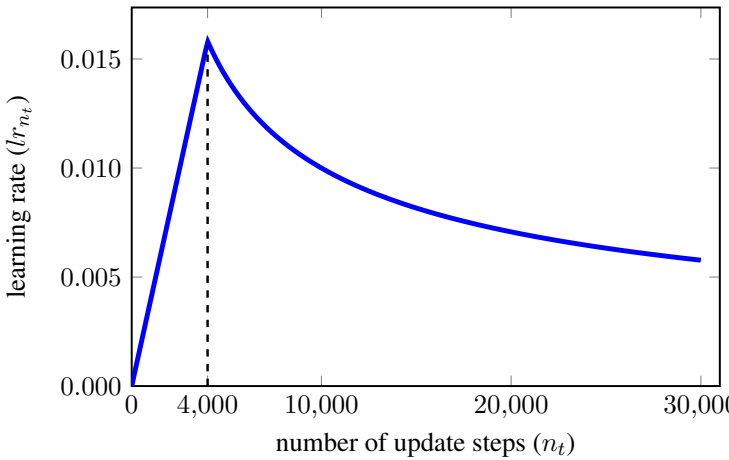
Nhãn trong hình: trục tung *learning rate* = tốc độ học; trục hoành *number of update steps* = số bước cập nhật.

Chú giải của người dịch: Tính thử Phương trình (2.73)

Với $n_{\text{warmup}} = 4000$, $n_{\text{decay}} = 1$, $\eta_0 = 1$, ta có $\eta_{n_t} = \min\left(n_t^{-0,5},\; n_t \cdot 4000^{-1,5}\right)$.

- Tại $n_t = 1000$: $1000^{-0,5} \approx 0{,}0316$, còn $1000 \cdot 4000^{-1,5} \approx 0{,}0040$ $\Rightarrow$ lấy $0{,}0040$ (đang ở pha tăng tuyến tính).
- Tại $n_t = 4000$: hai số bằng nhau, cùng bằng $4000^{-0,5} \approx 0{,}0158$ — đây là đỉnh trong Hình 2.13.
- Tại $n_t = 16000$: $16000^{-0,5} \approx 0{,}0079$, nhỏ hơn số thứ hai $\approx 0{,}0632$ $\Rightarrow$ lấy $0{,}0079$ (pha suy giảm).

Hàm min tự động "chuyển pha" tại $n_t = n_{\text{warmup}}$. Lưu ý: $a^{-0,5} = \frac{1}{\sqrt{a}}$ và



Hình 2.13: Lập lịch tốc độ học: khởi động rồi suy giảm ($n_{\text{warmup}} = 4,000$, $n_{\text{decay}} = 1$ và $\eta_0 = 1$). Tốc độ học tăng tuyến tính theo n_t trong 4,000 bước đầu tiên. Sau đó, tốc độ học đi theo một hàm nghịch đảo căn bậc hai và suy giảm khi quá trình huấn luyện tiếp diễn. Sự thay đổi của tốc độ học sẽ nhỏ nếu n_t đủ lớn, thể hiện việc tinh chỉnh nhẹ các tham số khi ta tiến gần cực tiểu của mất mát.

$a^{-1,5} = \frac{1}{a\sqrt{a}}$.

Chọn một chiến lược lập lịch tốc độ học phù hợp là một bài toán mang tính thực nghiệm cao, và không có lựa chọn nào tối ưu cho mọi trường hợp. Bài toán thậm chí còn phức tạp hơn khi xét đến mối tương quan giữa tốc độ học và các yếu tố động khác của quá trình huấn luyện, mặc dù lập lịch tốc độ học thường được coi là một nhiệm vụ riêng biệt. Ví dụ, khi dùng kích thước lô lớn hơn trong huấn luyện, thường cần một tốc độ học lớn hơn để đạt kết quả tốt [Ott và cộng sự, 2018b; Smith và cộng sự, 2018]. Do đó, chọn tốc độ học tối ưu vẫn là việc khó và tốn thời gian trong các ứng dụng mạng nơ-ron. Đôi khi, ta phải dựa vào rất nhiều lần chạy thử-và-sai để tìm được một thiết lập tốc độ học phù hợp cho bài toán cụ thể đang xét.

2.5 Các phương pháp chính quy hóa

Bây giờ chúng ta bàn về các phương pháp chính quy hóa (*regularization*) nhằm ngăn chặn hiện tượng quá khớp (*overfitting*). Chính quy hóa là một chủ đề rất rộng trong học máy; ở đây chúng tôi chỉ trình bày một số phương pháp thường được dùng khi huấn luyện mạng nơ-ron.

2.5.1 Phạt dựa trên chuẩn

Một trong những phương pháp phổ biến nhất là dùng một số hạng chính quy hóa dựa trên chuẩn l_p (l_p *norm*). Dạng tổng quát của hàm mục tiêu có chính quy hóa có thể được định nghĩa như sau:

$$\hat{\theta} = \arg\min_{\theta} \mathcal{L}(\theta) + \alpha \cdot R(\theta), \tag{2.74}$$

trong đó $R(\theta)$ là số hạng chính quy hóa (*regularization term*), được nhân với hệ số trọng số α. Nói chung, $R(\theta)$ đóng vai trò như một khoản mất mát bổ sung nhằm phạt các mô hình phức tạp. Động cơ của cách làm này là: các mô hình phức tạp dễ quá khớp dữ liệu hơn (xem Chương 1). Để phạt độ phức tạp của mô hình, một cách trực tiếp là định nghĩa $R(\theta)$ là

chuẩn l_1 của vectơ tham số θ. Hàm phạt dựa trên chuẩn l_1 có dạng

$$R(\theta) = \sum_i |\theta_i|. \tag{2.75}$$

Phương trình (2.75) phạt những mô hình có giá trị tham số rất lớn. Có thể hiểu điều này một cách trực quan qua phép so sánh với hàm đa thức: hệ số của các biến trong một hàm đa thức càng lớn thì đường cong càng phức tạp. Thông thường, chính quy hóa với chuẩn l_1 được gọi là chính quy hóa l_1 hay **chính quy hóa Lasso** (*Lasso regularization*). Phương pháp này không đòi hỏi thay đổi căn bản nào trong thuật toán tối ưu và có thể cài đặt bằng hạ gradient tiêu chuẩn. Thú vị hơn, chính quy hóa l_1 thường cho ra các nghiệm thưa (*sparse*) đối với hàm mục tiêu huấn luyện ban đầu. Nó buộc nhiều tham số phải bằng đúng 0, chỉ để lại một tập nhỏ các đặc trưng quan trọng có trọng số khác 0. Điều này cũng hàm ý một khả năng chọn lọc đặc trưng (*feature selection*) có sẵn, vì tham số của những đặc trưng kém quan trọng bị ép về 0.

Một lựa chọn thay thế cho chính quy hóa l_1 là **chính quy hóa** l_2 (l_2 *regularization*), còn được biết đến rộng rãi với tên **chính quy hóa Ridge** (*Ridge regularization*). Trong chính quy hóa l_2, số hạng chính quy hóa là bình phương chuẩn l_2 của các tham số

$$R(\theta) = \sum_i \theta_i^2. \tag{2.76}$$

Giống như chuẩn l_1, chuẩn l_2 phạt những vectơ tham số lệch xa gốc tọa độ. Tuy nhiên, nó hơi khác chuẩn l_1 ở chỗ chuẩn l_2 buộc mọi tham số đều có giá trị nhỏ (nhưng không nhất thiết bằng 0) và không có tham số nào có giá trị lớn. Theo nghĩa này, việc dùng chuẩn l_2 không đưa tính thưa chặt chẽ vào nghiệm, mà đúng hơn là tạo ra hiệu ứng "làm trơn" lên phân bố của các đặc trưng. Vì vậy, đôi khi nó được gọi là **suy giảm trọng số** (*weight decay*) để nhấn mạnh khả năng liên tục thu nhỏ giá trị tham số trong các bước cập nhật gradient.

Chú giải của người dịch: Chính quy hóa l_1 và l_2 qua ví dụ số

Phương trình (2.74) giống như một "bài thi có trừ điểm": ta muốn mất mát $\mathcal{L}(\theta)$ nhỏ, nhưng mỗi khi tham số "phình to" thì bị cộng thêm khoản phạt $\alpha \cdot R(\theta)$. Hệ số α là mức nghiêm khắc của khoản phạt: $\alpha = 0$ là không phạt,

α càng lớn thì mô hình càng bị ép giữ tham số nhỏ.
Ví dụ với $\theta = (3, -0.5, 0)$: chuẩn l_1 cho $R = |3| + |-0.5| + |0| = 3.5$; bình phương chuẩn l_2 cho $R = 9 + 0.25 + 0 = 9.25$. Để ý rằng l_2 phạt tham số lớn (số 3) rất nặng (thành 9) nhưng phạt tham số nhỏ (0.5) rất nhẹ (thành 0.25). Vì thế l_2 thích "nhiều tham số nhỏ" hơn là "vài tham số lớn", nhưng ít có động lực đẩy một tham số vốn đã nhỏ về đúng 0. Ngược lại, với l_1, giảm một tham số từ 0.5 về 0 vẫn bớt được 0.5 tiền phạt, nên nhiều tham số bị đẩy hẳn về 0 — đó là nguồn gốc của tính "thưa".
Về tên gọi "suy giảm trọng số": đạo hàm của $\alpha\theta_i^2$ theo θ_i là $2\alpha\theta_i$, nên mỗi bước hạ gradient trừ bớt một phần tỉ lệ với chính θ_i, tức là nhân tham số với một số hơi nhỏ hơn 1 — tham số cứ thế "co lại" dần.

Theo một nghĩa rộng hơn trong học máy, Phương trình (2.74) cung cấp một khuôn khổ tổng quát để đưa tri thức tiên nghiệm (*prior knowledge*) vào quá trình huấn luyện mạng nơ-ron. Có rất nhiều cách thiết kế số hạng chính quy hóa, và giảm quá khớp chỉ là một trong những mục tiêu của các thiết kế đó. Chúng ta sẽ gặp nhiều ứng dụng khác nhau của cách tiếp cận này trong NLP, và sẽ tìm hiểu một vài ví dụ trong các chương còn lại của cuốn sách.

2.5.2 Dropout

Trong một mạng nơ-ron thực tế, một tầng thường gồm hàng trăm hoặc hàng nghìn nơ-ron và tạo ra một vectơ đặc trưng tương ứng. Mặc dù mỗi đặc trưng được tính bởi một nơ-ron riêng, chúng cùng nhau tạo thành đầu vào cho từng nơ-ron của tầng tiếp theo. Do đó, một đặc trưng bị buộc phải hợp tác với các đặc trưng khác. Điều này giống như một nhóm người ngồi cùng nhau và đưa ra một quyết định tập thể. Dù mỗi thành viên có thể có ý kiến độc lập, đôi khi họ cố gắng sửa những lỗi do phần còn lại của nhóm gây ra. Trong trường hợp này, mỗi thành viên trong nhóm đều *đồng thích nghi* với những người khác [Hinton và cộng sự, 2012]. Từ góc độ thiết kế đặc trưng, sự **đồng thích nghi** (*co-adaptation*) giữa các nơ-ron có ích khi mô hình hóa các bài toán phức tạp, vì nó ngầm tạo ra một dạng đặc trưng bậc cao nào đó. Ngoài ra, thông tin giám sát mạnh (ví dụ, lan truyền sai số qua các tầng) có thể củng cố sự đồng thích nghi trong quá trình huấn luyện. Điều này phần nào giải thích vì sao một mạng nơ-ron với số lượng lớn nơ-ron có thể khớp những đường cong phức tạp. Tuy nhiên, tại thời

điểm kiểm tra, sự đồng thích nghi lại cản trở khả năng tổng quát hóa. Vì mọi nơ-ron của một tầng đã học cách phối hợp thật ăn ý trên dữ liệu huấn luyện, một thay đổi nhỏ ở đầu vào có thể ảnh hưởng tới tất cả các nơ-ron này và dẫn đến một thay đổi lớn trong hành vi của mạng nơ-ron.

Một cách để giảm bớt hoặc loại bỏ những sự đồng thích nghi phức tạp là cho mỗi nơ-ron học cách dự đoán khi vắng mặt các nơ-ron khác. Để làm vậy, ta chỉ cần bỏ đi một số nơ-ron trong lúc huấn luyện. Phương pháp này được gọi là **dropout** (*dropout*) [Srivastava và cộng sự, 2014]. Gọi n là số nơ-ron của một tầng. Cho trước một xác suất ρ (gọi $1-\rho$ là **tỉ lệ dropout** (*dropout rate*)), ta có thể sinh ra một vectơ mặt nạ (*mask vector*) n chiều $\mathbf{M}_{\text{drop}}$, trong đó mỗi phần tử được đặt bằng 1 với xác suất ρ và bằng 0 với xác suất $1-\rho$. Khi đó, một tầng dropout có thể được định nghĩa là

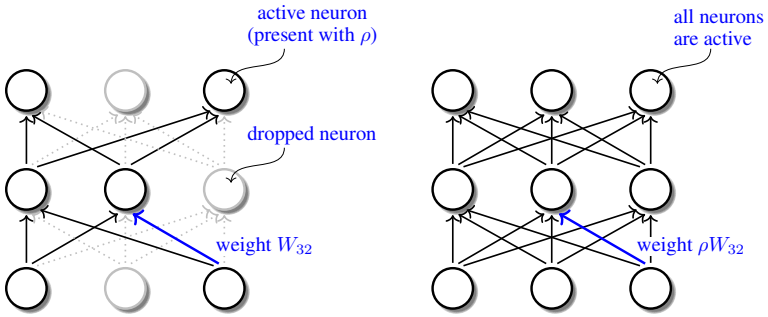
$$\mathbf{y} \;=\; \mathbf{M}_{\text{drop}} \odot \psi(\mathbf{x}\cdot\mathbf{W}+\mathbf{B}), \tag{2.77}$$

trong đó $\psi(\mathbf{x}\cdot\mathbf{W}+\mathbf{B})$ là một mạng nơ-ron một tầng thông thường. Phương trình (2.77) chỉ kích hoạt những nơ-ron có mặt nạ bằng 1. Với các nơ-ron bị bỏ, mọi kết nối đi tới hoặc đi ra từ chúng đều bị chặn (xem Hình 2.14 (a)). Trong quá trình huấn luyện, $\mathbf{M}_{\text{drop}}$ được sinh ngẫu nhiên ở mỗi lần gọi lượt truyền xuôi và lượt truyền ngược. Nhờ đó, mỗi lần một nơ-ron có thể học cách làm việc với những nơ-ron khác nhau và sẽ không thích nghi với cùng một nhóm "đồng nghiệp" cố định.

Nhãn trong hình: (a) huấn luyện mạng dropout — *active neuron (present with ρ)*: nơ-ron hoạt động (có mặt với xác suất ρ); *dropped neuron*: nơ-ron bị bỏ; *weight* W_{32}: trọng số W_{32}. (b) kiểm tra với mạng dropout — *all neurons are active*: mọi nơ-ron đều hoạt động; *weight* ρW_{32}: trọng số ρW_{32}.

Một cách khác để hiểu dropout là xem nó như việc học các mô hình con của một mô hình "lớn". Việc dùng Phương trình (2.77) về bản chất là một quá trình lấy mẫu, trích ra một mạng con từ mạng ban đầu. Vì vậy, huấn luyện với dropout giống như huấn luyện một tổ hợp mô hình (*ensemble*) gồm số lượng lớn theo cấp số mũ các mạng con[7]. Mặt khác, việc huấn luyện vẫn hiệu quả vì các mạng con này dùng chung tham số cho cùng một nơ-ron, và việc cập nhật một tham số có thể mang lại lợi ích cho số lượng lớn theo cấp số mũ các mạng con.

[7] Với một mạng một tầng có n nơ-ron, có 2^n mạng con khả dĩ.



(a) Training the dropout network

(b) Testing with the dropout network

Hình 2.14: Minh họa dropout cho một mạng nơ-ron nhiều tầng (huấn luyện so với kiểm tra). Khi huấn luyện, mỗi nơ-ron bị bỏ ngẫu nhiên với xác suất $1 - \rho$, tạo ra một mạng "gầy" hơn. Theo nghĩa này, huấn luyện dropout về bản chất là quá trình học một số lượng lớn theo cấp số mũ các mạng con. Khi kiểm tra, mạng đầy đủ được dùng như bình thường, đó là kết quả của việc kết hợp tất cả các mạng con ấy để dự đoán. Vì mọi kết nối giữa các nơ-ron chỉ được kích hoạt với xác suất ρ trong lúc huấn luyện, các trọng số của mạng suy luận được thu nhỏ theo hệ số ρ.

Chú giải của người dịch: Mặt nạ dropout hoạt động thế nào?

Ký hiệu $\odot$ là phép nhân *theo từng phần tử*: nhân hai vectơ cùng độ dài, vị trí nào nhân với vị trí đó. Giả sử một tầng có 4 nơ-ron cho đầu ra $(0.8,\ 1.2,\ 0.3,\ 2.0)$ và $\rho = 0.5$. Ở một bước huấn luyện, ta "tung đồng xu" cho từng nơ-ron và được mặt nạ $\mathbf{M}_{\text{drop}} = (1,\ 0,\ 1,\ 0)$; khi đó $\mathbf{y} = (0.8,\ 0,\ 0.3,\ 0)$: nơ-ron thứ 2 và thứ 4 coi như "nghỉ học" ở bước này. Bước sau, mặt nạ được tung lại, chẳng hạn $(0, 1, 1, 1)$, và một nhóm khác lại nghỉ. Vì không bao giờ chắc đồng đội nào có mặt, mỗi nơ-ron phải tự học những đặc trưng có ích, thay vì dựa dẫm vào vài đồng đội quen thuộc. Với 4 nơ-ron, có $2^4 = 16$ mặt nạ khả dĩ, tức 16 mạng con khác nhau dùng chung bộ trọng số.

Khi kiểm tra, tất cả các mạng con này được kết hợp lại để dự đoán. Trong trường hợp này, ta không cần bỏ nơ-ron nào mà dùng mạng ban đầu như bình thường. Điều này khiến dropout rất dễ cài đặt: trên dữ liệu huấn luyện, một nơ-ron có mặt với một xác suất nào đó, còn trên dữ liệu kiểm tra, tất cả các nơ-ron đều có mặt và cùng nhau làm việc. Vì các kết

nối giữa các nơ-ron chỉ tham gia với xác suất ρ trong lúc huấn luyện, các trọng số đã học được thu nhỏ theo hệ số ρ trong mạng suy luận, tức là một tầng có dạng:

$$\mathbf{y} \;=\; \psi(\mathbf{x}\cdot\rho\mathbf{W}+\rho\mathbf{B}), \tag{2.78}$$

trong đó các trọng số $\mathbf{W}$ và $\mathbf{B}$ được nhân với hệ số ρ[8].

Xem Hình 2.14 để so sánh việc huấn luyện và áp dụng một mạng dropout. Phương trình (2.78) đòi hỏi phải cập nhật hệ thống suy luận. Một cách khác là chỉ xử lý vấn đề co giãn trong quá trình huấn luyện và giữ nguyên hệ thống suy luận. Ví dụ, ta có thể phóng to mọi tham số theo hệ số $\frac{1}{\rho}$ khi huấn luyện dropout, như sau

$$\mathbf{y} \;=\; \mathbf{M}_{\text{drop}}\odot\psi\Big(\mathbf{x}\cdot\frac{1}{\rho}\mathbf{W}+\frac{1}{\rho}\mathbf{B}\Big). \tag{2.79}$$

Vì nhân $\frac{1}{\rho}\mathbf{W}$ với ρ sẽ được $\mathbf{W}$ (điều này cũng đúng với số hạng độ lệch $\mathbf{B}$), ta có thể dùng $\mathbf{W}$ (và $\mathbf{B}$) làm tham số của hệ thống suy luận.

Chú giải của người dịch: Vì sao phải nhân với ρ?

Lấy $\rho = 0.5$: khi huấn luyện, trung bình chỉ một nửa số nơ-ron ở tầng dưới "gửi tín hiệu" lên, nên tổng đầu vào mà một nơ-ron ở tầng trên nhận được chỉ khoảng một nửa so với khi mọi nơ-ron đều có mặt. Mạng đã quen với mức tín hiệu "một nửa" đó. Khi kiểm tra, mọi nơ-ron đều bật, tín hiệu sẽ gấp đôi mức quen thuộc nếu không điều chỉnh; vì thế ta nhân trọng số với $\rho = 0.5$ để đưa về đúng mức (Phương trình (2.78)). Cách thứ hai (Phương trình (2.79), thường gọi là *dropout đảo ngược*) làm điều ngược lại: phóng to $\times\frac{1}{\rho} = \times 2$ ngay lúc huấn luyện, để khi kiểm tra dùng nguyên trọng số mà không phải sửa gì. Đây là cách các thư viện học sâu hiện nay thường cài đặt.

[8] Lưu ý rằng việc đặt hệ số co giãn bên trong hàm kích hoạt ψ (ví dụ dùng $\rho\mathbf{W}$) ngầm giả định rằng ψ là một hàm tuyến tính hoặc có tính thuần nhất dương (ví dụ hàm ReLU, với $\text{ReLU}(cx) = c\,\text{ReLU}(x)$ khi $c > 0$). Nếu dùng một hàm kích hoạt phi tuyến, chẳng hạn Sigmoid, thì không thể đưa hệ số co giãn vào bên trong. Trong những trường hợp đó, phép co giãn được áp dụng trực tiếp lên đầu ra của hàm kích hoạt để bảo toàn đúng giá trị kỳ vọng. Khi ấy, dạng toán học của phép co giãn tiêu chuẩn khi suy luận và của huấn luyện dropout đảo ngược lần lượt là $\mathbf{y} = \rho\cdot\psi(\mathbf{x}\cdot\mathbf{W}+\mathbf{B})$ và $\mathbf{y} = \frac{1}{\rho}\big(\mathbf{M}_{\text{drop}}\odot\psi(\mathbf{x}\cdot\mathbf{W}+\mathbf{B})\big)$.

2.5.3 Dừng sớm

Ở Chương 1, chúng ta đã bàn một chút về cách dừng huấn luyện bằng việc theo dõi hiệu năng trên dữ liệu kiểm định. Có thể xem đây là một dạng lựa chọn mô hình (*model selection*) nhằm tìm điểm cân bằng tối ưu giữa chưa khớp (*underfitting*) và quá khớp. Lưu ý rằng dừng sớm (*early stopping*) không chỉ là một mẹo kinh nghiệm; nó còn có cơ sở vững chắc trong lý thuyết học thống kê. Ví dụ, các nhà nghiên cứu đã phát hiện rằng, dưới một số điều kiện nhất định, dừng sớm có tác dụng tương tự chính quy hóa l_2, giới hạn việc học trong một vùng có giá trị tham số nhỏ [Bishop, 1995a; Goodfellow và cộng sự, 2016]. Hơn nữa, các nghiên cứu cho thấy những quy tắc dừng sớm cụ thể có liên hệ chặt chẽ với sự đánh đổi độ chệch–phương sai (*bias-variance trade-off*) và có thể đảm bảo những tính chất hội tụ mong muốn [Yao và cộng sự, 2007].

Tuy nhiên, trong thực tế, dừng sớm cần đến một số mẹo kinh nghiệm để trở nên khả thi và hữu ích. Vấn đề chính nằm ở việc xác định chính xác tiêu chí dừng. Một cách lý tưởng, ta có thể hình dung rằng có một đường cong lỗi hình chữ U hoàn hảo trên dữ liệu kiểm định, và việc huấn luyện có thể dừng ngay khi lỗi bắt đầu tăng. Tuy nhiên, sự thật là đường cong lỗi không thể được mô tả đơn giản như một hàm lồi chặt theo thời gian huấn luyện. Sau khi lỗi giảm trong một số bước huấn luyện nhất định, hiệu năng của mô hình có xu hướng dao động, dẫn tới nhiều cực tiểu địa phương. Vấn đề còn thú vị hơn nếu ta muốn tiết kiệm thời gian và dừng huấn luyện càng sớm càng tốt. Nhưng ta không bao giờ biết lựa chọn hay quyết định hiện tại có phải là tốt nhất hay không, vì ta không biết điều gì sẽ xảy ra tiếp theo. Một phương pháp thường dùng là quyết định có nên dừng huấn luyện hay không bằng cách kiểm tra trạng thái của mô hình trong một số bước cập nhật (hoặc epoch) gần nhất [Prechelt, 1998]. Một số điều kiện dừng sớm là:

- Mức thay đổi của hiệu năng nhỏ hơn một ngưỡng trong một số bước (hoặc epoch) cho trước.
- Mức thay đổi của tham số mô hình nhỏ hơn một ngưỡng trong một số bước (hoặc epoch) cho trước.
- Hiệu năng trung bình trên một số bước (hoặc epoch) cho trước bắt đầu giảm.

- Hiệu năng cao nhất trên một số bước (hoặc epoch) cho trước bắt đầu suy giảm.

Chú giải của người dịch: Dừng sớm trong thực tế

Một quy tắc rất hay gặp là "kiên nhẫn" (*patience*): sau mỗi epoch ta đo lỗi trên tập kiểm định; nếu chẳng hạn suốt 5 epoch liền mà lỗi không thấp hơn mức tốt nhất từng đạt được, ta dừng và lấy lại mô hình ở thời điểm tốt nhất. Ví dụ, lỗi kiểm định qua các epoch là 0.50, 0.40, 0.35, 0.36, 0.34, 0.35, 0.36, 0.37, 0.38, 0.39: mức tốt nhất là 0.34 ở epoch 5; sau đó 5 epoch liền không cải thiện nên ta dừng ở epoch 10. Lưu ý rằng nếu dừng ngay khi lỗi tăng lần đầu (epoch 4), ta đã bỏ lỡ điểm tốt hơn ở epoch 5 — đó chính là khó khăn do đường cong lỗi "dao động" mà sách nhắc tới.

Tuy nhiên, dùng mô hình tại thời điểm dừng huấn luyện không phải lúc nào cũng là lựa chọn tốt. Trong thực tế, quanh thời điểm đó lỗi tổng quát hóa của mô hình thường có phương sai lớn, khiến việc lựa chọn mô hình khó khăn hơn. Thay vì "chọn" một mô hình, một cách khác là kết hợp nhiều mô hình. Ví dụ, ta có thể lưu lại mô hình sau mỗi một số bước huấn luyện cho trước (gọi mỗi bản sao của mô hình là một điểm lưu (*checkpoint*)). Mô hình cuối cùng thu được bằng cách lấy trung bình tham số của vài điểm lưu sau cùng. Để có kết quả tốt hơn, ta có thể dùng những phương pháp tổ hợp mô hình phức tạp hơn (xem Chương 1).

2.5.4 Làm trơn xác suất đầu ra

Trong thống kê, làm trơn (*smoothing*) là quá trình giảm ảnh hưởng của các điểm dữ liệu nhiễu (thường là các giá trị cực đoan) và phân bổ lại khối lượng xác suất cho các điểm dữ liệu bình thường hoặc cho các sự kiện chưa quan sát được. Nó được dùng rộng rãi khi một phân phối được ước lượng từ dữ liệu hạn chế, trong đó xác suất của các sự kiện hiếm được biểu diễn kém. Ví dụ, xét bài toán mô hình hóa ngôn ngữ đã mô tả ở Mục 2.2. Một mô hình ngôn ngữ được huấn luyện theo cách buộc mô hình phải xuất ra một phân phối one-hot, tức là toàn bộ khối lượng xác suất bằng 1 được gán cho một từ duy nhất, còn các từ khác đều có xác suất bằng 0. Có lẽ sẽ tốt hơn nếu phân bổ xác suất cho tất cả các từ, dù nhiều từ trong số đó không được quan sát là câu trả lời khi biết các từ đứng trước. Bằng cách này, mô hình học cách đưa ra dự đoán "mềm" về xác suất của các từ, nhờ

đó có thể tổng quát hóa tốt hơn trên dữ liệu chưa thấy.

Cho một phân phối $\mathbf{p} = \begin{bmatrix} p_1 & \cdots & p_n \end{bmatrix}$, mục đích của làm trơn là thu được một ước lượng mới nằm giữa $\mathbf{p}$ và phân phối đều $\frac{1}{n}$. Một cách tiếp cận phổ biến cho ý tưởng này là dùng một **bộ ước lượng co** (*shrinkage estimator*) để cải thiện $\mathbf{p}$ bằng cách kéo nó lại gần $\frac{1}{n}$. Ví dụ, làm trơn cộng đã nhắc tới ở Chương 1 là một dạng đơn giản của bộ ước lượng co. Ở đây, chẳng hạn, ta xét việc làm trơn một phân phối đa thức (*multinomial distribution*). Gọi p_k là xác suất của sự kiện k và s_k là một đại lượng mô tả "số đếm" quan sát được nào đó của sự kiện. Xác suất p_k được cho bởi

$$p_k \;=\; \frac{s_k}{\sum_{k=1}^{n} s_k}. \tag{2.80}$$

Khi đó, phiên bản đã làm trơn của p_k được định nghĩa là

$$\hat{p}_k \;=\; \frac{s_k + \alpha}{\sum_{k=1}^{n}(s_k + \alpha)}. \tag{2.81}$$

Nó đơn giản là cộng thêm một lượng α vào mỗi s_k. Giá trị của α điều khiển độ trơn của ước lượng thu được. Ví dụ, $\hat{p}_k = p_k$ nếu $\alpha = 0$, và $\hat{p}_k \approx \frac{1}{n}$ khi $\alpha \to \infty$.

Ngoài làm trơn cộng, ta có thể làm trơn một phân phối bằng hàm Softmax với co giãn nhiệt độ (*temperature scaling*), được định nghĩa như sau:

$$\hat{p}_k \;=\; \frac{\exp(s_k/\beta)}{\sum_{k=1}^{n} \exp(s_k/\beta)}. \tag{2.82}$$

Dạng này là một ví dụ kinh điển của **phân phối Boltzmann** (*Boltzmann distribution*) [Uffink, 2017], trong đó s_k có thể được xem là năng lượng âm của một trạng thái, còn β biểu diễn nhiệt độ (*temperature*) điều khiển mức độ làm trơn. Lưu ý rằng s_k có thể được hiểu theo nhiều cách. Chẳng hạn, trong một mạng nơ-ron, s_k thường là logit (*logit*) trước kích hoạt của một nơ-ron. Đôi khi s_k thậm chí có thể là một xác suất thô. Sự linh hoạt này hàm ý rằng ta có thể áp dụng trực tiếp Phương trình (2.81–2.82) cho bất kỳ phân phối $\mathbf{p}$ nào, ngay cả khi không biết trước $\mathbf{p}$ ban đầu được ước lượng ra sao. Ta có thể viết lại các phương trình này bằng cách thay s_k bởi p_k:

$$\hat{p}_k = \frac{p_k + \alpha}{\sum_{k=1}^{n}(p_k + \alpha)}, \tag{2.83}$$

$$\hat{p}_k = \frac{\exp(p_k/\beta)}{\sum_{k=1}^{n} \exp(p_k/\beta)}. \tag{2.84}$$

Chú giải của người dịch: Nhiệt độ β làm trơn phân phối như thế nào?

Lấy ba "điểm số" $s = (2,\ 1,\ 0)$. Với $\beta = 1$: $e^2 \approx 7.39$, $e^1 \approx 2.72$, $e^0 = 1$, tổng ≈ 11.11, nên $\hat{p} \approx (0.67,\ 0.24,\ 0.09)$. Với $\beta = 2$ (nhiệt độ cao hơn): $e^1 \approx 2.72$, $e^{0.5} \approx 1.65$, $e^0 = 1$, tổng ≈ 5.37, nên $\hat{p} \approx (0.51,\ 0.31,\ 0.19)$ — phân phối "phẳng" hơn, gần phân phối đều $(\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$ hơn. Ngược lại, β nhỏ (ví dụ 0.5) làm phân phối "nhọn" hơn, dồn gần hết xác suất vào phần tử lớn nhất. Tên gọi "nhiệt độ" mượn từ vật lý: ở nhiệt độ cao, các phân tử ở nhiều trạng thái khác nhau với khả năng gần ngang nhau; ở nhiệt độ thấp, chúng dồn về trạng thái năng lượng thấp nhất.

Một phương pháp làm trơn đơn giản khác là nội suy tuyến tính (*linear interpolation*) **p** với một phân phối đều. Một dạng chuẩn của phép nội suy này là:

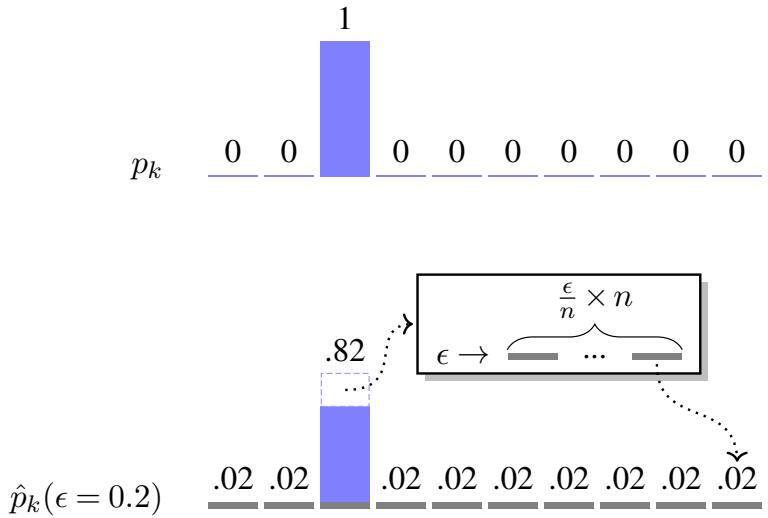
$$\hat{p}_k = (1 - \epsilon) \cdot p_k + \frac{\epsilon}{n}, \tag{2.85}$$

trong đó ϵ là một siêu tham số cho biết mức độ ta dựa vào phân phối đều. Để minh họa cách Phương trình (2.85) hoạt động, giả sử **p** là một vectơ one-hot (ví dụ $p_k = 1$ nếu $k = z$, và $p_k = 0$ trong các trường hợp còn lại). Khi áp dụng Phương trình (2.85), thực chất ta thu nhỏ phân phối ban đầu theo hệ số $(1 - \epsilon)$ và cộng một khối lượng xác suất đều bằng $\frac{\epsilon}{n}$ vào mọi chiều. Sự phân bổ lại này làm phân phối thu được phẳng hơn và trơn hơn. Xem Hình 2.15 để có minh họa trực quan.

Chú giải của người dịch: Kiểm tra các con số trong Hình 2.15

Trong hình có $n = 10$ chiều và $\epsilon = 0.2$. Mỗi chiều được cộng thêm $\frac{\epsilon}{n} = \frac{0.2}{10} = 0.02$. Chiều đúng (vốn có xác suất 1) trở thành $(1 - 0.2) \cdot 1 + 0.02 = 0.82$; chín chiều còn lại trở thành $(1 - 0.2) \cdot 0 + 0.02 = 0.02$. Kiểm tra tổng: $0.82 + 9 \times 0.02 = 1$, vẫn là một phân phối xác suất hợp lệ.

Trong NLP, vì nhiều hệ thống đưa ra dự đoán dưới dạng giống xác suất, một ứng dụng phổ biến của làm trơn là làm trơn đầu ra của hệ thống. Có hai cách. Thứ nhất, ta có thể làm trơn xác suất chuẩn đúng (*ground-truth*) để mô hình được dẫn dắt bởi sai số tổng quát hóa thay vì sai số do các quyết định cứng gây ra. Ví dụ, kỹ thuật **làm trơn nhãn** (*label smoothing*)



Hình 2.15: Minh họa việc làm trơn một phân phối bằng cách nội suy nó với một phân phối đều: $\hat{p}_k = (1 - \epsilon) \cdot p_k + \frac{\epsilon}{n}$. Về mặt toán học, phép toán này giảm mỗi xác suất ban đầu p_k đi một lượng ϵp_k, gom lại tổng cộng một khối lượng xác suất bằng ϵ. Khối lượng này sau đó được chia đều cho tất cả n chiều, bảo đảm mỗi biến đều nhận được một lượng làm trơn cộng thêm bằng $\frac{\epsilon}{n}$.

có cùng dạng với Phương trình (2.85) và cải thiện cách biểu diễn chuẩn đúng trên dữ liệu phân loại [Szegedy và cộng sự, 2016]. Thứ hai, ta có thể giảm độ "nhọn" và tăng độ "dày đuôi" của một phân phối dự đoán[9]. Phương pháp này thường được dùng khi cần đến xác suất hậu nghiệm (*posterior probability*) của dự đoán, chẳng hạn trong giải mã/huấn luyện rủi ro Bayes cực tiểu (*minimum Bayesian risk*) [Bickel và Doksum, 2015; Goodman, 1996a; Kumar và Byrne, 2004a].

2.5.5 Huấn luyện với nhiễu

Ở trên, ta đã thấy rằng cộng thêm một lượng nào đó vào mỗi số đếm quan sát được của các sự kiện khi dự đoán xác suất có thể cải thiện khả năng tổng quát hóa. Theo quan điểm **thống kê vững** (*robust statistics*) [Olive, 2022], điều này tương đương với việc tăng tính vững của một bộ ước lượng, trong tình huống mà một phân phối bị lệch thường dẫn tới một mô hình bị thiên lệch. Việc thêm một nhiễu loạn nhỏ vào ước lượng có thể ngăn những thiên lệch lớn gây ra bởi các điểm ngoại lai (*outliers*) và những lần quan sát bất ngờ của các sự kiện hiếm. Theo nghĩa này, làm trơn có thể được xem là một cách đưa nhiễu vào huấn luyện, tức là ta áp đặt một tiên nghiệm phân phối đều lên ước lượng, dù ước lượng đúng có thể không phải là phân phối đều.

Huấn luyện với nhiễu dựa trên ý tưởng rằng một mô hình nên học cách vận hành trong những điều kiện không lý tưởng, nhờ đó tránh quá khớp các điểm dữ liệu cực đoan. Ở đây, thuật ngữ *nhiễu* mang nghĩa rộng, và có nhiều cách khác nhau để chính quy hóa việc huấn luyện thông qua tiêm nhiễu (*noise injection*). Một trong những phương pháp đơn giản nhất là đưa nhiễu vào hàm mục tiêu huấn luyện. Ví dụ, làm trơn nhãn (như đã bàn ở trên) có thể được xem là tiêm nhiễu vào các nhãn chuẩn đúng. Ngoài ra, ta có thể tiêm nhiễu ngẫu nhiên trực tiếp vào đầu vào, các trạng thái ẩn trung gian, hoặc đầu ra của một mạng nơ-ron. Một lựa chọn rất phổ biến là **nhiễu Gauss** (*Gaussian noise*). Giả sử ta có một vectơ kích hoạt $\mathbf{x} \in \mathbb{R}^n$. Việc thêm nhiễu Gauss tạo ra một vectơ mới bị làm nhiễu như

[9]Trong thống kê, điều này có liên hệ lỏng lẻo với việc biến một phân phối nhọn đỉnh (*leptokurtic*) thành một phân phối có độ nhọn trung bình (*mesokurtic*) hoặc bẹt đỉnh (*platykurtic*) hơn để tránh quá tự tin.

sau:

$$\mathbf{x}_{\text{noise}} \;=\; \mathbf{x} + \mathbf{g}, \tag{2.86}$$

trong đó $\mathbf{g} \in \mathbb{R}^n$ là vectơ nhiễu. Mỗi phần tử của $\mathbf{g}$ được rút độc lập từ một phân phối Gauss:

$$\mathbf{g} \;\sim\; \mathcal{N}(\boldsymbol{\mu}, \text{diag}(\boldsymbol{\sigma}^2)). \tag{2.87}$$

Với phần tử thứ k của $\mathbf{g}$, phân phối này định nghĩa xác suất $\Pr(g_k)$ là:

$$\Pr(g_k) \;=\; \frac{1}{\sigma_k\sqrt{2\pi}} \cdot \exp\Big(-\frac{(g_k-\mu_k)^2}{2\sigma_k^2}\Big), \tag{2.88}$$

trong đó μ_k là giá trị trung bình và σ_k là độ lệch chuẩn. Thông thường, μ_k được đặt bằng 0. Độ lệch chuẩn σ_k là một siêu tham số điều khiển độ lớn của nhiễu được tiêm vào. σ_k càng lớn thì nhiễu ngẫu nhiên càng trải rộng quanh giá trị trung bình, làm tăng khả năng sinh ra những nhiễu loạn mạnh.

Chú giải của người dịch: Phân phối Gauss (phân phối chuẩn)

Phương trình (2.88) là đường cong "hình chuông" quen thuộc: các giá trị gần μ_k có khả năng xuất hiện cao nhất, càng xa μ_k thì càng hiếm. Chính xác hơn, $\Pr(g_k)$ ở đây là *mật độ xác suất* (chiều cao của đường cong), không phải xác suất của một giá trị cụ thể. Một quy tắc dễ nhớ: khoảng 68% giá trị rơi vào đoạn $[\mu_k - \sigma_k,\ \mu_k + \sigma_k]$ và khoảng 95% rơi vào $[\mu_k - 2\sigma_k,\ \mu_k + 2\sigma_k]$. Chẳng hạn với $\mu_k = 0$, $\sigma_k = 0.1$, hầu hết nhiễu nằm trong khoảng -0.2 đến 0.2; nếu $x_k = 1.5$ thì sau khi thêm nhiễu, x_k thường thành một số trong khoảng 1.3 đến 1.7. Ký hiệu $\text{diag}(\boldsymbol{\sigma}^2)$ trong Phương trình (2.87) chỉ đơn giản nói rằng mỗi phần tử có phương sai σ_k^2 riêng và các phần tử được rút độc lập với nhau.

Phương trình (2.86) rất tổng quát và có thể được áp dụng gần như ở bất cứ đâu trong một mạng nơ-ron. Cho một tầng $\mathbf{y} = \psi(\mathbf{x}\cdot\mathbf{W}+\mathbf{B})$, nhiễu (ký hiệu là $\mathbf{g}_{\text{input}}$) có thể được cộng trực tiếp vào đầu vào:

$$\mathbf{y} = \psi\big((\mathbf{x} + \mathbf{g}_{\text{input}})\cdot\mathbf{W} + \mathbf{B}\big). \tag{2.89}$$

Tương tự, nhiễu (chẳng hạn $\mathbf{g}_{\text{output}}$) có thể được cộng vào giá trị kích hoạt (hay đầu ra):

$$\mathbf{y} = \psi(\mathbf{x}\cdot\mathbf{W} + \mathbf{B}) + \mathbf{g}_{output}. \tag{2.90}$$

Ví dụ, ta có thể đơn giản làm nhiễu các đầu vào thô (hoặc các giá trị đích) rồi chạy mạng như bình thường, hoặc tiêm nhiễu ngẫu nhiên vào giá trị kích hoạt của tất cả các tầng ẩn. Trong khi việc thêm nhiễu vào đầu vào và giá trị kích hoạt của các tầng là cách làm phổ biến [Plaut và cộng sự, 1986; Holmström và Koistinen, 1992; Bishop, 1995b], một cách tiếp cận khác biệt là thêm nhiễu ngẫu nhiên trực tiếp vào tham số mô hình hoặc gradient [Graves và cộng sự, 2013b; Neelakantan và cộng sự, 2015]. Ví dụ, làm nhiễu ma trận trọng số có dạng sau:

$$\mathbf{y} = \psi\big(\mathbf{x} \cdot (\mathbf{W} + \mathbf{g}_{\text{w}}) + \mathbf{B}\big), \tag{2.91}$$

trong đó $\mathbf{g}_{\text{w}}$ là một ma trận nhiễu có cùng kích thước với $\mathbf{W}$. Hơn nữa, ta có thể tiêm nhiễu (ký hiệu là $\mathbf{g}_{\text{gradient}}$) vào gradient của hàm mất mát theo $\mathbf{W}$. Gọi $\mathbf{s} = \mathbf{x} \cdot \mathbf{W} + \mathbf{B}$ là các logit trước kích hoạt. Gradient có nhiễu có thể được viết là:

$$\begin{aligned}
\frac{\partial \mathcal{L}}{\partial \mathbf{W}} &= \mathbf{x}^\top \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{s}} + \mathbf{g}_{\text{gradient}} \\
&= \mathbf{x}^\top \cdot \Big(\frac{\partial \mathcal{L}}{\partial \mathbf{y}} \odot \frac{\partial \mathbf{y}}{\partial \mathbf{s}}\Big) + \mathbf{g}_{\text{gradient}} \\
&= \mathbf{x}^\top \cdot \Big(\frac{\partial \mathcal{L}}{\partial \mathbf{y}} \odot \psi'(\mathbf{s})\Big) + \mathbf{g}_{\text{gradient}}.
\end{aligned} \tag{2.92}$$

Việc dùng gradient có nhiễu được nhận thấy không chỉ giúp huấn luyện vững hơn mà còn làm cho dòng gradient chảy qua mạng dễ dàng hơn [Gulcehre và cộng sự, 2016].

Cần lưu ý rằng nhiễu chỉ xuất hiện trong lúc huấn luyện, còn mô hình hoạt động mà không thêm nhiễu khi dự đoán trên dữ liệu mới. Theo nghĩa này, nhiều phương pháp chính quy hóa có thể được xếp vào khuôn khổ huấn luyện có nhiễu, vốn được dùng để ngăn việc khớp quá chính xác dữ liệu huấn luyện và giúp hệ thống suy luận tổng quát hóa tốt trên dữ liệu kiểm tra. Ví dụ, dropout che ngẫu nhiên các giá trị kích hoạt của tầng để mọi nơ-ron học cách làm việc trong một môi trường có nhiễu. Khi kiểm tra, mọi nơ-ron làm việc cùng nhau như trong một mạng nơ-ron thông thường.

Một ưu điểm nữa của huấn luyện có nhiễu là việc dùng nhiễu ngẫu nhiên tạo ra các mẫu huấn luyện "mới". Ngay cả với cùng một mẫu, các nhiễu khác nhau cũng có thể dẫn tới các kết quả huấn luyện khác nhau.

Nói cách khác, về bản chất ta huấn luyện mô hình trên vô số mẫu. Ý tưởng này cũng gắn với một hướng nghiên cứu khác về huấn luyện bằng dữ liệu tổng hợp, gọi là **tăng cường dữ liệu** (*data augmentation*). Nói đơn giản, tăng cường dữ liệu bao gồm một loạt kỹ thuật được thiết kế để sinh ra các mẫu huấn luyện mới từ những mẫu sẵn có. Một ví dụ là **dịch ngược** (*back-translation*) [Sennrich và cộng sự, 2016a]. Khi phát triển một hệ thống dịch máy từ ngôn ngữ A sang ngôn ngữ B, trước tiên ta có thể huấn luyện một hệ thống dịch theo chiều ngược lại (chẳng hạn hệ thống B→A) trên dữ liệu song ngữ. Sau đó, ta dùng hệ thống B→A để dịch một lượng dữ liệu bổ sung ở ngôn ngữ đích sang ngôn ngữ nguồn. Kết quả là ta có dữ liệu song ngữ mới, trong đó phần ngôn ngữ đích là thật còn phần ngôn ngữ nguồn là tổng hợp. Dữ liệu mới này có thể được dùng cùng với dữ liệu song ngữ khác để huấn luyện hệ thống A→B. Ngoài dịch ngược, có nhiều phương pháp tăng cường dữ liệu khác trong NLP, bao gồm thay từ bằng từ đồng nghĩa, hoán đổi hai từ, và xóa hoặc chèn từ. Hơn nữa, việc tăng cường có thể được thực hiện trong không gian ẩn (*latent space*) liên tục, chẳng hạn thay một vectơ từ bằng một vectơ ở gần nó về mặt hình học. Vì tăng cường dữ liệu bao trùm rất nhiều chủ đề, chúng tôi giới thiệu bạn đọc một vài bài tổng quan để tìm hiểu thêm [Feng và cộng sự, 2021; Shorten và Khoshgoftaar, 2019].

Chú giải của người dịch: Dịch ngược qua ví dụ Việt–Anh

Giả sử ta muốn xây dựng hệ thống dịch Việt→Anh (A = tiếng Việt, B = tiếng Anh) nhưng chỉ có ít cặp câu song ngữ, trong khi văn bản tiếng Anh đơn ngữ thì vô số. Ta huấn luyện trước một hệ thống Anh→Việt, rồi dùng nó dịch hàng triệu câu tiếng Anh sang tiếng Việt. Ta được các cặp (câu Việt do máy dịch, câu Anh thật). Phía tiếng Việt có thể hơi "vụng", nhưng phía tiếng Anh — chính là phía mà hệ thống Việt→Anh phải học cách *sinh ra* — là câu thật, chuẩn mực. Nhờ vậy, mô hình học được cách viết câu đích tự nhiên từ lượng dữ liệu lớn hơn nhiều.

Một lưu ý cuối cùng về tăng cường dữ liệu: dữ liệu tổng hợp cũng có thể được tạo ra cho những mục tiêu cụ thể. Một ý tưởng phổ biến là **học máy đối kháng** (*adversarial machine learning*). Nó sinh ra các **mẫu đối kháng** (*adversarial samples*) mà trên đó mô hình sẽ mắc lỗi (gọi những quá trình như vậy là các **cuộc tấn công** (*attacks*)) [Szegedy và cộng sự, 2014a; Goodfellow và cộng sự, 2015]. Mô hình được huấn luyện để đưa

ra dự đoán đúng trên các mẫu này, tức là nó phòng thủ trước các cuộc tấn công. Ví dụ, trong một số trường hợp, một hệ thống dịch máy có thể cho ra bản dịch hoàn toàn sai chỉ vì giới tính của chủ ngữ trong câu đầu vào bị đảo ngược. Để có một hệ thống vững hơn, ta có thể huấn luyện mô hình dịch bằng dữ liệu cân bằng giới tính hơn, được thu thập thủ công hoặc tự động. Nhưng không dễ để tạo ra những mẫu trông giống câu bình thường mà lại đánh lừa được mô hình [Zhang và cộng sự, 2020b]. Điều này khiến việc sinh mẫu đối kháng trong NLP vừa thú vị vừa đầy thách thức, vì một thay đổi nhỏ trong câu (như thay một từ) có thể dẫn tới một câu mang nghĩa rất khác[10]. Thách thức này cũng thúc đẩy một hướng nghiên cứu về mẫu đối kháng trong NLP [Jia và Liang, 2017; Belinkov và Bisk, 2018; Ebrahimi và cộng sự, 2018; Alzantot và cộng sự, 2018].

2.6 Các phương pháp không giám sát và bộ tự mã hóa

Học không giám sát (*unsupervised learning*) quan tâm đến việc khám phá những khuôn mẫu ẩn bên dưới trong một tập điểm dữ liệu không có nhãn. Nhiều bài toán có thể được xem là bài toán học không giám sát kinh điển, dù chúng tôi sẽ không bàn chi tiết về chúng trong chương này. Ví dụ, phân cụm dữ liệu (*data clustering*) nhằm tìm ra các nhóm trong một tập đối tượng dữ liệu mà không có sự giám sát tường minh nào về cách nhóm đúng. Một ví dụ nổi tiếng khác là khai phá luật kết hợp (*association rule mining*), thường được phát biểu như một quá trình thiết lập mối quan hệ giữa các tập đối tượng dữ liệu. Dù những bài toán này đúng là thuộc phạm vi học không giám sát, chúng tôi sẽ tập trung vào các bài toán học biểu diễn (*representation learning*) không giám sát hay **học đặc trưng** (*feature learning*), trong đó mô hình học cách ánh xạ một đối tượng từ không gian đầu vào sang một không gian vectơ đặc trưng ít chiều[11].

Việc học các biểu diễn ít chiều đã được nghiên cứu sâu rộng trong bối cảnh tìm một phép biến đổi tuyến tính từ không gian ban đầu sang

[10] Ngược lại, trong thị giác máy tính, việc tạo mẫu đối kháng dễ hơn nhiều bằng cách thay đổi chút ít đầu vào (ví dụ các điểm ảnh), vì không gian đầu vào là liên tục và một nhiễu loạn nhỏ ở đầu vào hầu như không ảnh hưởng tới toàn bộ bức ảnh.

[11] Ngoài việc học cách biểu diễn các đối tượng dữ liệu, mục này cũng đề cập một số chủ đề về việc sinh ra các đối tượng dữ liệu. Chúng ta sẽ tìm hiểu chúng ở Mục 2.6.3.

một không gian mới. Ví dụ, **phân tích thành phần chính** (*principal component analysis, PCA*) và các biến thể của nó nhằm tìm một hàm ánh xạ tuyến tính sao cho một đối tượng dữ liệu (nhiều chiều) có thể được biểu diễn bởi tọa độ của nó dọc theo những hướng có phương sai lớn nhất [Pearson, 1901; Wold và cộng sự, 1987]. Ở đây, chúng ta mở rộng hàm ánh xạ thành dạng tổng quát hóa phi tuyến tự nhiên của nó và dùng mạng nơ-ron làm lời giải cho bài toán ánh xạ.

Như các mô hình học máy khác, một mạng nơ-ron thường được huấn luyện bằng cách tối ưu tham số mô hình theo một hàm mất mát nào đó. Một thách thức đáng kể với học không giám sát là không có nhãn chuẩn đúng nào để dẫn dắt quá trình học. Một giải pháp cho vấn đề này là dùng đến các phương pháp phi tham số (*non-parametric*) hoặc các mẹo kinh nghiệm (xem Chương 1). Tuy nhiên, bản thân những phương pháp này không được thiết kế để giải quyết các vấn đề tối ưu của mạng nơ-ron quy mô lớn, đặc biệt khi mạng được xây dựng với số lượng tham số khổng lồ. Vì vậy, trong huấn luyện không giám sát mạng nơ-ron, cách phổ biến hơn là rút ra tín hiệu giám sát trực tiếp từ chính dữ liệu đầu vào. Có nhiều cách để làm điều này [Hopfield, 1982; Ackley và cộng sự, 1985; Dayan và cộng sự, 1995; Hinton và Salakhutdinov, 2006], nhưng trong mục này chúng ta sẽ tập trung vào **bộ tự mã hóa** (*auto-encoders*). Chúng tôi chọn bộ tự mã hóa để thảo luận vì chúng có dạng giống các mô hình có giám sát nói chung và có thể được huấn luyện liền mạch bằng lan truyền ngược.

Bộ tự mã hóa là một loại mạng nơ-ron cố gắng tái tạo dữ liệu đầu vào từ biểu diễn của nó. Nó được truyền cảm hứng từ ý tưởng giảm chiều (*dimensionality reduction*):

> *Dữ liệu nhiều chiều có thể được chuyển thành các mã ít chiều bằng cách huấn luyện một mạng nơ-ron nhiều tầng có một tầng trung tâm nhỏ để tái tạo các vectơ đầu vào nhiều chiều.*
>
> – Hinton và Salakhutdinov [2006]

Điều này cũng phát triển ý tưởng học biểu diễn, theo đó thông tin về một đối tượng có thể được biểu diễn đầy đủ bằng một vectơ số thực ít chiều. Thông thường, một bộ tự mã hóa gồm một hàm giảm chiều (có thể phi tuyến) (gọi là **bộ mã hóa** (*encoder*)) để ánh xạ đối tượng đầu vào thành biểu diễn vectơ đặc trưng ít chiều của nó (gọi là **mã** (*code*)). Ngoài

ra, nó còn gồm một hàm ngược (gọi là **bộ giải mã** (*decoder*)) ánh xạ mã trở lại đối tượng. Vì vậy, mặc dù được gọi là "bộ mã hóa", bộ tự mã hóa không chỉ là một bộ mã hóa mà là sự kết hợp của một bộ mã hóa và một bộ giải mã. Một cách hình thức hơn, gọi $\mathbf{x} \in \mathbb{R}^{d_x}$ là vectơ đầu vào của mô hình, chẳng hạn một biểu diễn nhiều chiều của một từ. Bộ mã hóa xuất ra một vectơ mô tả mã hay biểu diễn ít chiều của $\mathbf{x}$, ký hiệu là $\mathbf{h} \in \mathbb{R}^{d_h}$. Vectơ này được định nghĩa là:

$$\mathbf{h} \;=\; \text{Encode}(\mathbf{x}), \tag{2.93}$$

trong đó $\text{Encode}(\cdot)$ là mạng mã hóa. $\text{Encode}(\cdot)$ thường là một mạng nơ-ron nhiều tầng và hoạt động như một mô-đun cắm thêm (*plug-in*) cho các hệ thống khác. Do đó, $\text{Encode}(\cdot)$ là một mô hình đa dụng. Trong các chương sau, chúng ta sẽ thấy nhiều ví dụ trong đó các bộ mã hóa được huấn luyện và áp dụng như những bộ phận của các hệ thống "lớn hơn".

Khi đã có mã, ta dùng bộ giải mã để ánh xạ nó trở lại không gian đầu vào:

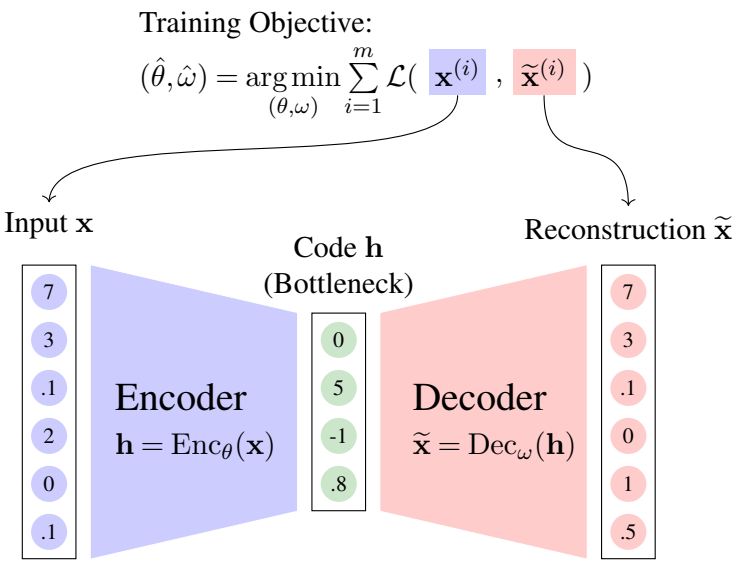
$$\widetilde{\mathbf{x}} \;=\; \text{Decode}(\mathbf{h}), \tag{2.94}$$

trong đó $\widetilde{\mathbf{x}} \in \mathbb{R}^{d_x}$ là bản tái tạo của đầu vào, và $\text{Decode}(\cdot)$ là mạng giải mã. Cho đầu vào ban đầu $\mathbf{x}$ và đầu vào được tái tạo $\widetilde{\mathbf{x}}$, mục tiêu của bộ tự mã hóa là cực tiểu hóa sự sai khác giữa $\mathbf{x}$ và $\widetilde{\mathbf{x}}$. Giả sử bộ mã hóa và bộ giải mã lần lượt được tham số hóa bởi θ và ω, ký hiệu là $\text{Encode}_\theta(\cdot)$ và $\text{Decode}_\omega(\cdot)$. Hàm mục tiêu huấn luyện trên một tập mẫu $\{\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(m)}\}$ được định nghĩa là

$$\begin{aligned}
(\hat{\theta}, \hat{\omega}) &= \arg\min_{(\theta,\omega)} \sum_{i=1}^{m} \mathcal{L}\Big(\mathbf{x}^{(i)}, \widetilde{\mathbf{x}}^{(i)}\Big) \\
&= \arg\min_{(\theta,\omega)} \sum_{i=1}^{m} \mathcal{L}\Big(\mathbf{x}^{(i)}, \text{Decode}_\omega(\mathbf{h}^{(i)})\Big) \\
&= \arg\min_{(\theta,\omega)} \sum_{i=1}^{m} \mathcal{L}\Big(\mathbf{x}^{(i)}, \text{Decode}_\omega(\text{Encode}_\theta(\mathbf{x}^{(i)}))\Big),
\end{aligned} \tag{2.95}$$

trong đó $\mathcal{L}(\cdot)$ là hàm mất mát tính sự sai khác giữa $\mathbf{x}$ và $\widetilde{\mathbf{x}}$. Đôi khi nó được gọi là **mất mát tái tạo** (*reconstruction loss*). Các hàm mất mát tái tạo phổ biến gồm mất mát sai số bình phương trung bình (*mean squared error*), mất mát entropy chéo (*cross-entropy*), v.v.

Khi ghép bộ mã hóa và bộ giải mã lại với nhau, ta dễ nghĩ tới một mạng mà ta đưa thứ gì vào tầng đầu vào thì nhận lại đúng thứ đó ở tầng đầu ra. Thách thức ở đây là vectơ ít chiều **h** đóng vai trò một nút thắt cổ chai (*bottleneck*) trong dòng thông tin. Có nguy cơ mất thông tin trong phép biến đổi từ **x** sang **h** hoặc từ **h** sang $\widetilde{\mathbf{x}}$, khiến việc "sao chép" đầu vào sang đầu ra trở nên khó khăn. Thay vào đó, ta cần "ép" một đối tượng từ không gian nhiều chiều vào một không gian ít chiều, dày đặc, rồi "giải ép" nó từ không gian mới trở về không gian nhiều chiều ban đầu. Hệ quả của quá trình ép–giải ép này là bộ mã hóa bị buộc phải nén dữ liệu nhưng vẫn giữ được thông tin nhiều nhất có thể. Vì vậy, bộ tự mã hóa được bàn ở đây còn được gọi là **bộ tự mã hóa thiếu đủ** (*undercomplete auto-encoder*), vì **h** có kích thước nhỏ hơn **x** và $\widetilde{\mathbf{x}}$. Hình 2.16 minh họa cấu trúc của bộ tự mã hóa thiếu đủ.



Hình 2.16: Minh họa một bộ tự mã hóa thiếu đủ với một bộ mã hóa, một bộ giải mã và một tầng nút thắt cổ chai kẹp giữa chúng. Một đầu vào **x** (bên trái) được biến đổi thành mã **h** (ở giữa) rồi thành bản tái tạo $\widetilde{\mathbf{x}}$ (bên phải). Tham số của cả bộ mã hóa lẫn bộ giải mã được tối ưu bằng cách cực tiểu hóa sự sai khác giữa đầu vào **x** và bản tái tạo $\widetilde{\mathbf{x}}$ trên một số mẫu không có nhãn $\{\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(m)}\}$. Với các mẫu mới, ta bỏ bộ giải mã đi và dùng bộ mã hóa để sinh ra các mã hay biểu diễn mới.

Nhãn trong hình: *Training Objective*: hàm mục tiêu huấn luyện; *Input*: đầu vào; *Code (Bottleneck)*: mã (nút thắt cổ chai); *Reconstruction*: bản tái tạo; *Encoder/Decoder*: bộ mã hóa/bộ giải mã (Enc, Dec là viết tắt của Encode, Decode).

Cho hàm mất mát $\mathcal{L}(\cdot)$, bộ mã hóa $\text{Encode}_\theta(\cdot)$ và bộ giải mã $\text{Decode}_\omega(\cdot)$, các tham số $\hat{\theta}$ và $\hat{\omega}$ có thể được tối ưu bằng phương pháp hạ gradient như trong học có giám sát (xem Mục 2.4.1). Khi áp dụng bộ tự mã hóa, ta có thể đơn giản bỏ bộ giải mã đi và dùng bộ mã hóa như một bộ trích xuất đặc trưng (*feature extractor*), tức là, với một đầu vào mới $\mathbf{x}_{\text{new}}$, ta sinh ra một biểu diễn mới

$$\hat{\mathbf{h}}_{\text{new}} = \text{Encode}_{\hat{\theta}}(\mathbf{x}_{\text{new}}). \tag{2.96}$$

Chú giải của người dịch: Bộ tự mã hóa giống "tóm tắt rồi kể lại"

Hãy tưởng tượng bạn phải tóm tắt một bài văn dài 6 trang vào đúng 4 dòng (bộ mã hóa), rồi một người khác chỉ đọc 4 dòng đó và phải viết lại gần đúng bài văn gốc (bộ giải mã). Muốn người kia viết lại tốt, 4 dòng tóm tắt buộc phải chứa những ý *cốt lõi nhất*. Trong Hình 2.16, vectơ 6 số $\mathbf{x} = (7, 3, 0.1, 2, 0, 0.1)$ bị "nén" thành mã 4 số $\mathbf{h} = (0, 5, -1, 0.8)$, rồi được khôi phục thành $\widetilde{\mathbf{x}} = (7, 3, 0.1, 0, 1, 0.5)$: gần giống nhưng không hoàn hảo. Điều đáng chú ý là ta *không cần nhãn*: chính $\mathbf{x}$ vừa là đầu vào vừa là "đáp án". Sau khi huấn luyện xong, phần tóm tắt $\mathbf{h}$ thường là một biểu diễn rất có ích cho các bài toán khác.

Lưu ý rằng bộ mã hóa không phải là một hệ thống độc lập mà thường làm việc cùng các mô hình khác để tạo thành một hệ thống hoàn chỉnh. Ví dụ, ta có thể huấn luyện một bộ tự mã hóa trên một số câu, rồi đặt một tầng Softmax lên đầu ra của bộ mã hóa để xây dựng một bộ phân loại câu. Bộ phân loại này có thể được huấn luyện thêm trên dữ liệu riêng của một tác vụ để giải một bài toán mới, chẳng hạn gán cho câu nhãn cực tính cảm xúc (*sentiment polarity*) của nó. Điều này cũng khiến việc ứng dụng bộ tự mã hóa thuộc về mô thức chung của tiền huấn luyện (*pre-training*): một mô hình con (tức bộ mã hóa) trước tiên được huấn luyện trên dữ liệu quy mô lớn, không phụ thuộc tác vụ, rồi được dùng như một thành phần của một mô hình lớn hơn trong một tác vụ hạ nguồn (*downstream task*).

2.6.1 Bộ tự mã hóa với bộ chính quy hóa tường minh

Khi dùng các mạng nơ-ron phức tạp hơn, bộ tự mã hóa có xu hướng học một phép biến đổi đồng nhất (*identity transformation*), mặc dù nút thắt cổ chai khiến việc truyền qua mà không mất thông tin khó hơn đôi chút. Đây là điều ta vẫn thường mong đợi: ta có thể biến **h** thành một vật thay thế cho **x** và giải mã **h** thành thứ gì đó rất giống **x**. Mặt khác, việc học một phép biến đổi đồng nhất chính xác đòi hỏi một mô hình cực kỳ phức tạp và rất dễ quá khớp. May mắn là, như với các mô hình học máy khác, ta có thể chính quy hóa việc huấn luyện bằng các phương pháp đã trình bày ở Mục 2.5. Một trong những phương pháp phổ biến nhất là thêm một số hạng chính quy hóa tường minh vào hàm mất mát. Kết hợp Phương trình (2.74) và Phương trình (2.95), ta có thể định nghĩa hàm mục tiêu huấn luyện là:

$$(\hat{\theta}, \hat{\omega}) \;=\; \arg\min_{(\theta,\omega)} \sum_{i=1}^{m} \mathcal{L}\Big(\mathbf{x}^{(i)}, \text{Decode}_{\omega}(\text{Encode}_{\theta}(\mathbf{x}^{(i)}))\Big) + \alpha \cdot R, \tag{2.97}$$

trong đó R là số hạng chính quy hóa thể hiện một tri thức tiên nghiệm nào đó mà ta muốn áp đặt lên việc huấn luyện, và α là hệ số trọng số của nó. Một lựa chọn phổ biến cho R trong bộ tự mã hóa là **hàm phạt thưa** (*sparsity penalty*) (tạo ra cái được gọi là **bộ tự mã hóa thưa** (*sparse auto-encoder*)). Cách đơn giản nhất để cài đặt hàm phạt này là áp dụng chuẩn l_1 hoặc l_2 lên mã, như sau:

$$R_{l_1} = \sum_{i=1}^{m} \sum_{k} \left| h_k^{(i)} \right|, \tag{2.98}$$

$$R_{l_2} = \sum_{i=1}^{m} \sqrt{\sum_{k} \left(h_k^{(i)} \right)^2}. \tag{2.99}$$

Cần lưu ý rằng, khác với các hàm phạt lên tham số mô hình (xem Mục 2.5.1), hàm phạt thưa chính quy hóa để chính mã **h** (hay đầu ra của bộ mã hóa) trở nên thưa. Ý tưởng khuyến khích tính thưa trong biểu diễn bắt nguồn từ **mã hóa thưa** (*sparse coding*) [Olshausen và Field, 1997]. Ý tưởng này cho rằng thông tin của một đối tượng được ẩn trong những phụ thuộc phức tạp giữa các thuộc tính (hay đặc trưng) ban đầu của đối

tượng. Một hệ thống học biểu diễn tốt nên trích xuất những phụ thuộc đó và biến đổi chúng thành một tập các đặc trưng độc lập. Và chỉ nên có một số ít trong các đặc trưng độc lập này được kích hoạt, trong khi các đặc trưng được kích hoạt sẽ thay đổi khi ta chuyển sang một đối tượng mới. Cũng lưu ý rằng, từ góc nhìn học Bayes, các hàm phạt khác trong huấn luyện có chính quy hóa có thể được hiểu là các tiên nghiệm trên mô hình. Tuy nhiên, hàm phạt thưa không phải là một tiên nghiệm, vì nó không phụ thuộc vào mô hình (hay tham số mô hình) mà phụ thuộc vào dữ liệu huấn luyện [Goodfellow và cộng sự, 2016]. Theo quan điểm này, hàm phạt thưa không nên được xem là một số hạng "chính quy hóa", mà chỉ đơn giản là một phân phối nào đó trên các trạng thái trung gian của mô hình. Mặt khác, tính thưa của mã, dù không được giải thích thỏa đáng theo cách dùng thông thường của số hạng chính quy hóa, thực sự hữu ích trong nhiều ứng dụng của bộ tự mã hóa, vì nó trực tiếp mô hình hóa cách biểu diễn đầu vào và áp đặt "tiên nghiệm" lên kết quả của bộ mã hóa. Khi xét từ góc độ thực nghiệm, hàm phạt thưa vẫn được coi là một bộ chính quy hóa, làm thiên lệch việc huấn luyện về phía một số mô hình nhất định.

Ngoài Phương trình (2.98–2.99), còn có những lựa chọn khác để định nghĩa số hạng chính quy hóa R. Ví dụ, một cách ép tính thưa là phạt những trường hợp mà giá trị trung bình của mỗi phần tử h_k cách xa một giá trị định trước [Nair và Hinton, 2009]. Khi h_k nhận giá trị trong $[0, 1]$, việc chính quy hóa có thể được cài đặt bằng cách định nghĩa R là độ phân kỳ KL (*KL divergence*) giữa mã trung bình trên một số mẫu và mã kỳ vọng (*expected code*)[12]. Gọi $\bar{\mathbf{h}}$ là mã trung bình trên $\{\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(m)}\}$, trong đó giá trị $\bar{h}_k$ là trung bình của biến thứ k của mã:

$$\bar{h}_k = \frac{1}{m} \cdot \sum_{i=1}^{m} \text{Encode}(\mathbf{x}^{(i)})(k). \tag{2.100}$$

Đồng thời, gọi $\mathbf{q}$ là mã kỳ vọng, trong đó $q_k = \tau$ với mọi k. Nếu mỗi phần tử của mã trung bình được xem là một biến ngẫu nhiên Bernoulli có trung bình $\bar{h}_k$, và mỗi phần tử của mã kỳ vọng được xem là một biến ngẫu nhiên Bernoulli khác có trung bình τ, thì số hạng chính quy hóa có thể được định nghĩa là tổng độ phân kỳ KL giữa $\bar{h}_k$ và q_k trên tất cả các phần

[12]Nói chung, ta có thể đặt mọi phần tử của mã kỳ vọng bằng $\tau \in [0, 1]$. Các mã thưa sẽ được ưu tiên nếu τ gần 0, vì trong hầu hết các trường hợp các đặc trưng đều "không hoạt động". Ngược lại, các mã dày đặc sẽ được ưu tiên nếu τ gần 1.

tử:

$$R = \sum_k D_{\mathrm{KL}}(q_k \,\|\, \bar{h}_k)$$
$$= \sum_k \tau \cdot \log \frac{\tau}{\bar{h}_k} + (1-\tau) \cdot \log \frac{1-\tau}{1-\bar{h}_k}. \tag{2.101}$$

Ở dạng này, R phạt mô hình khi $\bar{\mathbf{h}}$ lệch khỏi $\mathbf{q}$.

Chú giải của người dịch: Ý nghĩa của Phương trình (2.101) qua ví dụ số

Độ phân kỳ KL đo mức "khác nhau" giữa hai phân phối: bằng 0 khi chúng trùng nhau và dương khi chúng khác nhau. Ở đây mỗi phần tử k được coi như một "công tắc" bật với xác suất $\bar{h}_k$, và ta muốn nó bật với xác suất nhỏ τ. Lấy $\tau = 0.05$ (dùng logarit tự nhiên):

- Nếu $\bar{h}_k = 0.05$: hai số hạng đều là $\log 1 = 0$, nên không bị phạt.
- Nếu $\bar{h}_k = 0.5$ (nơ-ron bật quá thường xuyên): $0.05 \log \frac{0.05}{0.5} + 0.95 \log \frac{0.95}{0.5} \approx -0.115 + 0.610 = 0.495$, bị phạt khá nặng.

Như vậy, số hạng này "kéo" tần suất bật trung bình của mỗi nơ-ron trong mã về gần τ, khiến mã trở nên thưa.

Một biến thể khác của bộ tự mã hóa là **bộ tự mã hóa co** (*contractive auto-encoder, CAE*), cố gắng tăng tính vững của biểu diễn bằng cách đưa một số hạng chính quy hóa mới vào quá trình huấn luyện [Rifai và cộng sự, 2011]:

$$R = \sum_{i=1}^{m} \left\| \frac{\partial \mathbf{h}^{(i)}}{\partial \mathbf{x}^{(i)}} \right\|_F^2$$
$$= \sum_{i=1}^{m} \left\| \frac{\partial \mathrm{Encode}(\mathbf{x}^{(i)})}{\partial \mathbf{x}^{(i)}} \right\|_F^2, \tag{2.102}$$

trong đó $\frac{\partial \mathrm{Encode}(\mathbf{x}^{(i)})}{\partial \mathbf{x}^{(i)}}$ (hay $\frac{\partial \mathbf{h}^{(i)}}{\partial \mathbf{x}^{(i)}}$) là **ma trận Jacobi** (*Jacobian matrix*) của biểu diễn[13], và $\|\cdot\|_F$ là **chuẩn Frobenius** (*Frobenius norm*) của một ma

[13]Giả sử bộ mã hóa là một hàm $\mathrm{Encode}(\cdot)$: $\mathbf{x} \in \mathbb{R}^{d_x} \to \mathbf{h} \in \mathbb{R}^{d_h}$. Ma trận Jacobi của

trận[14]. Hàm phạt co giúp chống lại ảnh hưởng của những nhiễu loạn nhỏ ở đầu vào. Theo nghĩa hình học, nó khuyến khích quan hệ lân cận được giữ nguyên ở các điểm dữ liệu đầu ra nếu các điểm dữ liệu đầu vào là những láng giềng gần nhau; nói cách khác, nó buộc Encode$(\cdot)$ hành xử giống một **ánh xạ co** (*contraction mapping*)[15] hơn, từ đó có tên bộ tự mã hóa co.

Chú giải của người dịch: Ma trận Jacobi và ý nghĩa của hàm phạt co

Mỗi phần tử $\frac{\partial h_j}{\partial x_i}$ của ma trận Jacobi cho biết: nếu ta nhích x_i một chút thì h_j thay đổi nhiều hay ít — giống đạo hàm $f'(x)$ ở THPT, nhưng giờ có nhiều đầu vào và nhiều đầu ra nên ta xếp tất cả các "đạo hàm" đó thành một bảng. Chuẩn Frobenius bình phương chỉ đơn giản là tổng bình phương mọi phần tử của bảng, ví dụ $\left\| \begin{bmatrix} 1 & 2 \\ 0 & -1 \end{bmatrix} \right\|_F^2 = 1+4+0+1 = 6$. Phạt đại lượng này nghĩa là ép mã **h** ít nhạy với những thay đổi nhỏ của **x**. Ví dụ dễ hình dung về ánh xạ co: $f(x) = x/2$, vì $|f(x_1) - f(x_2)| = \frac{1}{2}|x_1 - x_2|$ — hai điểm

---

$\mathbf{h} = \text{Encode}(\mathbf{x})$ là một ma trận $d_h \times d_x$:

$$\begin{aligned} \text{Jacobian} &= \frac{\partial \mathbf{h}}{\partial \mathbf{x}} \\ &= \begin{bmatrix} \frac{\partial \mathbf{h}}{\partial x_1} & \cdots & \frac{\partial \mathbf{h}}{\partial x_{d_x}} \end{bmatrix} \\ &= \begin{bmatrix} \frac{\partial h_1}{\partial x_1} & \cdots & \frac{\partial h_1}{\partial x_{d_x}} \\ \vdots & \ddots & \vdots \\ \frac{\partial h_{d_h}}{\partial x_1} & \cdots & \frac{\partial h_{d_h}}{\partial x_{d_x}} \end{bmatrix}. \end{aligned} \tag{2.103}$$

[14]Với một ma trận $\mathbf{A} \in \mathbb{R}^{d_x \times d_h}$, chuẩn Frobenius về bản chất là chuẩn l_2 của dạng vectơ được "duỗi phẳng" của ma trận, cho bởi phương trình:

$$\|\mathbf{A}\|_F = \sqrt{\sum_{i,j} A_{i,j}^2}. \tag{2.104}$$

Lưu ý rằng trong đại số tuyến tính chặt chẽ, chuẩn l_2 của một ma trận là giá trị kỳ dị lớn nhất của nó (chuẩn phổ); đó là lý do ở đây thuật ngữ chuẩn Frobenius được dùng một cách tường minh để mô tả độ lớn tính theo từng phần tử.

[15]Cho X là một không gian metric với metric d. Cho một hàm $f(\cdot)$ từ X vào X, $f(\cdot)$ là một *ánh xạ co* nếu tồn tại một hằng số $\epsilon \in [0, 1)$ sao cho với mọi $x_1, x_2 \in X$:

$$d(f(x_1), f(x_2)) \leq \epsilon \cdot d(x_1, x_2). \tag{2.105}$$

gần nhau sau khi qua f lại càng gần nhau hơn.

2.6.2 Bộ tự mã hóa khử nhiễu

Một nguồn cảm hứng khác để tăng tính vững của mô hình đến từ ý tưởng **khử nhiễu** (*denoising*): ta thêm nhiễu vào đầu vào rồi loại bỏ nó để khôi phục dữ liệu sạch. **Bộ tự mã hóa khử nhiễu** (*denoising auto-encoders, DAEs*) là một lớp mạng nơ-ron kết hợp nguyên lý tự mã hóa và khử nhiễu. Đầu tiên, nhiễu được thêm vào vectơ đầu vào một cách ngẫu nhiên. Có thể mô tả điều này như một quá trình sinh ra đầu vào có nhiễu $\mathbf{x}_{\text{noise}} \in \mathbb{R}^{d_x}$ khi biết đầu vào ban đầu $\mathbf{x} \in \mathbb{R}^{d_x}$:

$$\mathbf{x}_{\text{noise}} \sim \Pr_{\text{noise}}(\mathbf{x}_{\text{noise}}|\mathbf{x}), \tag{2.106}$$

trong đó $\Pr_{\text{noise}}(\cdot|\mathbf{x})$ là một phân phối có điều kiện dùng để lấy mẫu $\mathbf{x}_{\text{noise}}$. Ví dụ, ta có thể làm theo phương pháp đã trình bày ở Mục 2.5.5 và rút đầu vào có nhiễu từ một phân phối Gauss nhiều chiều:

$$\mathbf{x}_{\text{noise}} \sim \mathcal{N}(\mathbf{x}, \sigma^2\mathbf{I}), \tag{2.107}$$

trong đó $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ ký hiệu một phân phối Gauss với vectơ trung bình $\boldsymbol{\mu} \in \mathbb{R}^{d_x}$ và ma trận hiệp phương sai (*covariance matrix*) $\boldsymbol{\Sigma} \in \mathbb{R}^{d_x \times d_x}$. Một cách tương đương, nhiễu cộng này có thể được viết tường minh bằng cách tách riêng số hạng nhiễu có trung bình bằng 0:

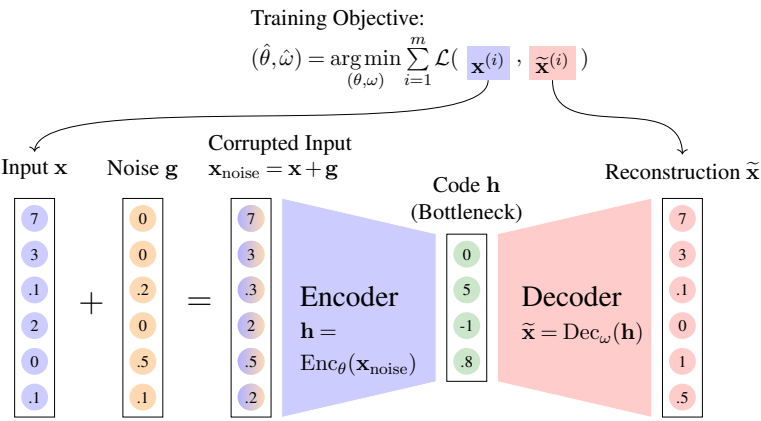
$$\mathbf{x}_{\text{noise}} = \mathbf{x} + \mathbf{g}, \tag{2.108}$$

$$\mathbf{g} \sim \mathcal{N}(\mathbf{0}, \sigma^2\mathbf{I}), \tag{2.109}$$

trong đó $\sigma^2\mathbf{I}$ là một ma trận hiệp phương sai đường chéo, cho biết nhiễu Gauss độc lập với phương sai σ^2 được áp dụng vào từng chiều của đầu vào, và $\mathbf{I}$ là ma trận đơn vị $d_x \times d_x$.

Đôi khi quá trình này được gọi là sự **làm hỏng** (*corruption*) đầu vào, và $\mathbf{x}_{\text{noise}}$ được gọi là **đầu vào bị làm hỏng** (*corrupted input*). Ngoài nhiễu Gauss cộng, còn có một vài cách khác để làm hỏng đầu vào [Vincent và cộng sự, 2010]. Một trong những phương pháp phổ biến là đặt bằng 0 một số phần tử của $\mathbf{x}$. Ví dụ, ta có thể đặt mỗi phần tử bằng 0 với một xác suất định trước. Cách này còn được gọi là **nhiễu che** (*masking noise*).

Một phương pháp khác là dùng **nhiễu muối tiêu** (*salt-and-pepper noise*) hay **nhiễu xung** (*impulse noise*) để làm hỏng. Nó chọn ngẫu nhiên một số phần tử và đặt mỗi phần tử đó bằng giá trị nhỏ nhất hoặc lớn nhất với một xác suất định trước. Các loại nhiễu khác nhau được dùng cho những ứng dụng khác nhau của bộ tự mã hóa. Ví dụ, nhiễu che rất phổ biến trong việc huấn luyện mô hình ngôn ngữ, còn nhiễu muối tiêu được dùng phổ biến hơn trong xử lý ảnh.



Hình 2.17: Minh họa một bộ tự mã hóa khử nhiễu. Một đầu vào $\mathbf{x}$ trước tiên bị làm hỏng thành đầu vào có nhiễu hay đầu vào bị làm hỏng $\mathbf{x}_{\text{noise}}$. Sau đó, nó được đưa qua một bộ mã hóa để tạo thành mã $\mathbf{h}$. Tiếp theo, mã được đưa qua một bộ giải mã để tạo thành đầu vào tái tạo $\widetilde{\mathbf{x}}$. Việc huấn luyện được thực hiện bằng cách cực tiểu hóa mất mát giữa $\mathbf{x}$ và $\widetilde{\mathbf{x}}$. Quá trình này được gọi là "khử nhiễu" vì nó cố gắng loại bỏ nhiễu khỏi $\mathbf{x}_{\text{noise}}$ và khôi phục đầu vào ban đầu $\mathbf{x}$.

Nhãn trong hình: *Noise*: nhiễu; *Corrupted Input*: đầu vào bị làm hỏng; các nhãn khác như ở Hình 2.16.

Sau đó, đầu vào bị làm hỏng $\mathbf{x}_{\text{noise}}$ được đưa vào một mạng mã hóa–giải mã, và mạng này tạo ra đầu vào tái tạo $\widetilde{\mathbf{x}} = \text{Decode}(\text{Encode}(\mathbf{x}_{\text{noise}}))$. Quá trình huấn luyện diễn ra như bình thường. Ta dùng lại Phương trình (2.95) để cực tiểu hóa mất mát khi thay $\mathbf{x}$ bằng $\widetilde{\mathbf{x}}$. Như vậy, ta có thể viết lại Phương trình (2.95) để điều chỉnh hàm mục tiêu cho trường hợp khử nhiễu:

$$(\hat{\theta}, \hat{\omega}) = \arg\min_{(\theta,\omega)} \sum_{i=1}^{m} \mathcal{L}\Big(\mathbf{x}^{(i)}, \text{Decode}_{\omega}(\text{Encode}_{\theta}(\mathbf{x}^{(i)}_{\text{noise}}))\Big). \quad (2.110)$$

Phương trình (2.110) chỉ khác Phương trình (2.95) ở chỗ đầu vào của bộ tự mã hóa là $\mathbf{x}_{\text{noise}}$ thay vì $\mathbf{x}$. Nói cách khác, ta khử nhiễu đầu vào bị làm hỏng để khôi phục đầu vào ban đầu. Xem Hình 2.17 để biết cấu trúc của bộ tự mã hóa khử nhiễu.

Chú giải của người dịch: Khử nhiễu: "đoán chữ bị che"

Hãy nhớ lại trò chơi điền vào chỗ trống: "Hà Nội là ___ đô của Việt Nam". Muốn điền đúng chữ "thủ", ta phải hiểu cả câu. Bộ tự mã hóa khử nhiễu làm đúng như vậy: đầu vào bị làm hỏng (một số phần bị che hoặc bị nhiễu), nhưng đáp án mà mô hình phải tái tạo là bản *sạch* $\mathbf{x}$. Vì không thể chỉ "chép" đầu vào, mô hình buộc phải học quy luật bên trong dữ liệu. Đây chính là ý tưởng "nhiễu che" mà các mô hình ngôn ngữ như BERT (Chương 7) dùng khi che ngẫu nhiên một số từ rồi bắt mô hình đoán lại.

Lưu ý rằng cả bộ tự mã hóa co/thưa lẫn bộ tự mã hóa khử nhiễu đều có thể được xem là những cách tăng tính vững của bộ tự mã hóa. Điểm khác biệt nằm ở chỗ chúng chính quy hóa việc huấn luyện tại những vị trí khác nhau của mô hình. Bộ tự mã hóa co nhắm tới việc tăng tính vững của quá trình mã hóa, tức là biểu diễn được huấn luyện để không quá nhạy với những nhiễu loạn nhỏ ở đầu vào. Trong khi đó, bộ tự mã hóa khử nhiễu nhắm tới việc tăng tính vững của quá trình tái tạo. Chúng tác động đồng thời lên cả bộ mã hóa lẫn bộ giải mã. Theo một nghĩa nào đó, bộ tự mã hóa khử nhiễu là ứng dụng trực tiếp của huấn luyện có nhiễu vào bộ tự mã hóa (xem Mục 2.5.5). Tất nhiên, khó có thể nói mô hình nào tốt hơn. Ví dụ, bộ tự mã hóa co có sự dẫn dắt trực tiếp hơn đối với việc học biểu diễn, vốn thường là mục tiêu chính của chúng ta trong học đặc trưng. Việc huấn luyện bộ tự mã hóa khử nhiễu, dù chỉ có tác động gián tiếp lên quá trình mã hóa, lại nhận thêm tín hiệu khử nhiễu từ bộ giải mã. Điều này mang lại một góc nhìn mới về huấn luyện vững: một biểu diễn vững có thể được học ở cả điểm sinh ra nó (bộ mã hóa khử nhiễu) lẫn điểm sử dụng nó (bộ giải mã khử nhiễu).

2.6.3 Bộ tự mã hóa biến phân

Dù mang tên như vậy, **bộ tự mã hóa biến phân** (*variational auto-encoders, VAEs*) ban đầu không được đề xuất để giải các bài toán mã hóa biểu diễn thông thường. Về cơ bản, chúng được thiết kế để sinh ra dữ liệu

mới tương tự dữ liệu quan sát được, do đó có cách phát biểu toán học rất khác so với các bộ tự mã hóa kinh điển đã bàn ở trên. Trong thống kê và học máy, bộ tự mã hóa biến phân thường được xem là những ví dụ của các phương pháp Bayes biến phân (*variational Bayesian methods*) và được dùng để thực hiện suy diễn thống kê (*statistical inference*) hiệu quả trên các biến ẩn (*latent variables*) khi xác suất hậu nghiệm của các biến này không tính được (*intractable*) [Kingma và Welling, 2014; 2019]. Mặt khác, bộ tự mã hóa biến phân, dù ngầm định hay tường minh, cũng giải quyết đúng việc mà ta làm khi suy ra biểu diễn ẩn bên dưới của một đối tượng quan sát được. Vì vậy, chúng tôi đưa chúng vào mục này để phần thảo luận được tương đối đầy đủ.

Ta bắt đầu với một "câu chuyện sinh" mô tả cách mỗi điểm dữ liệu được tạo ra. Giả sử rằng, với một mẫu quan sát được $\mathbf{x}$ trong tập dữ liệu, có một biến ẩn $\mathbf{h}$ không quan sát được mô tả $\mathbf{x}$. Bây giờ ta muốn xây dựng một mô hình xác suất mô tả quá trình sinh ra $\mathbf{x}$, cụ thể là ước lượng xác suất $\Pr(\mathbf{x})$. Xác suất này có thể được tính như phân phối biên (*marginal distribution*):

$$\Pr(\mathbf{x}) \;=\; \int \Pr(\mathbf{x}, \mathbf{h}) d\mathbf{h}, \tag{2.111}$$

trong đó ta đưa biến ẩn $\mathbf{h}$ một cách tường minh vào việc suy diễn $\mathbf{x}$. Để giải Phương trình (2.111), ta dùng một mô hình $p_\omega(\mathbf{x}, \mathbf{h})$ để xấp xỉ $\Pr(\mathbf{x}, \mathbf{h})$ (tức $p_\omega(\mathbf{x}, \mathbf{h}) \approx \Pr(\mathbf{x}, \mathbf{h})$), và ta có

$$p_\omega(\mathbf{x}) \;=\; \int p_\omega(\mathbf{x}, \mathbf{h}) d\mathbf{h}, \tag{2.112}$$

trong đó $p_\omega(\mathbf{x}, \mathbf{h})$ là một hàm mật độ xác suất (*probability density function*) được tham số hóa bởi ω. Ta thay vế trái của Phương trình (2.112) bằng $p_\omega(\mathbf{x})$ để nhấn mạnh rằng xác suất được xác định bởi mô hình $p_\omega(\cdot)$. Nói chung có nhiều cách định nghĩa $p_\omega(\mathbf{x}, \mathbf{h})$. Ở đây ta có thể đơn giản coi nó là một mạng nơ-ron.

Sau đó, ta có thể viết lại Phương trình (2.112) bằng quy tắc chuỗi (*chain rule*) (của xác suất):

$$p_\omega(\mathbf{x}) \;=\; \int p_\omega(\mathbf{h}) \cdot p_\omega(\mathbf{x}|\mathbf{h}) d\mathbf{h}, \tag{2.113}$$

trong đó $p_\omega(\mathbf{h})$ là phân phối tiên nghiệm (*prior*) trên $\mathbf{h}$, ví dụ một tiên nghiệm Gauss. Xác suất có điều kiện $p_\omega(\mathbf{x}|\mathbf{h})$ mô tả khả năng quan sát

được $\mathbf{x}$ khi biết biến ẩn $\mathbf{h}$. Để mô hình hóa quá trình sinh này, $p_\omega(\mathbf{x}|\mathbf{h})$ thường được giả định là một phân phối Gauss được tham số hóa bởi trung bình $\boldsymbol{\mu}_p$ và phương sai σ_p của nó:

$$p_\omega(\mathbf{x}|\mathbf{h}) \;=\; \mathcal{N}(\boldsymbol{\mu}_p, \sigma_p^2\mathbf{I}), \tag{2.114}$$

trong đó $\boldsymbol{\mu}_p \in \mathbb{R}^{d_x}$ và $\sigma_p \in \mathbb{R}$ được xác định bởi một mạng giải mã $\text{Decode}_\omega(\cdot)$ (chúng tôi sẽ giải thích sau vì sao nó được gọi là "giải mã"):

$$(\boldsymbol{\mu}_p, \sigma_p) \;=\; \text{Decode}_\omega(\mathbf{h}). \tag{2.115}$$

Chú giải của người dịch: Biến ẩn và "câu chuyện sinh"

Hãy hình dung việc vẽ một khuôn mặt: trước tiên ta (ngầm) chọn vài "yếu tố ẩn" như tuổi, kiểu tóc, nụ cười — đó là $\mathbf{h}$; rồi từ các yếu tố đó mới vẽ ra bức ảnh cụ thể — đó là $\mathbf{x}$. Ta chỉ nhìn thấy bức ảnh, không thấy $\mathbf{h}$. Phương trình (2.111) nói rằng xác suất của một bức ảnh bằng "tổng" (tích phân) xác suất của mọi cách chọn yếu tố ẩn có thể dẫn tới bức ảnh đó — giống công thức xác suất đầy đủ: $P(A) = \sum_i P(B_i)\,P(A|B_i)$, chỉ khác là $\mathbf{h}$ liên tục nên tổng thành tích phân. Ký hiệu $p(\mathbf{x}|\mathbf{h})$ đọc là "xác suất của $\mathbf{x}$ khi đã biết $\mathbf{h}$" (xác suất có điều kiện).

Tuy nhiên, hàm hợp lý biên (*marginal likelihood*) trong Phương trình (2.112) vẫn không tính được. Dù ta có thể dễ dàng tính tiên nghiệm $p_\omega(\mathbf{h})$ và xác suất có điều kiện $p_\omega(\mathbf{x}|\mathbf{h})$, việc thực hiện chính xác phép tích phân trên toàn bộ không gian ẩn liên tục của $\mathbf{h}$ là quá tốn kém về tính toán. Điều này cũng dẫn tới một phân phối hậu nghiệm (*posterior*) không tính được:

$$p_\omega(\mathbf{h}|\mathbf{x}) \;=\; \frac{p_\omega(\mathbf{h}) \cdot p_\omega(\mathbf{x}|\mathbf{h})}{p_\omega(\mathbf{x})}. \tag{2.116}$$

Có vẻ như ta bị mắc kẹt với $p_\omega(\mathbf{x})$ và $p_\omega(\mathbf{h}|\mathbf{x})$! Bộ tự mã hóa biến phân giải quyết vấn đề này bằng cách xấp xỉ $p_\omega(\mathbf{h}|\mathbf{x})$ bằng một phân phối hậu nghiệm tính được $q_\theta(\mathbf{h}|\mathbf{x})$:

$$q_\theta(\mathbf{h}|\mathbf{x}) \;\approx\; p_\omega(\mathbf{h}|\mathbf{x}), \tag{2.117}$$

trong đó θ là tham số của mô hình mới. Giống như Phương trình (2.114–2.115), $q_\theta(\mathbf{h}|\mathbf{x})$ được định nghĩa là một phân phối Gauss khác:

$$q_\theta(\mathbf{h}|\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}_q, \sigma_q^2\mathbf{I}), \tag{2.118}$$

$$(\boldsymbol{\mu}_q, \sigma_q) = \text{Encode}_\theta(\mathbf{x}), \tag{2.119}$$

trong đó $\text{Encode}_\theta(\cdot)$ là mạng mã hóa đọc $\mathbf{x}$ và sinh ra trung bình và phương sai của phân phối $q_\theta(\mathbf{h}|\mathbf{x})$. Điều này thật thú vị! Giờ đây ta có một con đường khả thi để tính $\Pr(\mathbf{x})$: ta có thể lấy mẫu một biến ẩn $\mathbf{h}$ từ phân phối hậu nghiệm biến phân (*variational posterior*) $q_\theta(\mathbf{h}|\mathbf{x})$, rồi tính xác suất đồng thời (*joint probability*) $p_\omega(\mathbf{x}, \mathbf{h})$. Về mặt toán học, ta có thể viết lại log-hợp lý (*log-likelihood*) của quan sát dưới dạng một kỳ vọng theo phân phối hậu nghiệm biến phân:

$$\begin{aligned}
&\log p_\omega(\mathbf{x}) \\
&= \mathbb{E}_{\mathbf{h}\sim q_\theta(\mathbf{h}|\mathbf{x})}\left[\log p_\omega(\mathbf{x})\right] \\
&= \mathbb{E}_{\mathbf{h}\sim q_\theta(\mathbf{h}|\mathbf{x})}\left[\log \frac{p_\omega(\mathbf{h}) \cdot p_\omega(\mathbf{x}|\mathbf{h})}{p_\omega(\mathbf{h}|\mathbf{x})}\right] \\
&= \mathbb{E}_{\mathbf{h}\sim q_\theta(\mathbf{h}|\mathbf{x})}\left[\log \frac{p_\omega(\mathbf{h}) \cdot p_\omega(\mathbf{x}|\mathbf{h})}{p_\omega(\mathbf{h}|\mathbf{x})} \cdot \frac{q_\theta(\mathbf{h}|\mathbf{x})}{q_\theta(\mathbf{h}|\mathbf{x})}\right] \\
&= \mathbb{E}_{\mathbf{h}\sim q_\theta(\mathbf{h}|\mathbf{x})}\left[\log \frac{q_\theta(\mathbf{h}|\mathbf{x})}{p_\omega(\mathbf{h}|\mathbf{x})}\right] + \mathbb{E}_{\mathbf{h}\sim q_\theta(\mathbf{h}|\mathbf{x})}\left[\log \frac{p_\omega(\mathbf{h}) \cdot p_\omega(\mathbf{x}|\mathbf{h})}{q_\theta(\mathbf{h}|\mathbf{x})}\right].
\end{aligned} \tag{2.120}$$

Chú giải của người dịch: Đọc từng bước của Phương trình (2.120)

- Bước 1: $\log p_\omega(\mathbf{x})$ không phụ thuộc vào $\mathbf{h}$, nên "lấy trung bình" nó theo $\mathbf{h}$ vẫn được chính nó (trung bình của một hằng số là hằng số đó).
- Bước 2: dùng công thức Bayes (2.116) đảo lại: $p_\omega(\mathbf{x}) = \frac{p_\omega(\mathbf{h})\, p_\omega(\mathbf{x}|\mathbf{h})}{p_\omega(\mathbf{h}|\mathbf{x})}$.
- Bước 3: nhân với $\frac{q}{q} = 1$, không làm thay đổi giá trị — một "mẹo" quen thuộc kiểu thêm bớt ở THPT.
- Bước 4: dùng $\log(ab) = \log a + \log b$ để tách thành hai phần, và kỳ vọng của một tổng bằng tổng các kỳ vọng.

Ký hiệu $\mathbb{E}_{\mathbf{h}\sim q}[\cdot]$ là *kỳ vọng* (giá trị trung bình) của biểu thức trong ngoặc khi $\mathbf{h}$ được rút ngẫu nhiên theo phân phối q.

Số hạng thứ nhất ở vế phải của Phương trình (2.120) chính là độ phân kỳ KL (hay entropy tương đối (*relative entropy*)) giữa phân phối

hậu nghiệm xấp xỉ $q_\theta(\mathbf{h}|\mathbf{x})$ và phân phối hậu nghiệm thật $p_\omega(\mathbf{h}|\mathbf{x})$:

$$D_{\mathrm{KL}}\big(q_\theta(\mathbf{h}|\mathbf{x}) \,\|\, p_\omega(\mathbf{h}|\mathbf{x})\big) \;=\; \mathbb{E}_{\mathbf{h}\sim q_\theta(\mathbf{h}|\mathbf{x})}\Big[\log \frac{q_\theta(\mathbf{h}|\mathbf{x})}{p_\omega(\mathbf{h}|\mathbf{x})}\Big]. \tag{2.121}$$

Vì độ phân kỳ KL luôn không âm ($D_{\mathrm{KL}}(q \,\|\, p) \geq 0$, và bằng 0 khi và chỉ khi $q = p$), ta có thể thiết lập một cận dưới chặt chẽ cho log-hợp lý:

$$\begin{aligned}
\log p_\omega(\mathbf{x}) &\geq \mathbb{E}_{\mathbf{h}\sim q_\theta(\mathbf{h}|\mathbf{x})}\Big[\log \frac{p_\omega(\mathbf{h}) \cdot p_\omega(\mathbf{x}|\mathbf{h})}{q_\theta(\mathbf{h}|\mathbf{x})}\Big] \\
&= \mathbb{E}_{\mathbf{h}\sim q_\theta(\mathbf{h}|\mathbf{x})}\Big[\log p_\omega(\mathbf{x}|\mathbf{h}) + \log \frac{p_\omega(\mathbf{h})}{q_\theta(\mathbf{h}|\mathbf{x})}\Big] \\
&= \mathbb{E}_{\mathbf{h}\sim q_\theta(\mathbf{h}|\mathbf{x})}\big[\log p_\omega(\mathbf{x}|\mathbf{h})\big] - D_{\mathrm{KL}}\big(q_\theta(\mathbf{h}|\mathbf{x}) \,\|\, p_\omega(\mathbf{h})\big).
\end{aligned} \tag{2.122}$$

Vế phải của Phương trình (2.122) tạo thành **cận dưới bằng chứng** (*evidence lower bound, ELBO*) nền tảng của hàm hợp lý dữ liệu. Số hạng thứ nhất của ELBO (log-hợp lý tái tạo kỳ vọng) có thể được xấp xỉ bằng cách lấy mẫu Monte Carlo (*Monte Carlo sampling*) **h**. Số hạng thứ hai dễ tính vì độ phân kỳ KL giữa hai phân phối Gauss có lời giải giải tích. Gọi $\mathcal{L}(\mathbf{x}, \theta, \omega)$ là ELBO lấy dấu âm. Khi đó, quá trình huấn luyện một bộ tự mã hóa biến phân được phát biểu là cực tiểu hóa $\mathcal{L}(\cdot)$ trên một tập dữ liệu gồm các mẫu quan sát được $\{\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(m)}\}$:

$$(\hat{\theta}, \hat{\omega}) \;=\; \arg\min_{(\theta,\omega)} \sum_{i=1}^{m} \mathcal{L}\Big(\mathbf{x}^{(i)}, \theta, \omega\Big). \tag{2.123}$$

Lưu ý rằng vì việc lấy mẫu **h** từ $q_\theta(\mathbf{h} \mid \mathbf{x})$ là một phép toán ngẫu nhiên, không khả vi, nên không thể tính trực tiếp gradient của nó. Để đưa việc huấn luyện bộ tự mã hóa vào khuôn khổ lan truyền ngược tiêu chuẩn, một cách thường dùng là sử dụng cái gọi là **mẹo tái tham số hóa** (*reparameterization trick*). Ở đây chúng tôi bỏ qua chi tiết và giới thiệu bạn đọc một vài bài báo để biết thêm thông tin [Kingma và Welling, 2014; Doersch, 2016].

Chú giải của người dịch: ELBO nói gì một cách trực quan?

Phương trình (2.122) cho thấy cực đại hóa ELBO là cân bằng hai mong muốn: (i) số hạng thứ nhất — từ mã $\mathbf{h}$ phải tái tạo lại $\mathbf{x}$ thật tốt (giống mất mát tái tạo của bộ tự mã hóa thông thường); (ii) số hạng thứ hai — phân phối mã $q_\theta(\mathbf{h}|\mathbf{x})$ không được lệch quá xa phân phối tiên nghiệm $p_\omega(\mathbf{h})$ (thường là phân phối Gauss chuẩn), giống như một số hạng chính quy hóa. Nhờ (ii), không gian mã trở nên "gọn gàng", nên sau này rút ngẫu nhiên một $\mathbf{h}$ từ tiên nghiệm rồi giải mã sẽ ra dữ liệu hợp lý.

Về mẹo tái tham số hóa (chỉ nêu ý chính): thay vì rút trực tiếp $\mathbf{h} \sim \mathcal{N}(\boldsymbol{\mu}_q, \sigma_q^2\mathbf{I})$, ta rút $\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ rồi tính $\mathbf{h} = \boldsymbol{\mu}_q + \sigma_q \cdot \boldsymbol{\varepsilon}$. Tính ngẫu nhiên được "đẩy" sang $\boldsymbol{\varepsilon}$, còn $\mathbf{h}$ là một hàm bình thường của $\boldsymbol{\mu}_q, \sigma_q$ nên lấy đạo hàm và lan truyền ngược được.

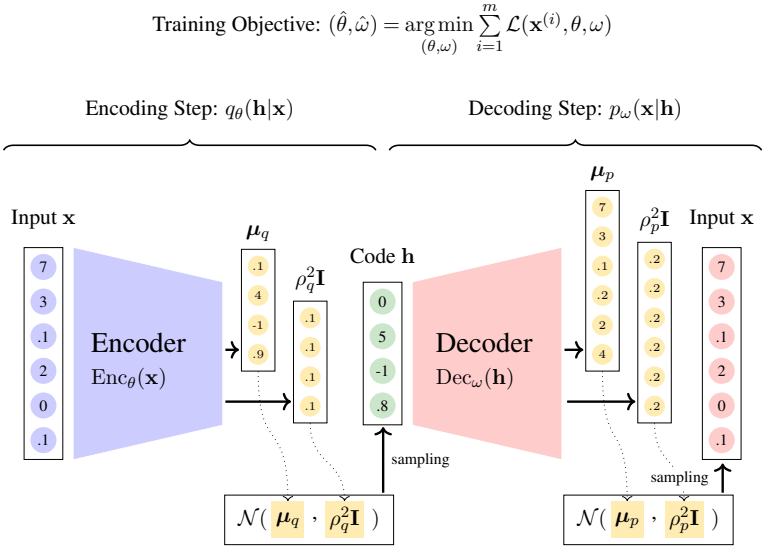
Hình 2.18 minh họa cách một bộ tự mã hóa biến phân hoạt động. Nó gồm một quá trình sinh hai bước:

- **Mã hóa**. Với một mẫu đầu vào $\mathbf{x}$, ta lấy mẫu một biến ẩn $\mathbf{h}$ từ $q_\theta(\mathbf{h}|\mathbf{x})$. Bước này cần một mạng mã hóa $\text{Encode}_\theta(\mathbf{x})$ sinh ra trung bình $\boldsymbol{\mu}_q$ và phương sai σ_q^2 của phân phối Gauss $q_\theta(\mathbf{h}|\mathbf{x})$. Sau đó, biến ẩn được sinh ra bằng cách lấy mẫu từ $\mathcal{N}(\boldsymbol{\mu}_q, \sigma_q^2\mathbf{I})$.

- **Giải mã**. Với biến ẩn $\mathbf{h}$, ta lấy mẫu đầu vào ban đầu $\mathbf{x}$ từ $p_\omega(\mathbf{x}|\mathbf{h})$. Bước này lại theo một quá trình lấy mẫu Gauss: một mạng giải mã $\text{Decode}_\omega(\mathbf{h})$ được dùng để xác định trung bình $\boldsymbol{\mu}_p$ và phương sai σ_p^2 của phân phối. $\mathbf{x}$ được sinh ra theo $\mathcal{N}(\boldsymbol{\mu}_p, \sigma_p^2\mathbf{I})$.

Nhãn trong hình: *Encoding Step/Decoding Step*: bước mã hóa/bước giải mã; *sampling*: lấy mẫu; các nhãn khác như ở Hình 2.16. Ký hiệu $\rho_q^2\mathbf{I}$, $\rho_p^2\mathbf{I}$ trong hình ứng với $\sigma_q^2\mathbf{I}$, $\sigma_p^2\mathbf{I}$ trong văn bản.

Đôi khi $q_\theta(\mathbf{h}|\mathbf{x})$ và $p_\omega(\mathbf{x}|\mathbf{h})$ được gọi là "bộ mã hóa" và "bộ giải mã", vì chúng cố gắng "ánh xạ" một đầu vào thành một biểu diễn rồi "ánh xạ" nó trở lại đầu vào. Tuy nhiên, bản thân $q_\theta(\mathbf{h}|\mathbf{x})$ và $p_\omega(\mathbf{x}|\mathbf{h})$ hàm ý những mô hình không tất định, tức là chúng xuất ra hàm mật độ xác suất của các biến thay vì các ước lượng điểm (*point estimates*). Một hệ quả quan trọng của kết quả này là bộ tự mã hóa biến phân không có xu hướng tìm biểu diễn "tốt nhất" cho đầu vào. Thoạt nhìn, điều này có vẻ phản trực giác, vì các mô hình thông thường thường cho ra các biểu diễn có giá trị cố định. Tuy nhiên, đây chính là đặc điểm của suy diễn Bayes — ta chỉ học một phân phối trên các giá trị khả dĩ của một biến ẩn. Từ góc độ thực tiễn, nếu



Hình 2.18: Minh họa một bộ tự mã hóa biến phân. Với một mẫu đầu vào $\mathbf{x}$, ta sinh ra một biến ẩn $\mathbf{h}$ bằng một bộ mã hóa $q_\theta(\mathbf{h}|\mathbf{x})$. Trong bước mã hóa, trước tiên một mạng nơ-ron $\text{Enc}_\theta(\cdot)$ được dùng để tạo ra trung bình và phương sai của một phân phối Gauss, tức $\boldsymbol{\mu}_q$ và σ_q^2. Sau đó, biến ẩn $\mathbf{h}$ được rút theo $\mathcal{N}(\boldsymbol{\mu}_q, \sigma_q^2\mathbf{I})$. Tiếp theo, ta tái sinh mẫu ban đầu $\mathbf{x}$ bằng một bộ giải mã $p_\omega(\mathbf{x}|\mathbf{h})$. Trong bước giải mã, tương tự quá trình sinh ở bộ mã hóa, một mạng nơ-ron $\text{Dec}_\omega(\cdot)$ được dùng để sinh ra trung bình $\boldsymbol{\mu}_p$ và phương sai σ_p^2 của $\mathcal{N}(\boldsymbol{\mu}_p, \sigma_p^2\mathbf{I})$. Cùng đầu vào $\mathbf{x}$ đó được sinh ra bằng cách lấy mẫu từ $\mathcal{N}(\boldsymbol{\mu}_p, \sigma_p^2\mathbf{I})$.

tác vụ hạ nguồn cần một biểu diễn tất định, ta có thể đơn giản dùng giá trị kỳ vọng (tức trung bình $\boldsymbol{\mu}_q$) của phân phối suy ra được, vốn nắm bắt một cách đáng tin cậy các đặc trưng thiết yếu của đầu vào [Knight, 2009].

Trong thực tế, công dụng chính của bộ tự mã hóa biến phân là để sinh dữ liệu chứ không phải để biểu diễn. Khi kiểm tra, với các tham số đã tối ưu $\hat{\theta}$ và $\hat{\omega}$, bộ mã hóa (tức $q_{\hat{\theta}}(\mathbf{h}|\mathbf{x})$) bị loại bỏ, và bộ giải mã (tức $p_{\hat{\omega}}(\mathbf{x}|\mathbf{h})$) làm việc với các $\mathbf{h}$ được sinh ngẫu nhiên. Chính xác hơn, ta lấy mẫu một biến ẩn $\mathbf{h}_{\text{new}}$ từ một phân phối Gauss, rồi suy ra một mẫu $\mathbf{x}_{\text{new}}$ bằng $p_{\hat{\omega}}(\mathbf{x}|\mathbf{h}_{\text{new}})$ như thường lệ. Trong các chương sau, chúng ta sẽ thấy nhiều bài toán NLP có thể được xếp vào loại bài toán sinh, trong đó các đối tượng dữ liệu dạng chuỗi hoặc dạng phân cấp được sinh ra với điều kiện là một số đối tượng dữ liệu hoặc biến ẩn cho trước.

2.7 Tóm tắt

Trong chương này, chúng ta đã bàn về mạng nơ-ron là gì, cũng như một vài kiến trúc cơ bản thường được dùng làm khối xây dựng để phát triển các hệ thống học sâu mạnh mẽ. Ngoài ra, chúng ta đã xem xét cách huấn luyện mạng nơ-ron, cách chính quy hóa quá trình huấn luyện, và cách áp dụng mạng nơ-ron vào việc học đặc trưng theo cách không giám sát.

Tuy nhiên, mạng nơ-ron và học sâu là những chủ đề rất rộng, và những gì chúng ta thảo luận mới chỉ chạm tới bề mặt. Để có một phần giới thiệu toàn diện hơn, cuốn sách của Goodfellow và cộng sự [2016] là một tài liệu tuyệt vời. Cuốn sách này cũng đề cập một số kỹ thuật nâng cao, như các mô hình có cấu trúc sâu và các phương pháp ngẫu nhiên hóa, để phát triển các hệ thống tiên tiến nhất. Như mọi khi, có một khoảng cách đáng kể giữa việc hiểu một kỹ thuật về mặt lý thuyết và việc thành thạo áp dụng nó để giải quyết các bài toán thực tế. Vì vậy, với những người làm thực tế muốn áp dụng mạng nơ-ron và học sâu trong nhiều tình huống khác nhau, có nhiều cuốn sách trình bày chi tiết việc cài đặt các hệ thống học sâu [Géron, 2019; Zhang và cộng sự, 2021a; Chollet, 2021], cùng với các dự án mã nguồn mở cung cấp mã nguồn tham khảo[16].

[16]Địa chỉ của một vài hướng dẫn trực tuyến phổ biến: `https://pytorch.org/tutorials`, `https://keras.io/examples/nlp`, và `https://www.tensorflow.org/tutorials`

Trong các chương tiếp theo, chúng ta sẽ tập trung vào cách dùng các mô hình nơ-ron để giải quyết các bài toán NLP. Trên hành trình đó, chúng ta sẽ tìm hiểu cách học biểu diễn của từ và câu bằng các phương pháp đã bàn tới đây (Chương 3–4), và cách mô hình hóa nhiều bài toán NLP khác nhau bằng một số phương pháp nâng cao dựa trên mạng nơ-ron. Những phương pháp này bao gồm cơ chế chú ý và Transformer (Chương 5–6), tiền huấn luyện (Chương 7), và các mô hình ngôn ngữ lớn (Chương 8–11).

Bản dịch tiếng Việt có chú giải (không chính thức, phi thương mại) của *Natural Language Processing: Neural Networks and Large Language Models*, © 2021–2026 Tong Xiao và Jingbo Zhu, `https://github.com/NiuTrans/NLPBook`. Nguyên tác và bản dịch theo giấy phép CC BY-NC 4.0.